LUMS

CENTER FOR SPEECH AND LANGUAGE TECHNOLOGIES

CS535/EE514

Machine Learning

Fall 2023

Agha Ali Raza

Sequence Models

Agha Ali Raza

Reading List suggested by me

1. MIT 6.S191: Recurrent Neural Networks, Transformers, and Attention
2. Sequence Models Complete Course by Andrew NG
3. Stat Quest by Josh Starmer
    1. Word Embedding and Word2Vec, Clearly Explained!!!
    2. Recurrent Neural Networks (RNNs), Clearly Explained!!!
    3. Long Short-Term Memory (LSTM), Clearly Explained
    4. Sequence-to-Sequence (seq2seq) Encoder-Decoder Neural Networks, Clearly Explained!!!
    5. Attention for Neural Networks, Clearly Explained!!!
    6. Transformer Neural Networks, ChatGPT's foundation, Clearly Explained!!!
    7. Decoder-Only Transformers, ChatGPTs specific Transformer, Clearly Explained!!!
4. Visualizing A Neural Machine Translation Model (Mechanics of Seq2seq Models With Attention) by Jay Alammar
5. The Illustrated Transformer by Jay Alammar
6. Drawing the Transformer Network from Scratch (Part 1)
7. CS4780 Transformers (additional lecture 2023) by Kilian Weinberger
8. Speech and Language Processing, 3rd edition, by Jurafsky and Martin
    1. Vector Semantics and Embeddings
    2. Neural Networks and Neural Language Models
    3. RNNs and LSTMs
    4. Transformers and Pretrained Language Models
9. Attention is all you need

Readings suggested by the TAs

- Andrej Karpathy's "The Unreasonable Effectiveness of RNNs"
- Ava Amini, "Recurrent Neural Networks, Transformers, and Attention"
- CS231n 2018, Recurrent Neural Networks
- D2L Chapter 9 - Recurrent Neural Networks
- D2L Chapter 10 - Modern Recurrent Neural Networks
- Chris Olah's "Understanding LSTM Networks"
- Jay Alammar's "Visualizing a Neural Machine Translation Model (Mechanics of seq2seq Models with Attention)"

Mission 1: Using Neural Networks with Language

- Neural networks work with numbers and vectors
  - No inherent capabilities to understand characters and symbols

How to represent characters/words as vectors?

Representing Documents and Words as Vectors

Document to Vector: Bag of Words

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!

... ...

Bag of Words

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!

...

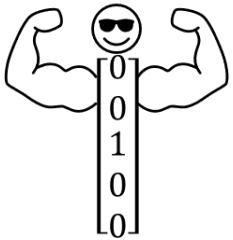
...

Bag of Words

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!

Types	Token Count
it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
…	…

One Hot Vectors



Vectors with a length the size of the defined vocabulary, and a single 1 to indicate the position/index of each entity in the vocabulary.

- One-hot and one-cold vectors
- Size: $V \times 1$, with only one 1

e.g. Looking at a dictionary of 10,000 words

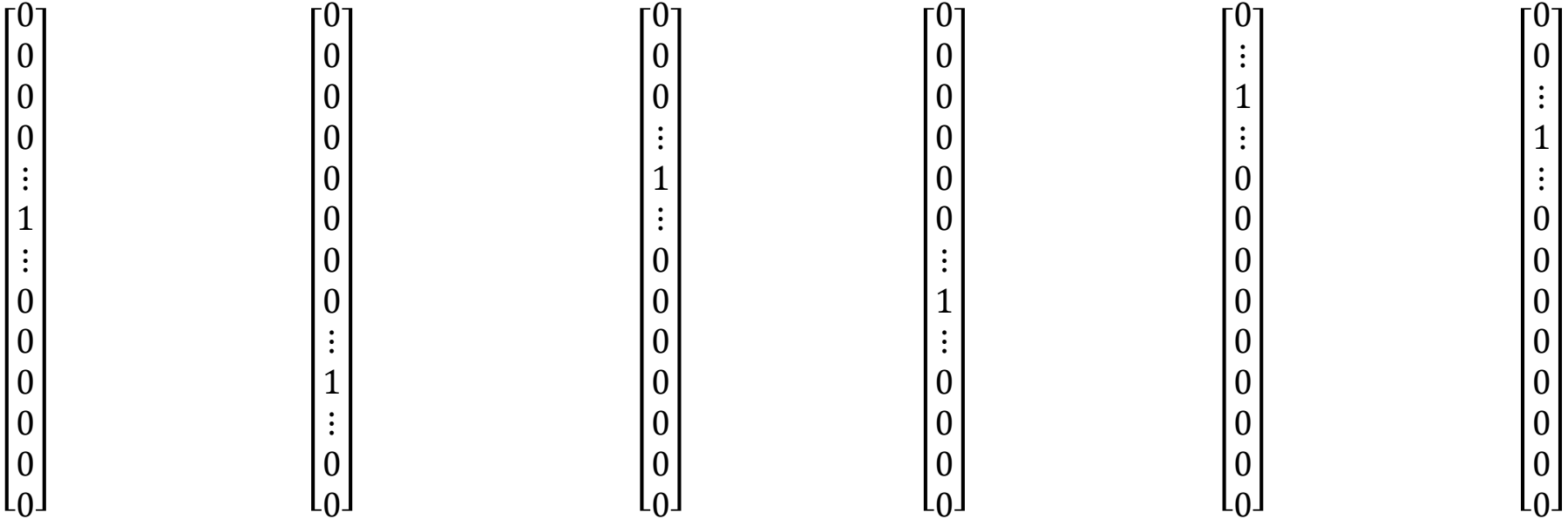
- V = [a, aaron, aardvark, …, man, …, woman, …, zoo, zulu, <UNK>]

This means

- the vector for “a” has a 1 at position 1
- the vector for “aaron” has a 1 at position 2
- the vector for “man” has a 1 at position 5391
- the vector for “zulu” has a 1 at position 9999

One-Hot Vectors

man (5391) woman (9853) king (4914) queen (7157) apple (456) orange (6257)



Problem:

I want a glass of orange _____

I want a glass of apple _____

- We have only seen “orange juice” in the training data, not “apple juice”

How do we capture semantics?

- One Hot Vectors are ***meaning***-less
  - The dot product between “apple” and “banana” is the same as that between “apple” and “airplane”. Does this mean an apple is as different from an airplane as it is from a banana?

- A new idea:
  - Embed meaning in these vectors - the meaning of a word will now depend on its position in a multidimensional vector space.
  - These new vectors for representing words are called **embeddings**.
  - One Hot Vectors can be called *sparse* representations, and these new embeddings are *dense* representations.

Vector Semantics

- The standard way to represent word meaning in NLP
  - Joos (1950), Harris (1954), and Firth (1957):
  - Two words that occur in *very similar distributions* (whose neighboring words are similar) have *similar meanings*.
    - Zellig Harris (1954):
    - If A and B have almost identical environments, we say that they are *synonyms*.

Vector Semantics

- Suppose we see these sentences with the word Ongchoi – recently borrowed from Cantonese:
  - Ongchoi is delicious sautéed with garlic.
  - Ongchoi is superb over rice
  - … ongchoi leaves with salty sauces
- And you've also seen these:
  - … spinach sautéed with garlic over rice
  - Chard stems and leaves are delicious
  - Collard greens and other salty leafy greens
- Conclusion:
  - Ongchoi is a leafy green like spinach, chard, or collard greens
  - We could conclude this based on words like “rice”, “garlic”, “salty”, "leaves" and "delicious" and "sauteed"
  - We can do this computationally by just counting words in the context of ongchoi

Ongchoi: Ipomoea aquatica "Water Spinach"
(Yamaguchi, Wikimedia Commons, public domain)

Representing words as vectors

- Now equipped with our new ideas:
  - Idea 1: Defining meaning by linguistic distribution
  - Idea 2: Meaning as a point in multidimensional space
- We can do better:
  - Define meaning as a point in space based on distribution
  - These new vectors for representing words are called embeddings
  - The word “embedding” derives from its mathematical sense as a mapping from one space or structure to another – although the meaning has shifted

Representations with Meanings

- Embeddings have a fixed length: they do not depend on the size of the vocabulary. Assume these vectors are 300-dimensional for now.
- “Similar words should have similar context word-vectors”
- Similarities and differences between entities in certain features:

	man (5391)	woman (9853)	king (4914)	queen (7157)	apple (456)	orange (6257)
Gender	-1	1	-0.95	0.97	0.00	0.01
Royal	0.01	0.02	0.93	0.95	-0.01	0.00
Age	0.03	0.02	0.70	0.69	0.03	-0.02
Food	0.04	0.01	0.02	0.01	0.95	0.97
…	…	…	…	…	…	…

Representations with Meanings

- Man and woman, king and queen, apple and orange are very similar
- These are our new vector representations of these words

Now we can approach our open problems:

- I want a glass of orange _____
- I want a glass of apple _____

	man (5391)	woman (9853)	king (4914)	queen (7157)	apple (456)	orange (6257)
Gender	-1	1	-0.95	0.97	0.00	0.01
Royal	0.01	0.02	0.93	0.95	-0.01	0.00
Age	0.03	0.02	0.70	0.69	0.03	-0.02
Food	0.04	0.01	0.02	0.01	0.95	0.97
...	...	...	...	...	...	...

Representing words as vectors

- Similar words are "nearby in semantic space"
- We build this space automatically by seeing which words are nearby in text

Embeddings vs. One Hot Vectors

Pros:

- Carry meaning - the semantics are baked in to the vector representation.
- Have a fixed length unlike One Hot Vectors whose size depends on that of the vocabulary. Can be more efficient for space.
- Allow for meaningful computations such as finding the similarity between words, sentences, or documents, using measures like the cosine similarity.
- The dimensionality can be arbitrarily scaled to incorporate more meaningful features: e.g. GPT-3 has 1536 dimensions for each Embedding.

Cons:

- Often have to be learned - better representations like word2vec and GLoVE can take a lot of resources to produce decent embeddings.

Source: From Encodings to Embeddings, Medium - https://towardsdatascience.com/from-encodings-to-embeddings-5b59bceef094

Benefits

- Consider sentiment analysis:
- With **words** and **one-hot vectors**, the feature is the word identity:
  - Feature 5: 'The previous word was "terrible"'
  - requires the exact same word to be in training and test
- With **embeddings**:
  - The feature is a word vector
  - The previous word was the vector [35,22,17…]
  - Now in the test set we might see a similar vector [34,21,14]
  - We can generalize to similar but unseen words!!!

Representing words as word vectors

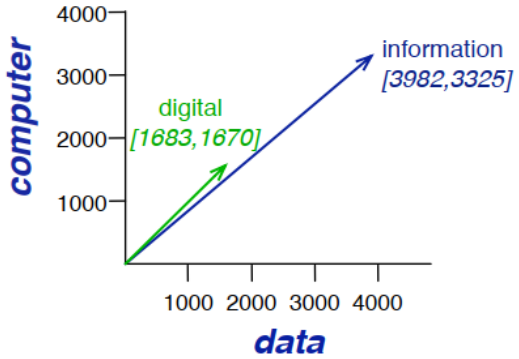
- The term-term matrix, also called the word-word matrix or the term-context matrix
  - Similar words have similar context word-vectors
  - The columns are labeled by context words.
  - |V|×|V| dimensions

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

  - Each cell records the number of times the **row (target) word** and the **column (context) word** co-occur in some context in some training corpus
  - The context could be:
    - A document, in which case the cell represents the number of times the two words appear in the same document
    - A window, e.g., ± 4 words, in which case the cell represents the number of times (in some training corpus) the column word occurs within a window of 4 words to the left or right of the row word.

Visualizing the Co-occurence

- *cherry* and *strawberry* are more similar to each other than they are to other words like digital
  - both *pie* and *sugar* tend to occur in their window
- *digital* and *information* are more similar to each other than to strawberry

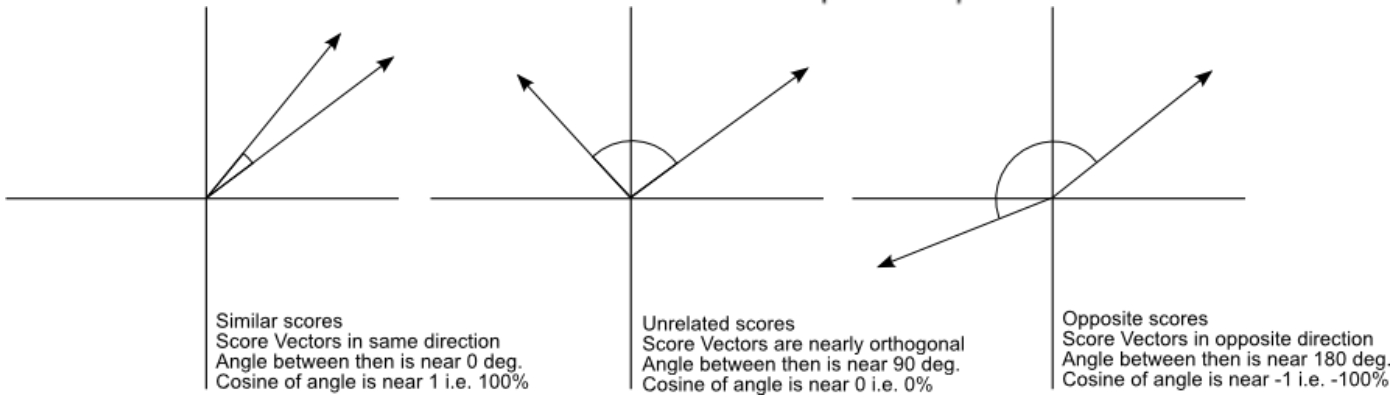


	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

Similar Vectors: Cosine Similarity

- Cosine similarity measures the cosine of the angle between two vectors.
  - For word embeddings, the vectors represent the semantic meaning of words in a high-dimensional space.
  - Cosine similarity (**scaled dot product**) ranges from -1 to 1, with 1 indicating identical vectors, 0 indicating no similarity, and -1 indicating opposite vectors.

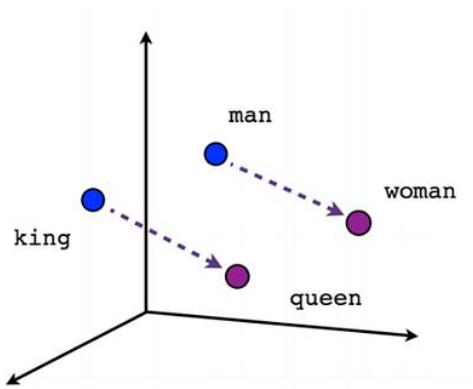
$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}},$$



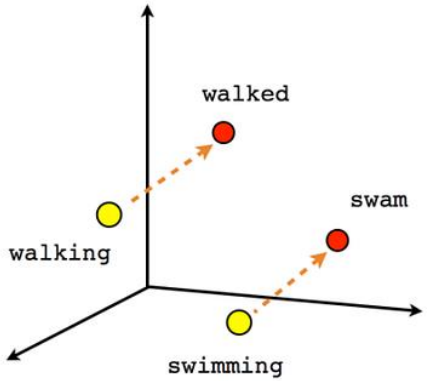
Cosine similarity between two arrays for word embeddings, Word Vectors-Cosine Similarity

Visualizing Embeddings

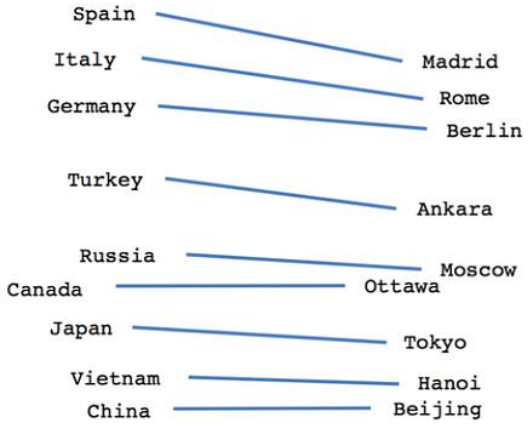
Now we will try to find a word that associates the other three words according to their analogies. For example;
(a, b) → (c, _) we'll try to find 'd'



Male-Female



Verb tense



Country-Capital

Source: Mapping Word Embeddings with word2vec - https://towardsdatascience.com/mapping-word-embeddings-with-word2vec-99a799dc9695

Types of Word Embeddings

Summary

- Word embeddings are a type of word representation that allows words to be represented as vectors in a continuous vector space.
- The position of each word within the vector space is learned from text and is based on the words that surround the target word when it is used.
- The idea is that words that are used in similar contexts will have similar meanings and thus will be close to each other in the vector space.
- Types:
  - Frequency-based embeddings
  - Prediction-based Static Embeddings
  - Pre-trained Language Model-based Dynamic Embeddings

Frequency-based Embeddings

- **Count Vectors**:
  - Count vectors represent the frequency of a word's occurrence in a document.
  - This is a basic form of word representation and doesn't capture semantic similarities.
- **TF-IDF (Term Frequency-Inverse Document Frequency)**:
  - TF-IDF is an improvement over count vectors.
  - It reflects the importance of a word in a document relative to a corpus.
  - Unlike count vectors, it takes into account not just the occurrence but the significance of the word in the document.
- **Co-Occurrence Matrix**:
  - This captures the co-occurrence frequencies of words in a specified context, which helps in preserving semantic relationships.

Count Vectors

1. **Vocabulary Construction**:
   - First, a vocabulary is constructed by listing all the unique words across all documents in the corpus. Each unique word is assigned a unique index.
2. **Vectorization**:
   - For each document, a vector is created where each element/index in the vector corresponds to a word in the vocabulary.
   - The value at each index is the count of occurrences of the corresponding word in the document.

If we have a vocabulary of size V and a document d, the count vector $\boldsymbol{c_d}$ for document d is:

$$c_{d,i} = count(w_i, d)$$

where:

- $c_{d,i}$ is the element at index i in the count vector $\boldsymbol{c_d}$ for document d,
- w_i is the word in the vocabulary corresponding to index i,
- $count(w_i, d)$ is the number of occurrences of word w_i in document d.
- The resulting vector will have as many elements as there are unique words in the vocabulary, and each element value represents the count of that word in the document.

Term Frequency-Inverse Document Frequency (TF-IDF)

- TF-IDF, is a numerical statistic that reflects how important a word is to a document in a collection or corpus.
- It is often used in text mining and information retrieval systems to score the relevance of documents based on keyword queries.

1. **Term Frequency (TF)**:
   This is the number of times a word appears in a document. It's often normalized to prevent a bias towards longer documents (which may have higher term frequency regardless of the word's importance).

$$TF(t) = \frac{Number\ of\ times\ term\ t\ appears\ in\ a\ document}{Total\ number\ of\ terms\ (tokens)\ in\ the\ document}$$

2. **Inverse Document Frequency (IDF)**:
   This part measures the importance of the word in the corpus. If a word appears in many documents, it's not a unique identifier, therefore it has less importance. The formula for IDF is:

$$IDF(t) = \log_e\left(\frac{Total\ number\ of\ documents}{Number\ of\ documents\ with\ term\ t\ in\ it}\right)$$

$$\boldsymbol{TFIDF(t) = TF(t) \cdot IDF(t)}$$

- The TF-IDF score will be high for words that are frequent in a given document and rare across other documents in the corpus, thus helping to identify words that are particularly meaningful or characteristic of the given document.

Prediction-based Static Embeddings

These are referred to as static as the embeddings of a word do not change based on its context.

- **Word2Vec**:
  - Word2Vec uses neural networks to learn word embeddings by predicting the context given a word (Continuous Bag of Words - CBOW) or predicting a word given the context (Skip-gram).
  - It is capable of capturing semantic relationships and is computationally efficient compared to frequency-based methods.
- **FastText**:
  - FastText is an extension of Word2Vec that considers subword information, which enables it to handle out-of-vocabulary words and morphologically rich languages better than Word2Vec.
  - It represents each word as a bag of character n-grams.
- **Glove (Global Vectors for Word Representation)**:
  - GloVe combines aspects of co-occurrence matrix factorization and prediction-based methods to learn word embeddings.
  - It leverages both global statistical information and local context.

Downsides of Static Embeddings

- Static embeddings are limited in that they cannot capture the **polysemy** of words (i.e., words with multiple meanings will have the same vector representation regardless of the context in which they are used).
- This limitation has led to the development of context-dependent embeddings like those produced by **ELMo, BERT, and GPT,** which generate different vector representations for a word based on its contextual usage.

Pre-trained Language Model Dynamic Embeddings

- **ELMo (Embeddings from Language Models)**:
  - ELMo embeddings are learned from a language model trained on a large text corpus.
  - Unlike Word2Vec and GloVe, ELMo embeddings are context-dependent, meaning the embedding for a word can change based on the context in which it appears.
- **BERT (Bidirectional Encoder Representations from Transformers)**:
  - BERT also generates context-dependent embeddings and leverages a transformer architecture to consider words in both left and right contexts simultaneously, which is a significant improvement over unidirectional models (e.g. GPT).
- **GPT (Generative Pre-trained Transformer)**:
  - GPT, like BERT, uses a transformer architecture but is trained in a generative setting.
  - GPTs are unidirectional (left-to-right) and excels in tasks that require understanding of **long-range context**
  - While GPT models are unidirectional, they can still capture a form of bidirectional context through the use of certain techniques or in specific configurations, such as by processing a text sequence forwards and then backwards separately, and combining the results.

GPT

- GPT (Generative Pre-trained Transformer) is trained on a large corpus of text data in a generative setting, where it learns to predict the next word in a sequence given the preceding words.
- Through this training process, GPT learns rich representations of words and their relationships, effectively learning embeddings for words in the process.

1. **Language Model**:

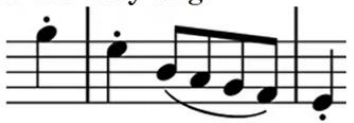
   GPT is designed as a language model. It's capable of generating text that follows the statistical properties of the training data.

   The primary task for GPT is to predict the next word in a sequence, which is a characteristic task for language models.

2. **Word Embeddings**:
   - While GPT is not primarily categorized as a word embedding model, during its training process, it learns to represent words as vectors in a high-dimensional space. These vectors, or embeddings, capture semantic relationships between words based on their co-occurrence patterns in the training data.
   - The embeddings learned by GPT are context-dependent, meaning that the embedding for a word can change based on the surrounding context, unlike traditional word embedding models like Word2Vec or GloVe which learn a single static embedding for each word.

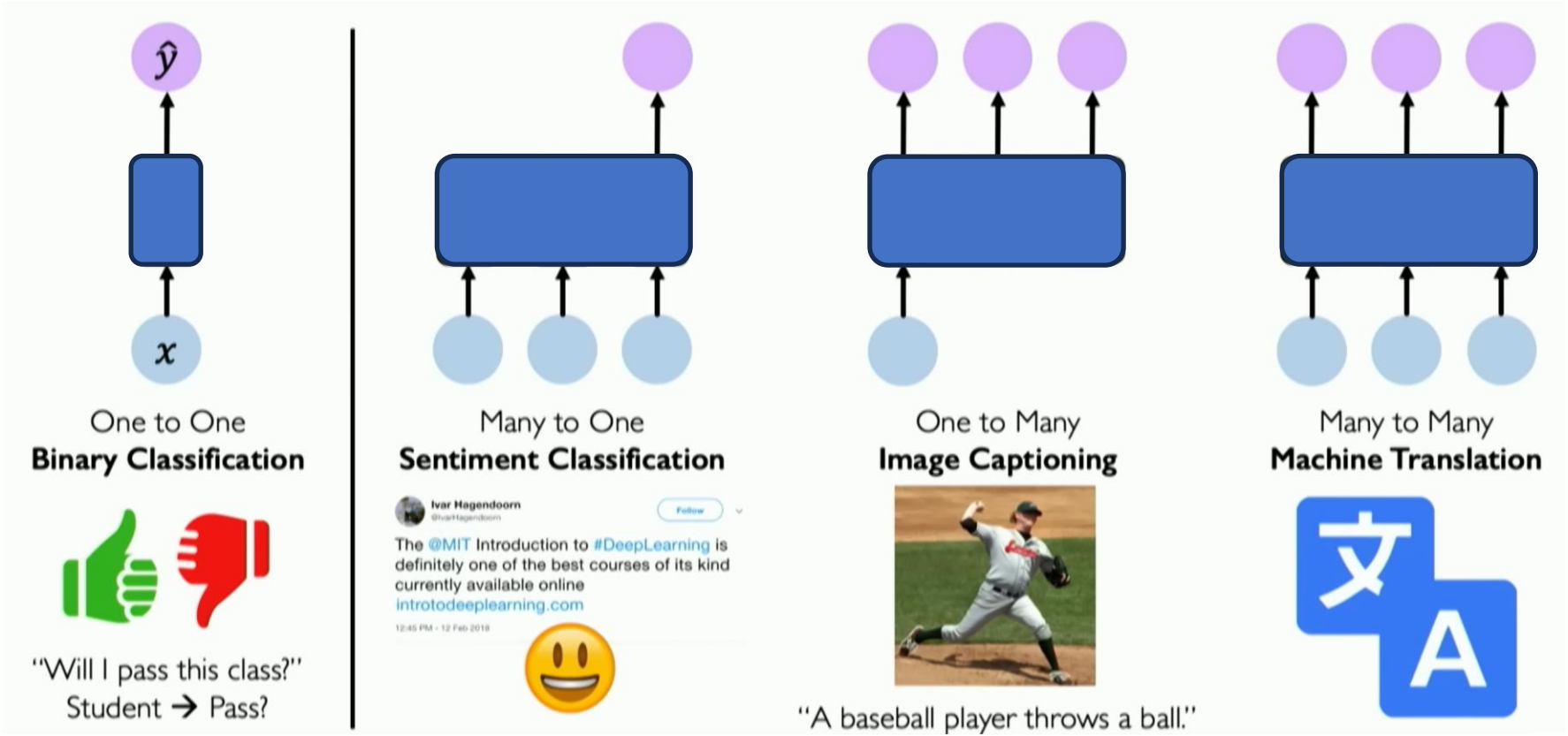
Processing Sequences with Context

Sequence Modeling Tasks

Speech recognition		→	"The quick brown fox jumped over the lazy dog."
Music generation	∅	→	
Sentiment classification	"There is nothing to like in this movie."	→	★☆☆☆☆
DNA sequence analysis	AGCCCCTGTGAGGAACTAG	→	AGCCCCTGTGAGGAACTAG
Machine translation	Voulez-vous chanter avec moi?	→	Do you want to sing with me?
Video activity recognition		→	Running
Name entity recognition	Yesterday, Harry Potter met Hermione Granger.	→	Yesterday, Harry Potter met Hermione Granger.

Source: DeepLearning.AI, *Sequence Models*, Coursera

Sequence Modelling Applications



Examples

- Image Classification:
  one to one
- Sentiment Classification:
  many to one
- Image Captioning:
  one to many
- Video Captioning:
  many to many
- Machine Translation:
  many to many
- Image Generation Models (like Stable Diffusion):
  many to one
- Language Models (like GPT-4):
  many to many

Language Models

Language Models have one objective:

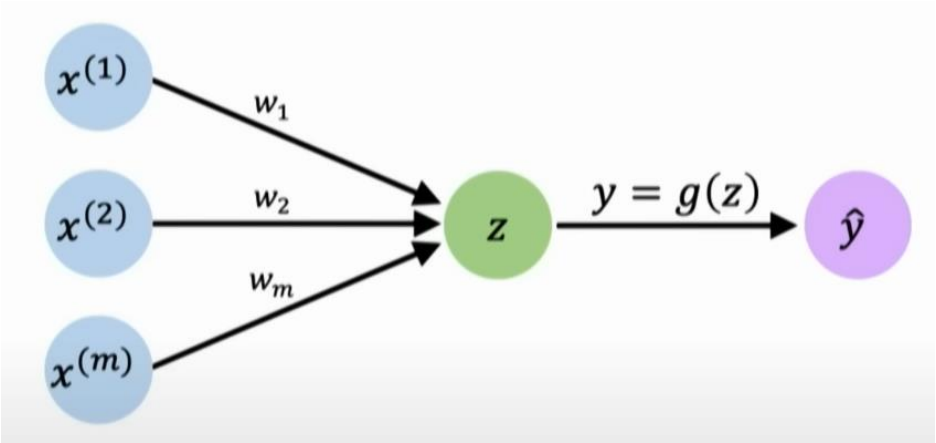
- "**Predict the next word** (given some context)"
- This is the same as predicting the **probability distribution** over a vocabulary, regards to which word comes next in the sequence.

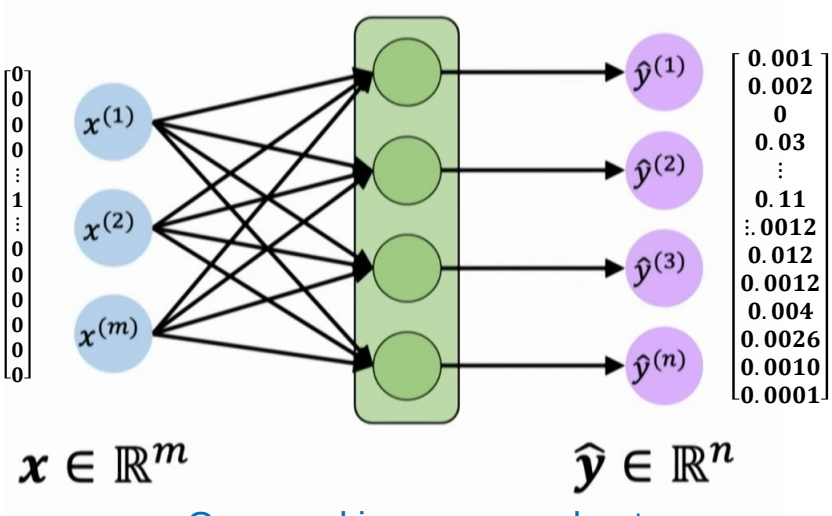
Simple Feed Forward Neural Networks are fixed-length input/output models

- This limits their ability to account for context
- NNs do not have ordered inputs (X is just a vector of n features)
- Language Modeling can still be framed as a generic classification task

Neural Language Models

- Language Models
  - Predict the next word
  - Find the probability of a sentence/sequence
- Simple Neural Networks are Fixed-Length input/output devices
  - This severely limits the ability to account for context

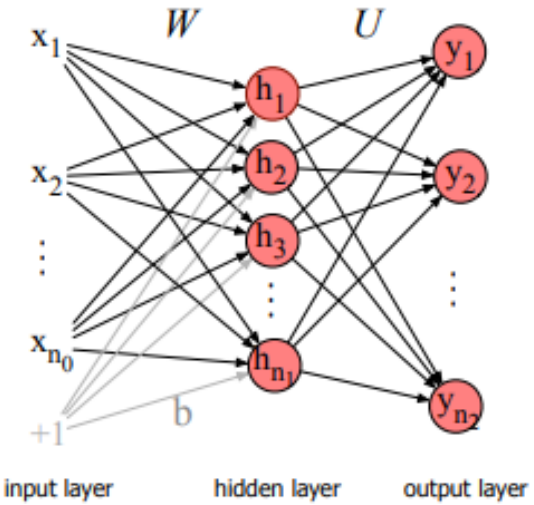




One word in – one word out

Neural Language Models

- Language Models
  - Predict the next word
  - Find the probability of a sentence/sequence
- Simple Neural Networks are Fixed-Length input/output devices
  - This severely limits the ability to account for context



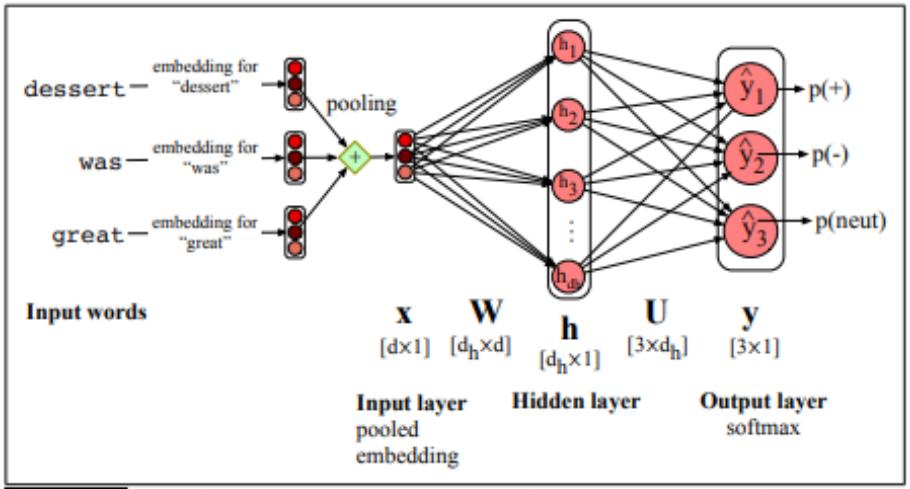


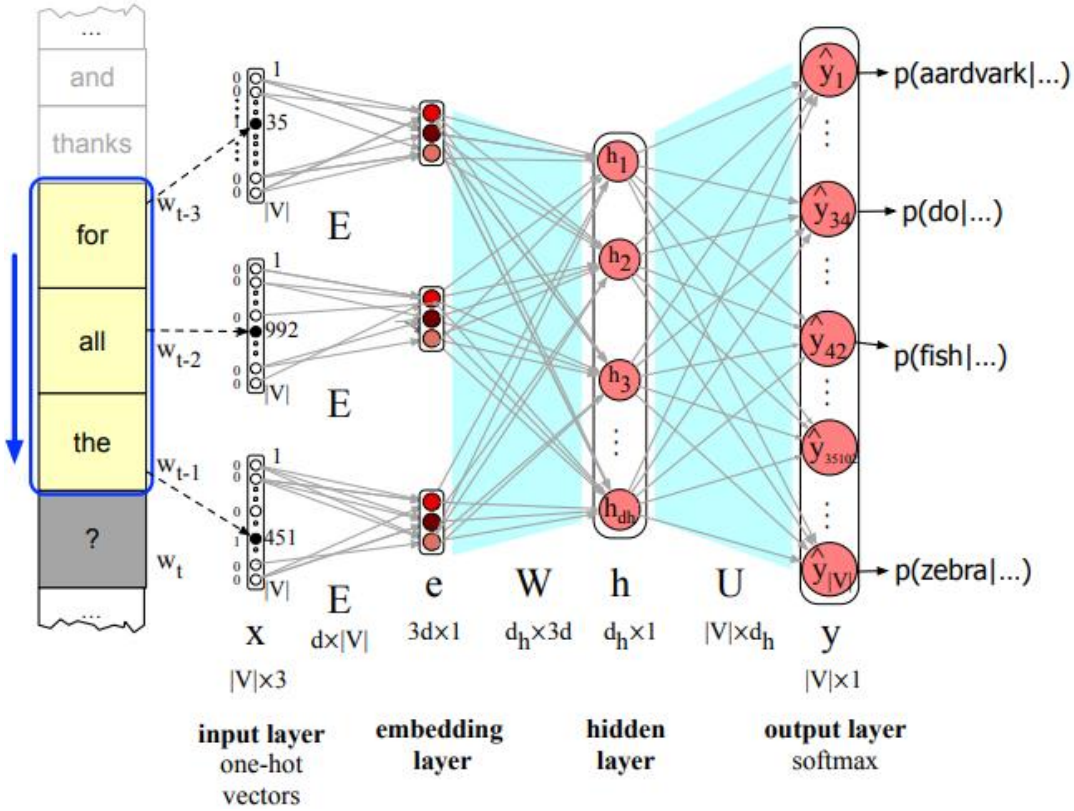
Figure 7.11 Feedforward network sentiment analysis using a pooled embedding of the input words.

Image Reference: SLP3, https://web.stanford.edu/~jurafsky/slp3/

Neural Language Models

Image Reference: SLP3, https://web.stanford.edu/~jurafsky/slp3/

- Predict the word w_t given the context.
  - Cannot deal with variable context length

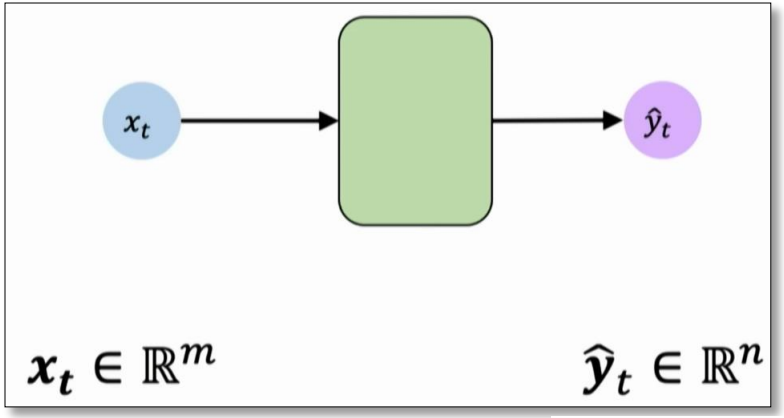


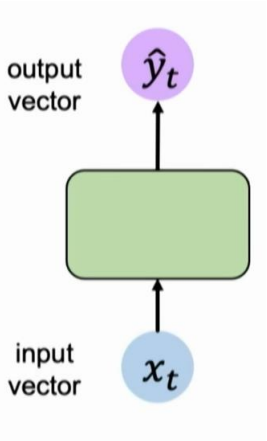
Problems

- Fixed length input/output models
- No explicit notion of word order (no temporal dimension)
  - Sentences are represented the same way in whatever order
  - “The food was good, not bad at all” vs. “The food was bad, not good at all”
- “Thinking from scratch” at every time step
  - Understanding of a sequence should evolve as we read more words
- Not sharing their parameters “across time”
- Not adaptive to the dynamic nature of meaning in sequences

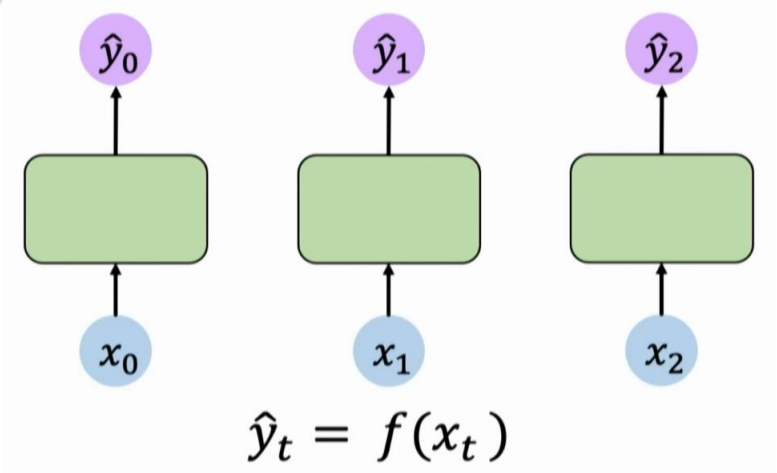
Source: https://www.simplilearn.com/tutorials/deep-learning-tutorial/rnn

Feedforward Networks with index (time)

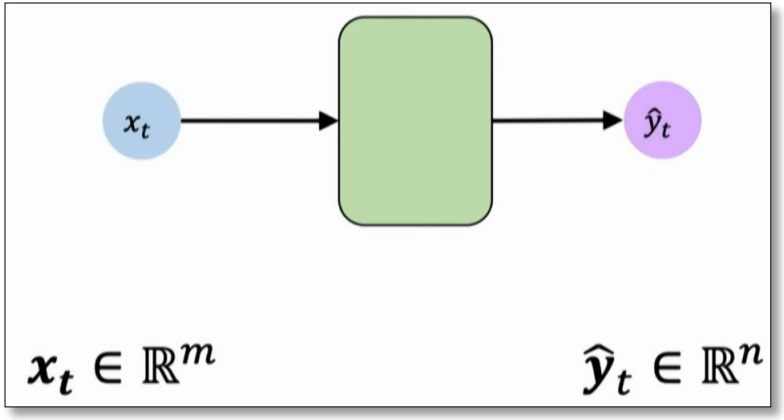


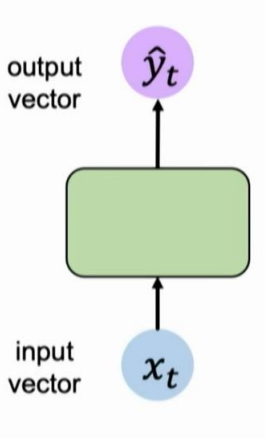


- It may be helpful to think of these are separate neural networks.
- Until we realize that they don't have to be separate ones,
- And we can just use the same recursively

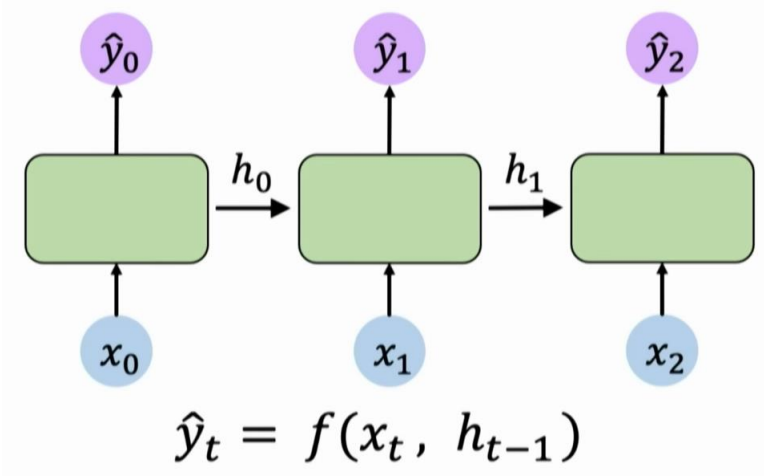


Feedforward Networks with index (time)

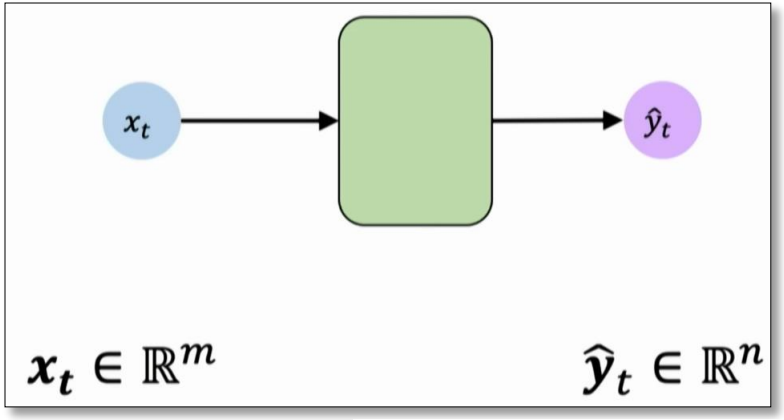


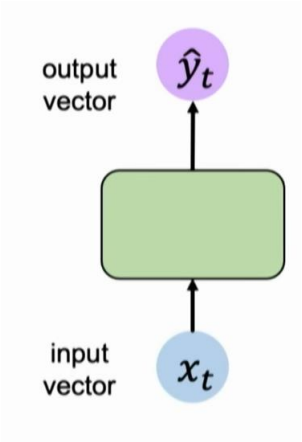


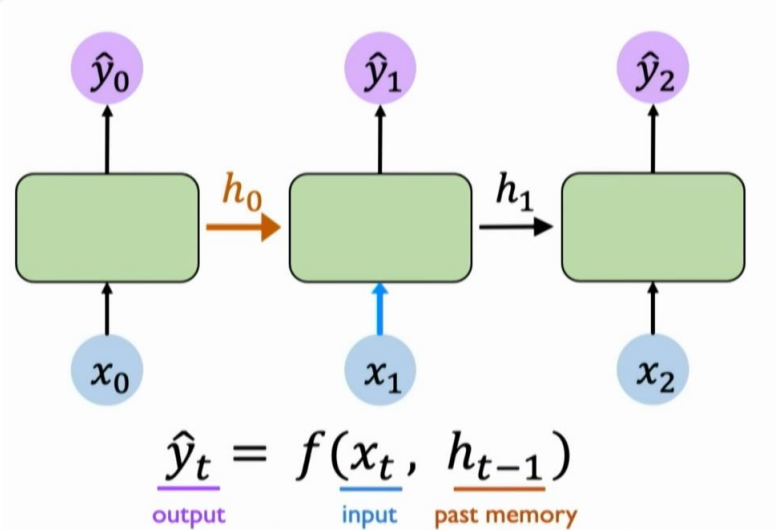
- It may be helpful to think of these are separate neural networks.
- Until we realize that they don't have to be separate ones,
- And we can just use the same recursively



Feedforward Networks with index (time)



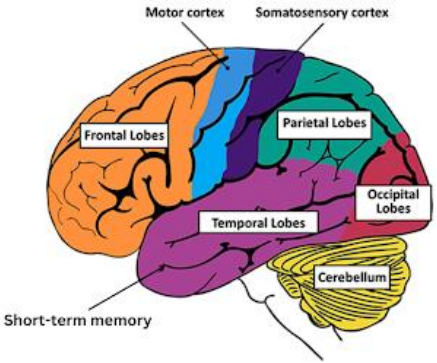




Recurrent Neural Networks

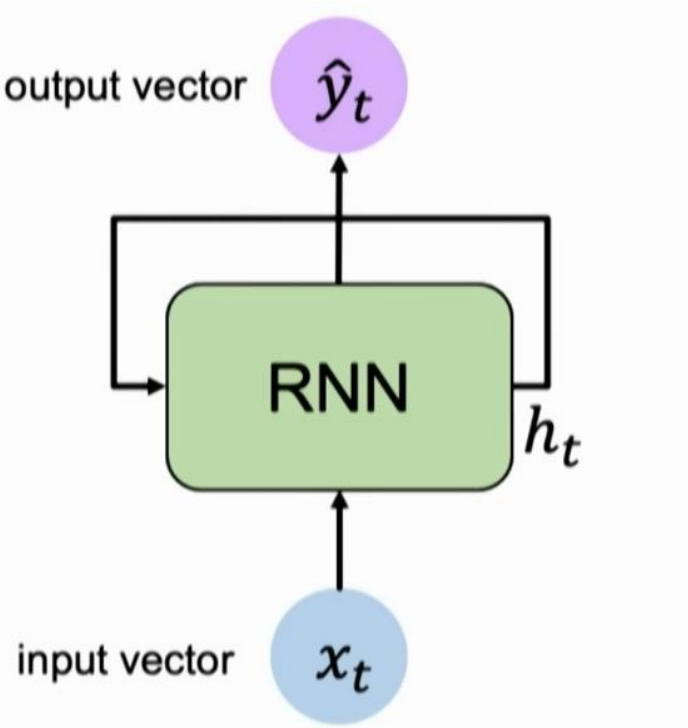
Intuition

- **Recurrent Neural Networks** are inspired by a part of the brain called the **Temporal Lobe** which resides in *short-term memory*.
- Short-term memory is the part of the brain that helps us to remember what happened around us **recently**.
  - It stores information temporarily.
- Contributes a lot to our ability to handle languages, understand patterns, and understand what is going around us recently.
- RNNs allow the network to capture short-term dependencies in the given dataset.
  - Useful for handling sequential temporary information that can be used for different tasks like Language Modelling, Speech Recognition, Machine Translation, and more.



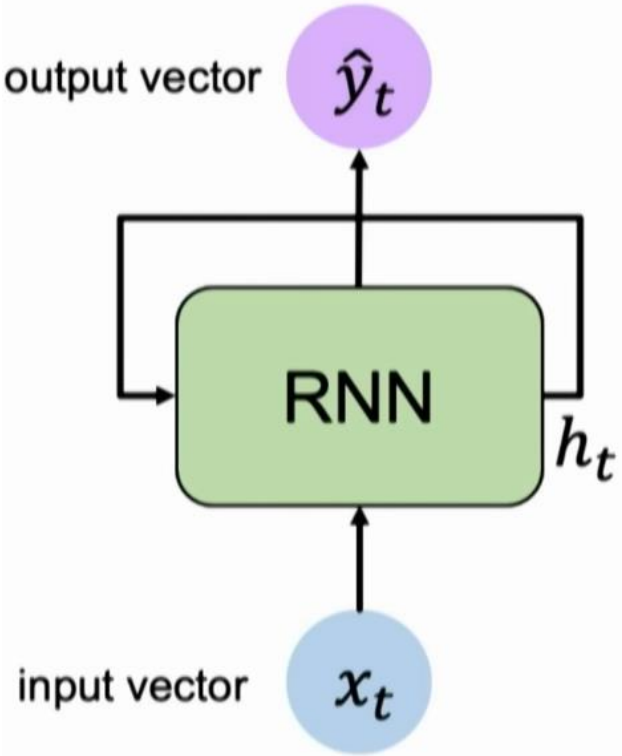
Recurrent Neural Networks: An Overview With Python Example

RNNs



RNNs have a **state**, h_t, that is updated **at each time step** as a sequence is processed

RNNs

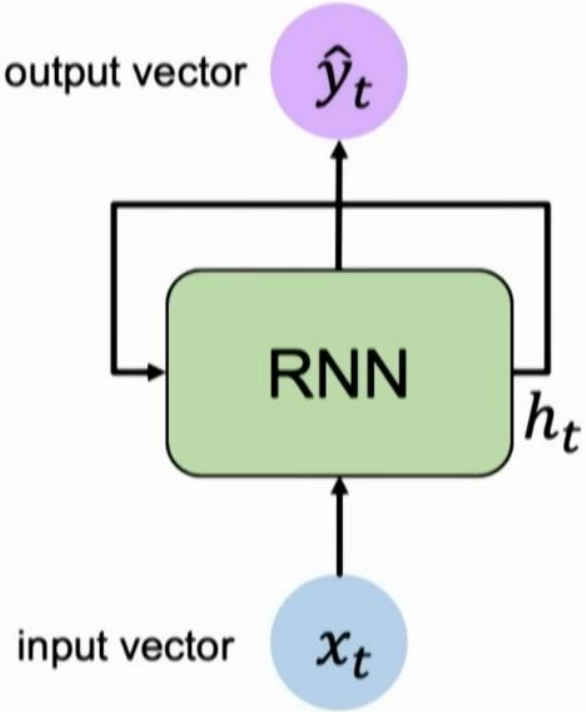


Apply a **recurrence relation** at every time step to process a sequence:

$$h_t = f_W(x_t, h_{t-1})$$

RNNs have a **state**, h_t, that is updated **at each time step** as a sequence is processed

RNNs



Apply a **recurrence relation** at every time step to process a sequence:

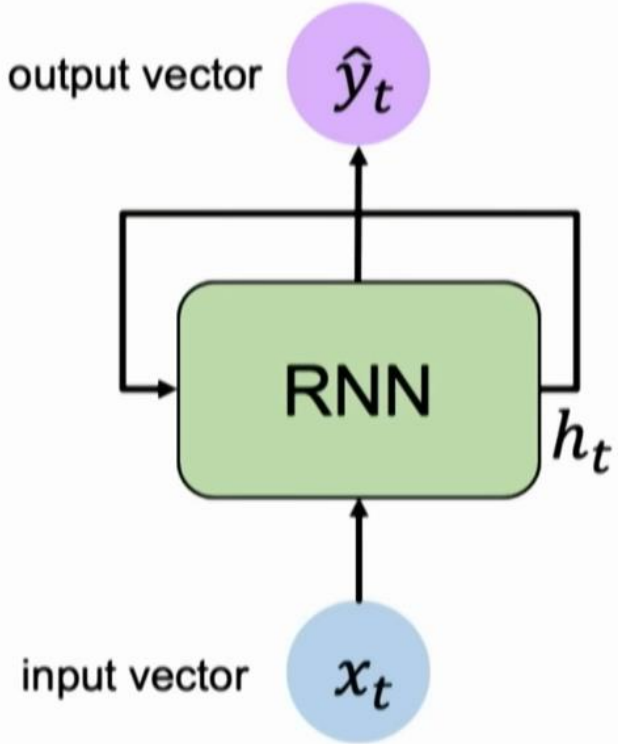
$$h_t = f_W(x_t, h_{t-1})$$

cell state | function with weights W | input | old state

Note: the same function and set of parameters are used at every time step

RNNs have a **state**, h_t, that is updated **at each time step** as a sequence is processed

Inputs and Outputs



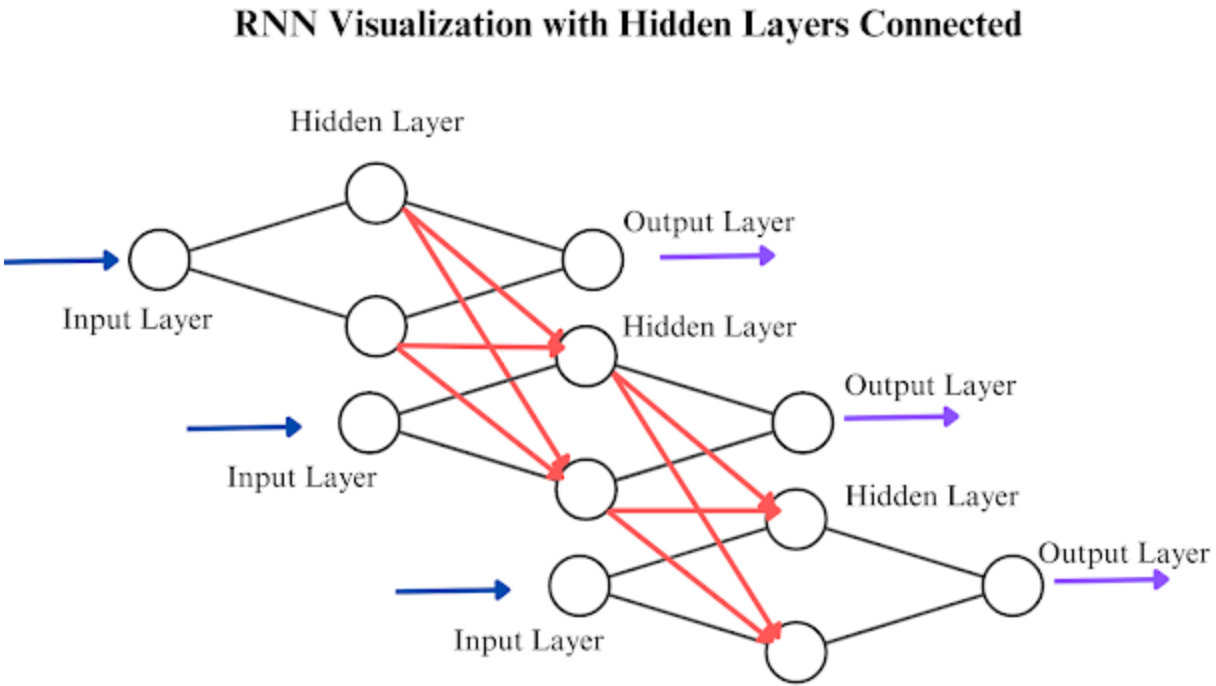
Update Hidden State

$$h_t = \tanh(\boldsymbol{W}_{\boldsymbol{hh}}^{\boldsymbol{T}} h_{t-1} + \boldsymbol{W}_{\boldsymbol{xh}}^{\boldsymbol{T}} x_t)$$

Input Vector

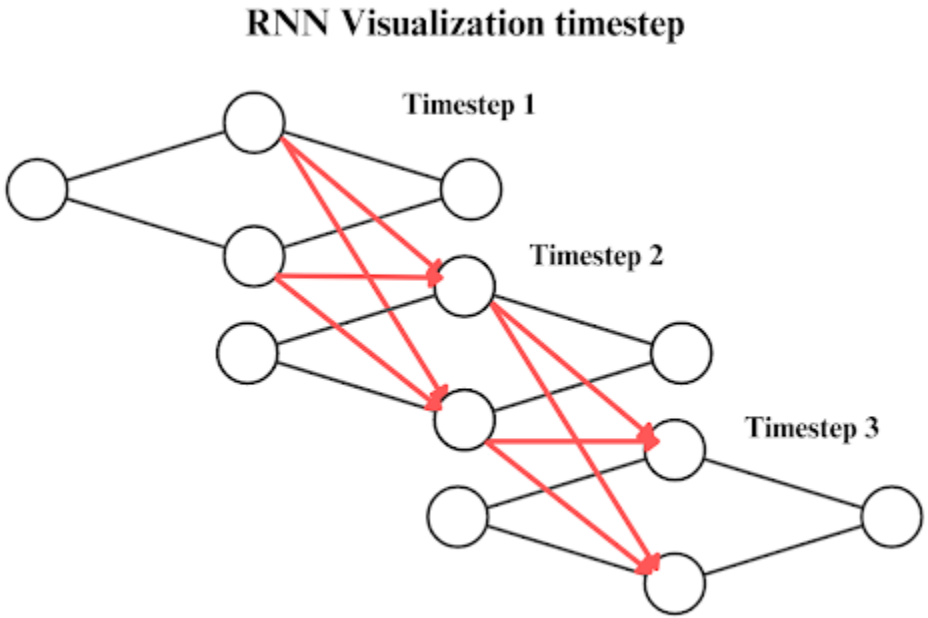
x_t

Peeking inside the Neurons



Recurrent Neural Networks: An Overview With Python Example

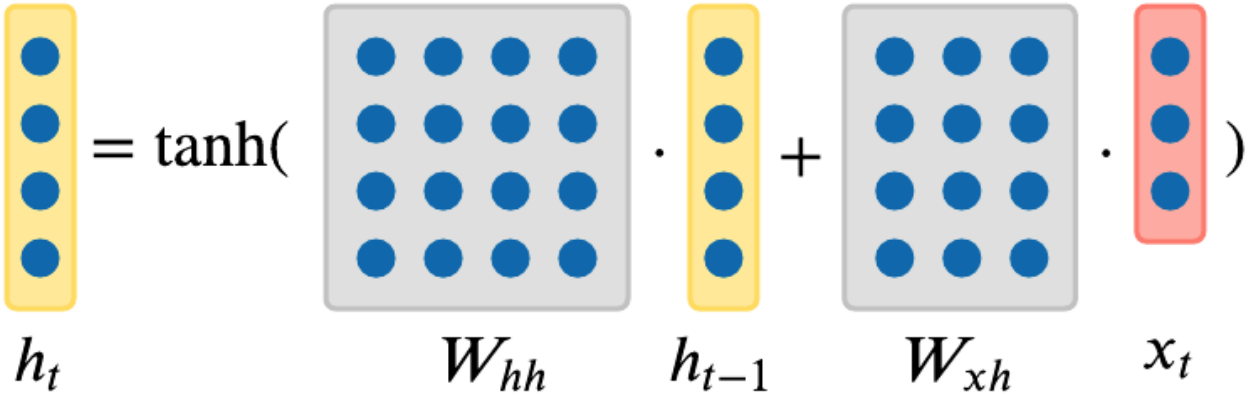
Peeking inside the Neurons



Recurrent Neural Networks: An Overview With Python Example

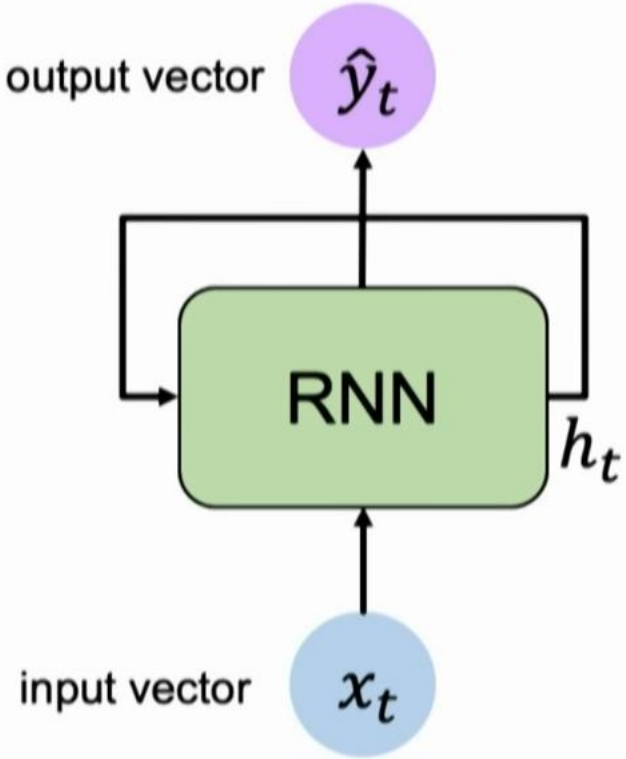
Update the Hidden State

Two sets of projections with learnable weights, and an activation function



Source: CS231n 2018, https://cs231n.github.io/rnn/

Inputs and Outputs



Output Vector

$$\hat{y}_t = \boldsymbol{W}_{hy}^{T} h_t$$

Update Hidden State

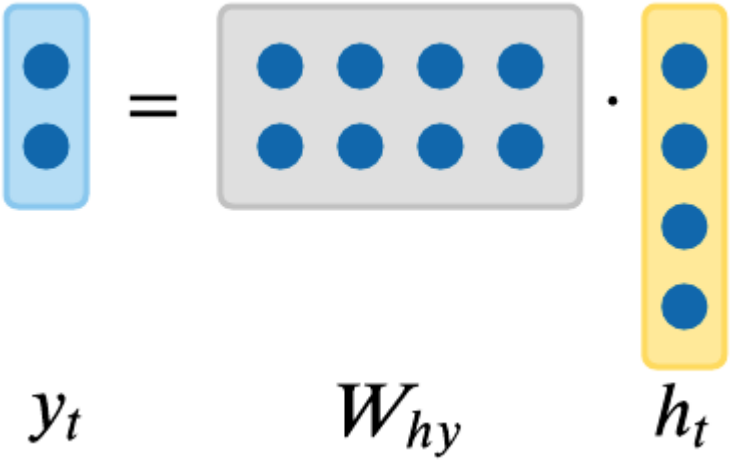
$$h_t = \tanh(\boldsymbol{W}_{hh}^{T} h_{t-1} + \boldsymbol{W}_{xh}^{T} x_t)$$

Input Vector

$$x_t$$

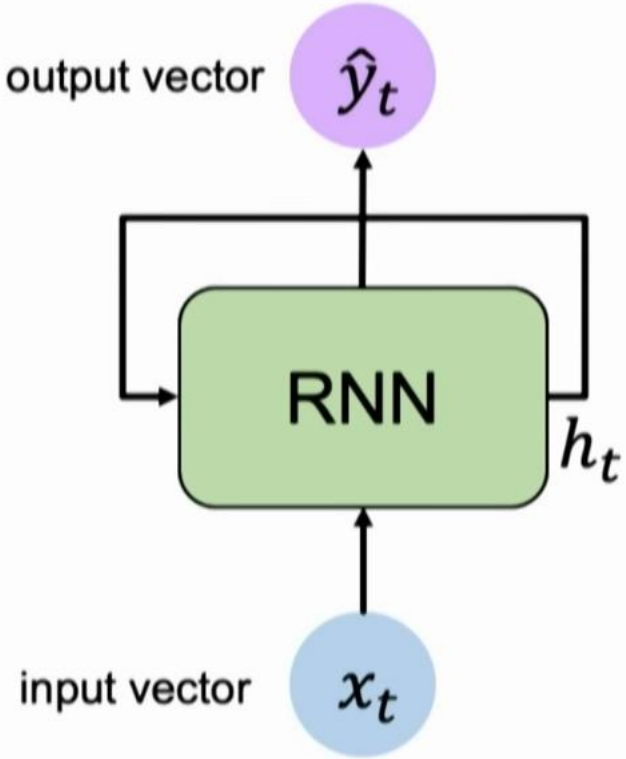
Making Predictions

Use the Hidden State and another set of weights to make predictions



Source: CS231n 2018, https://cs231n.github.io/rnn/

Inputs and Outputs



Output Vector

$$\hat{y}_t = \boldsymbol{W}_{\boldsymbol{hy}}^{\boldsymbol{T}} h_t$$

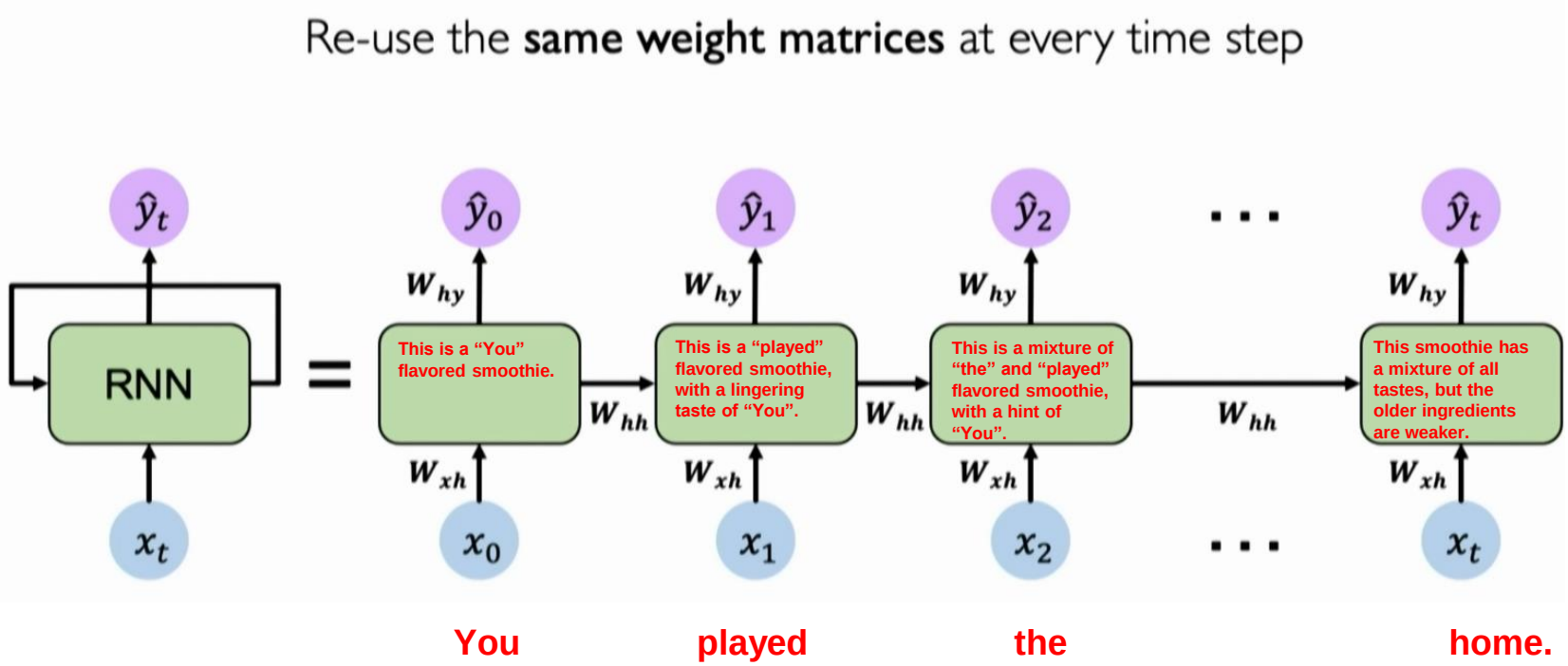
Update Hidden State

$$h_t = \tanh(\boldsymbol{W}_{\boldsymbol{hh}}^{\boldsymbol{T}} h_{t-1} + \boldsymbol{W}_{\boldsymbol{xh}}^{\boldsymbol{T}} x_t)$$

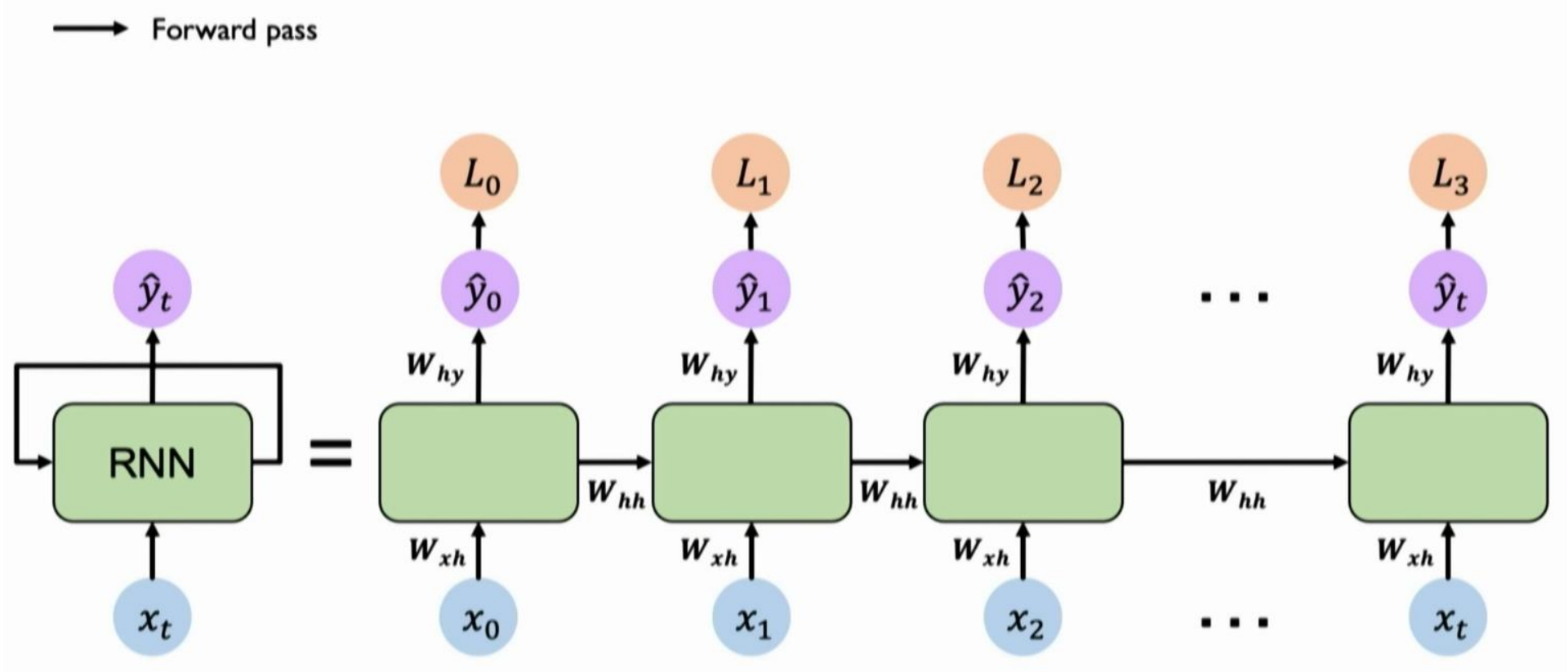
Input Vector

$$x_t$$

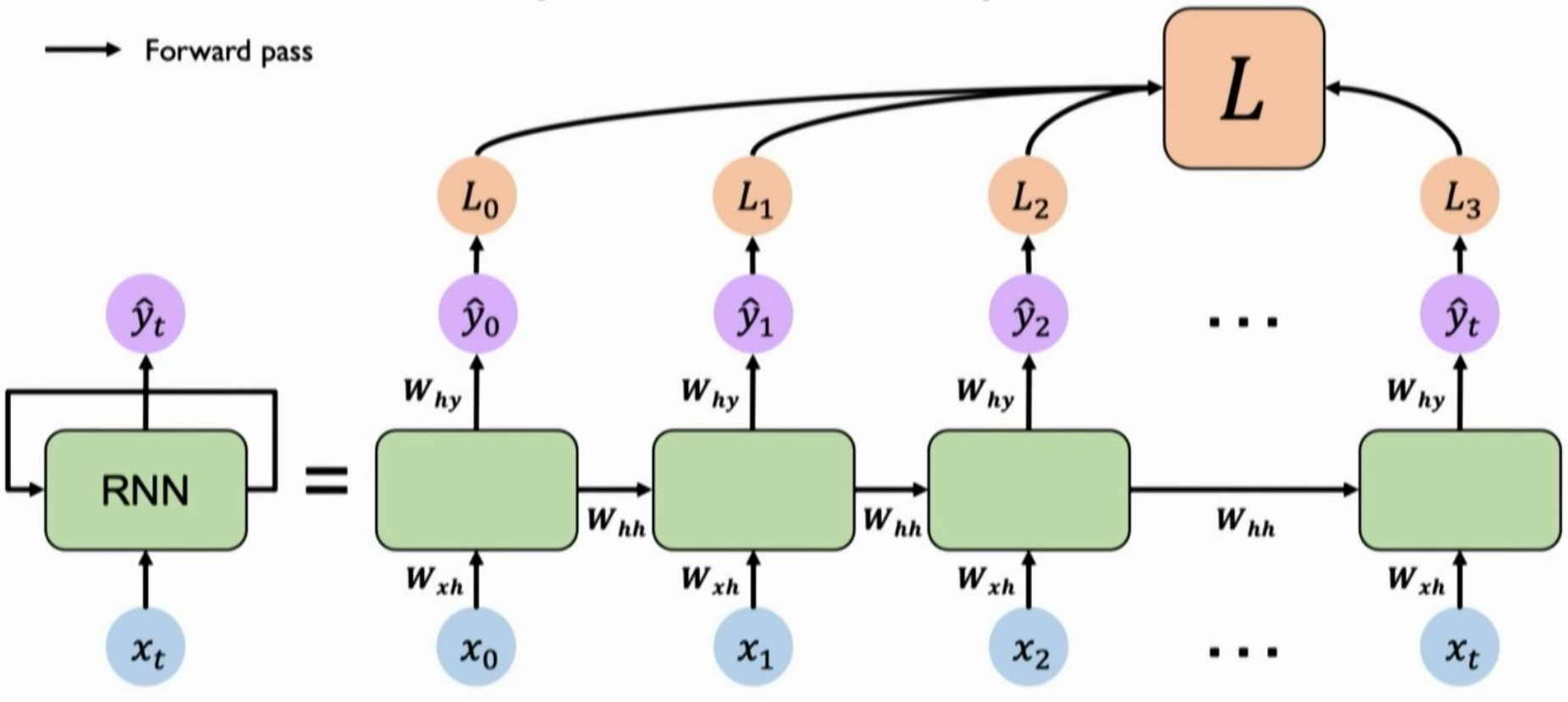
Computational Graph across Time



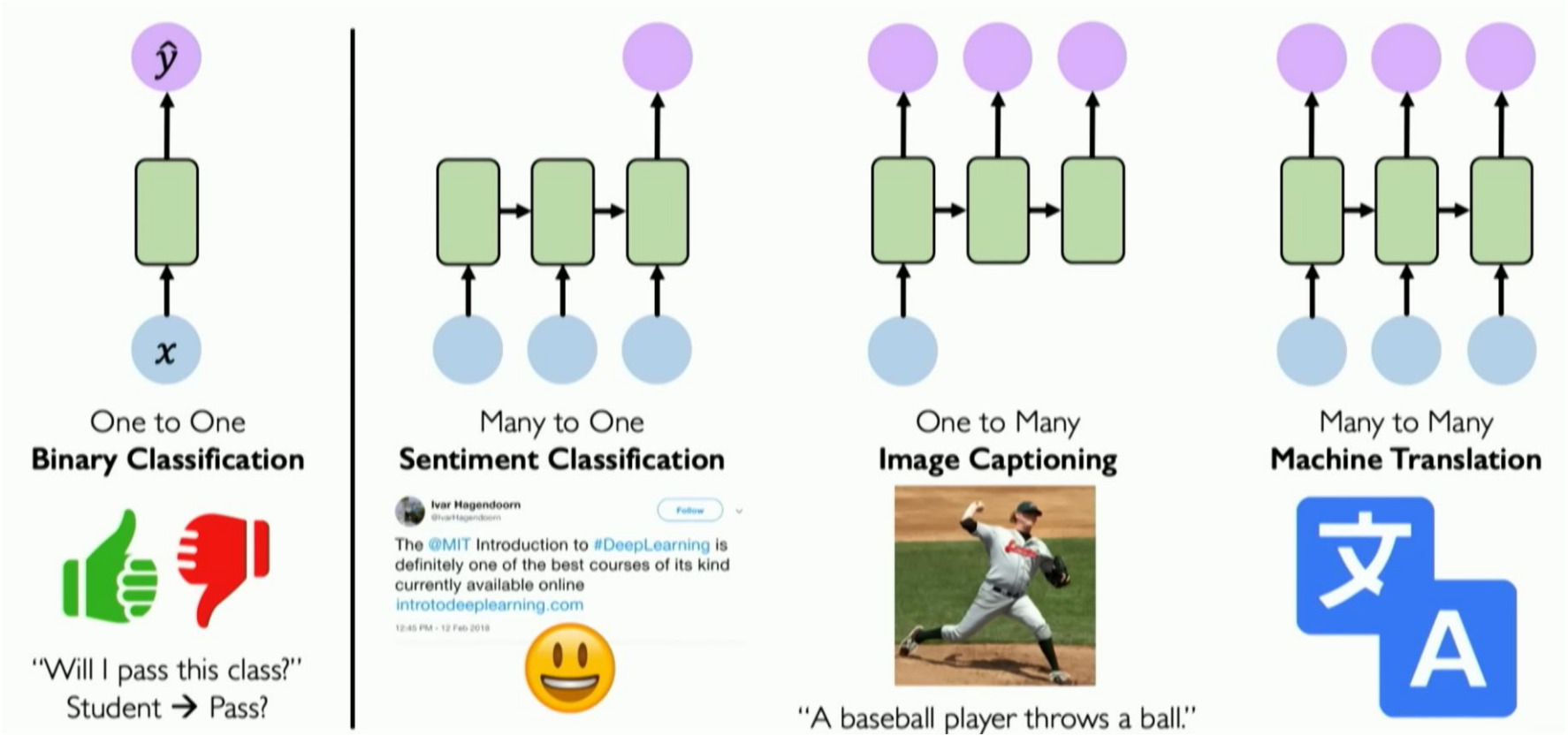
Forward Pass



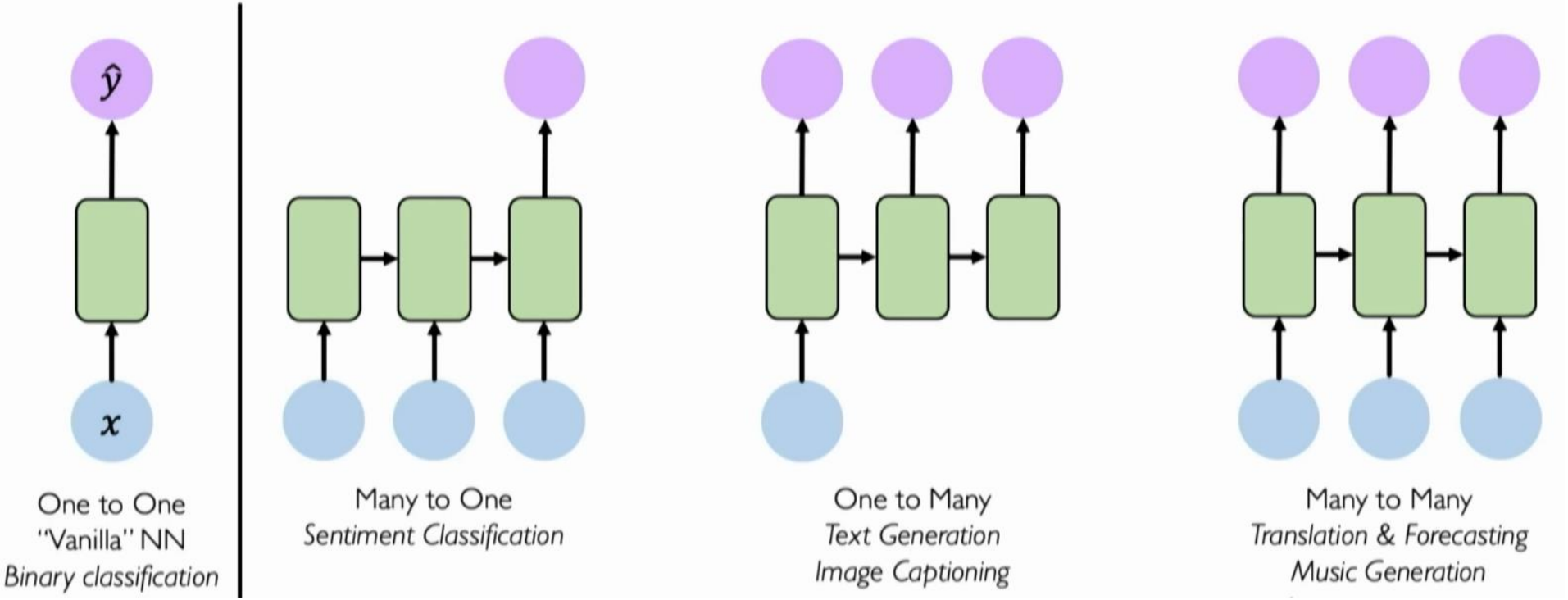
Forward Pass and Loss



Sequence Modelling Applications



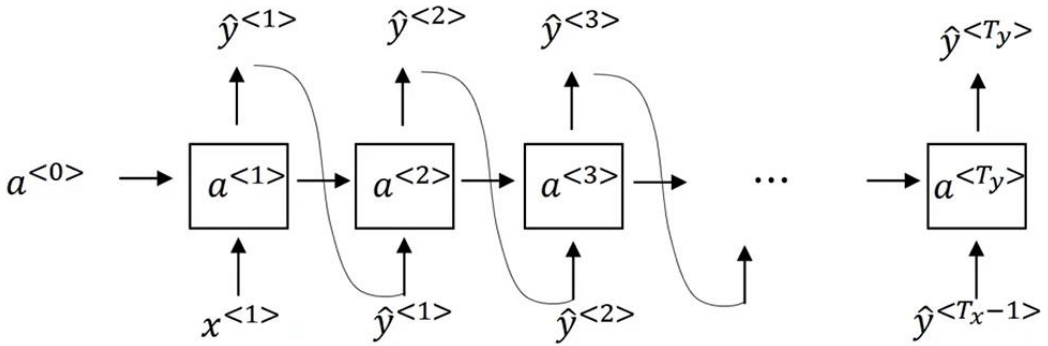
Sequence Modelling Applications



Auto-regressive generation

Sampling technique suggested by Claude Shannon and the psychologists George Miller and Jennifer Self-ridge (Miller and Selfridge, 1950):

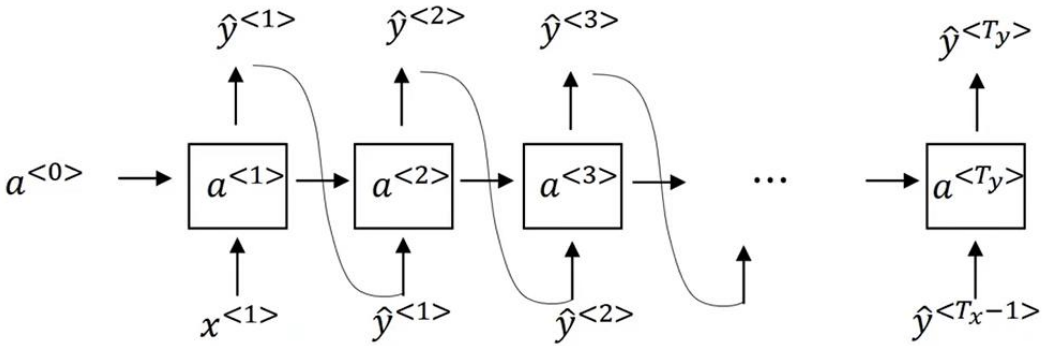
- Randomly sample a word to begin a sequence based on its suitability as the start of a sequence.
- Continue to sample words conditioned on our previous choices until we reach a pre-determined length, or an end of sequence token is generated.



Auto-regressive generation

This approach of using a language model to incrementally generate words by repeatedly sampling the next word conditioned on our previous choices is called **autoregressive generation** or **causal LM generation**.

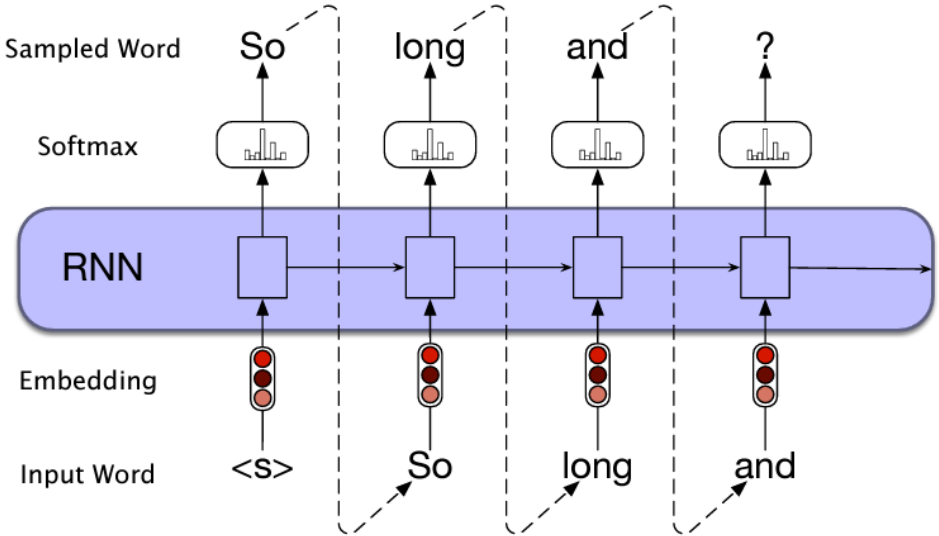
- Sample a word in the output from the *softmax* distribution that results from using the beginning of sentence marker, <s>, as the first input.
- Use the word embedding for that first word as the input to the network at the next time step, and then sample the next word in the same fashion.
- Continue generating until the end of sentence marker, </s>, is sampled or a fixed length limit is reached.



Auto-regressive generation

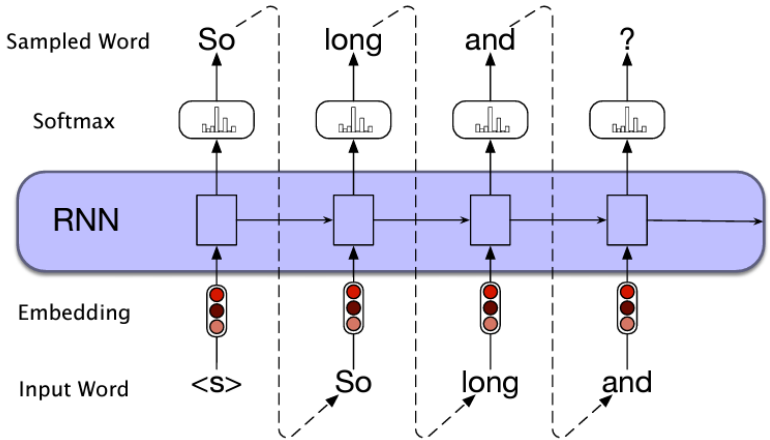
This approach of using a language model to incrementally generate words by repeatedly sampling the next word conditioned on our previous choices is called **autoregressive generation** or **causal LM generation**.

- Sample a word in the output from the *softmax* distribution that results from using the beginning of sentence marker, <s>, as the first input.
- Use the word embedding for that first word as the input to the network at the next time step, and then sample the next word in the same fashion.
- Continue generating until the end of sentence marker, </s>, is sampled or a fixed length limit is reached.

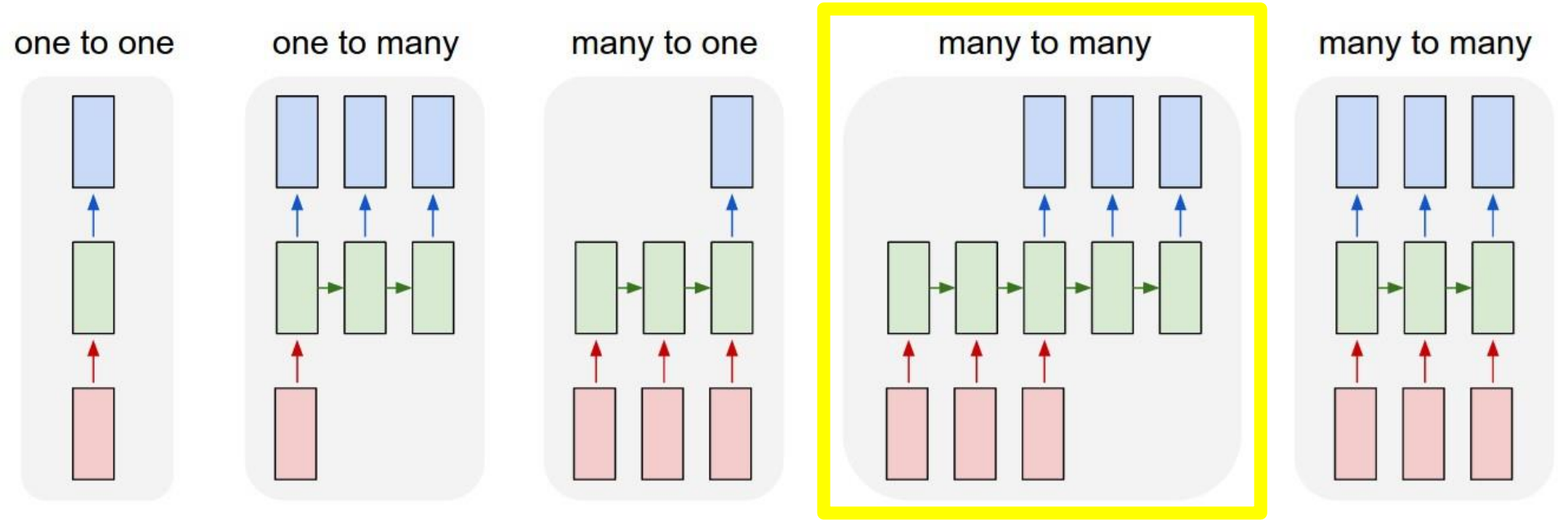


Auto-regressive generation

- This simple architecture underlies state-of-the-art approaches to applications such as machine translation, summarization, and question answering.
- The key to these approaches is to prime the generation component with an appropriate context.
- For translation, the context is the sentence in the source language; for summarization it's the long text we want to summarize



Types of RNN Architectures

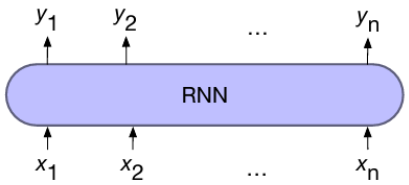


Encoder Decoder Architecture

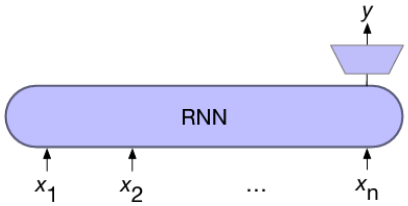
Source: Andrej Karpathy, The Unreasonable Effectiveness of Recurrent Neural Networks - http://karpathy.github.io/2015/05/21/rnn-effectiveness/

NLP Architectures

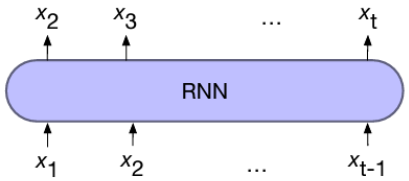
- **Encoder-decoder Architecture:** When the lengths of the input and output sequences are not bound to be the same, we use encoder-decoder models.
- **For example,** machine translation, speech recognition, summarization, question-answering…
- In the encoder-decoder model we have two separate RNN models
  - One maps from an input sequence $\boldsymbol{x}$ to an intermediate representation we call the ***context***, and
  - A second which maps from the context to an output sequence $\boldsymbol{y}$.



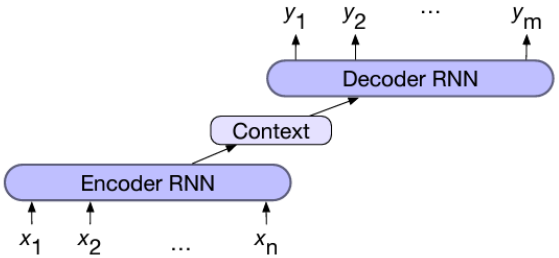
a) sequence labeling



b) sequence classification



c) language modeling

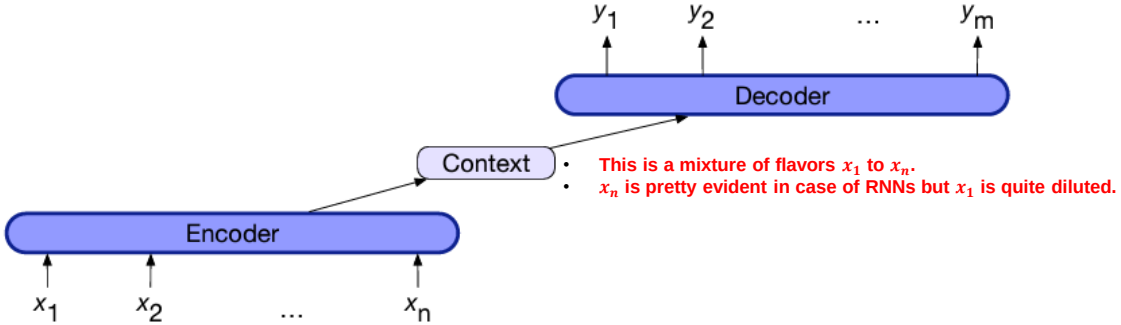


d) encoder-decoder

The Encoder-Decoder Architecture

Encoder-decoder networks consist of three components:

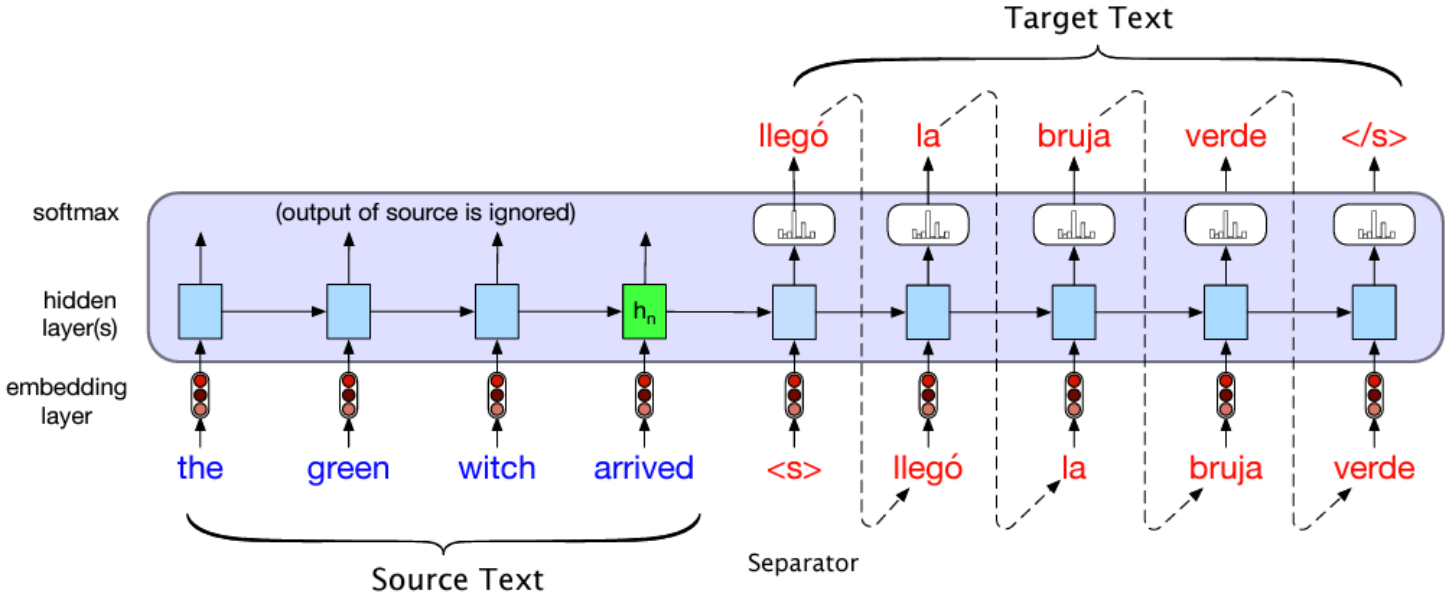
1. An encoder that accepts an input sequence, x_1^n, and generates a corresponding sequence of contextualized representations, h_1^n.
   - RNNs, LSTMs, CNNs, and Transformers can all be employed as encoders
2. A context vector, c, which is a function of h_1^n, and conveys the essence of the input to the decoder.
3. A decoder, which accepts c as input and generates an arbitrary length sequence of hidden states h_1^m, from which a corresponding sequence of output states y_1^m, can be obtained.
   - Just as with encoders, decoders can be realized by any kind of sequence architecture.



The Encoder-Decoder Architecture

- Translating English source text: “the green witch arrived” to a Spanish sentence “lleg´o la bruja verde” (‘arrived the witch green’)
- x is the source text plus the separator token <s> , and y is the target text
- Then an encoder-decoder model computes the probability $p(y|x)$ as follows:

$$\boldsymbol{p(y|x) = p(y_1|x)p(y_2|y_1,x)p(y_3|y_1,y_2,x) \ldots P(y_m|y_1,\ldots,y_{m-1},x)}$$



Sequence Modelling Design Criteria

To model sequences, we need to:

- Handle **variable-length** sequences
- Track **long-term** dependencies
- Maintain information about **order**
- **Share parameters** across the sequence

RNNs meet these criteria

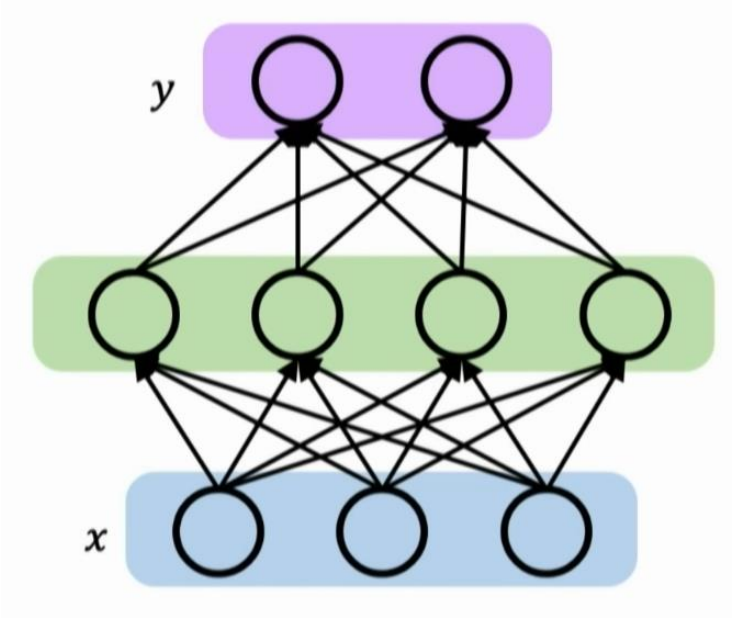
So, why RNNs?

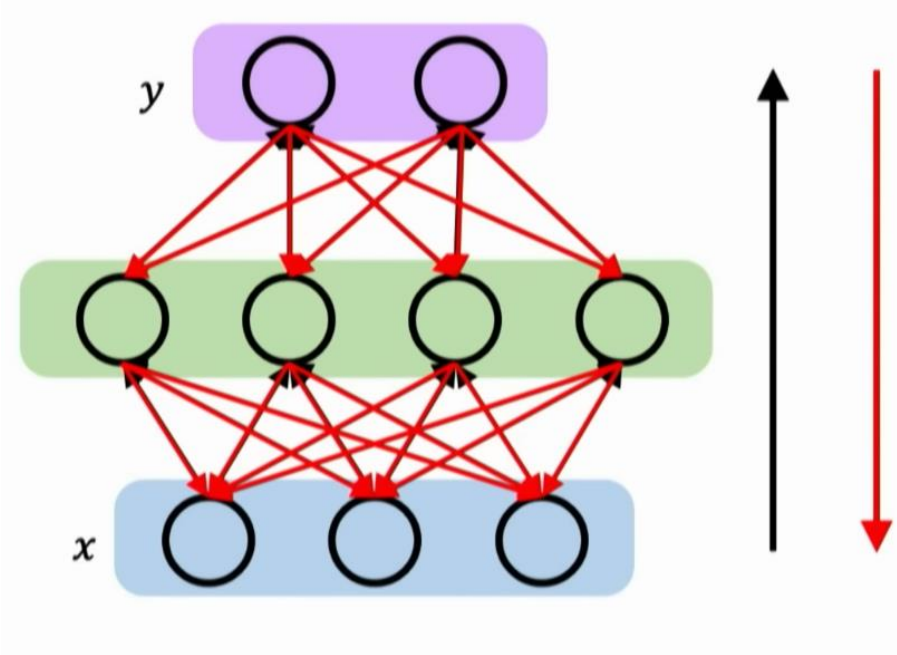
- They allow us to *operate* over *sequences of vectors*.
  - A lot more flexibility in how we can phrase the problems.
- They have temporal dependencies!
  - The model will behave differently if we change up the order of words in a sentence.
  - This is done by “sharing” their parameters and activations across time.
  - Their “recurrent” nature captures dependencies and patterns across time.
  - Understanding evolves over time. Feed words in one at a time to have the model deepen its understanding. Impossible with plain Neural Networks.
- Great results!

Source: D2L Chapter 9.4 - https://d2l.ai/chapter_recurrent-neural-networks/rnn.html

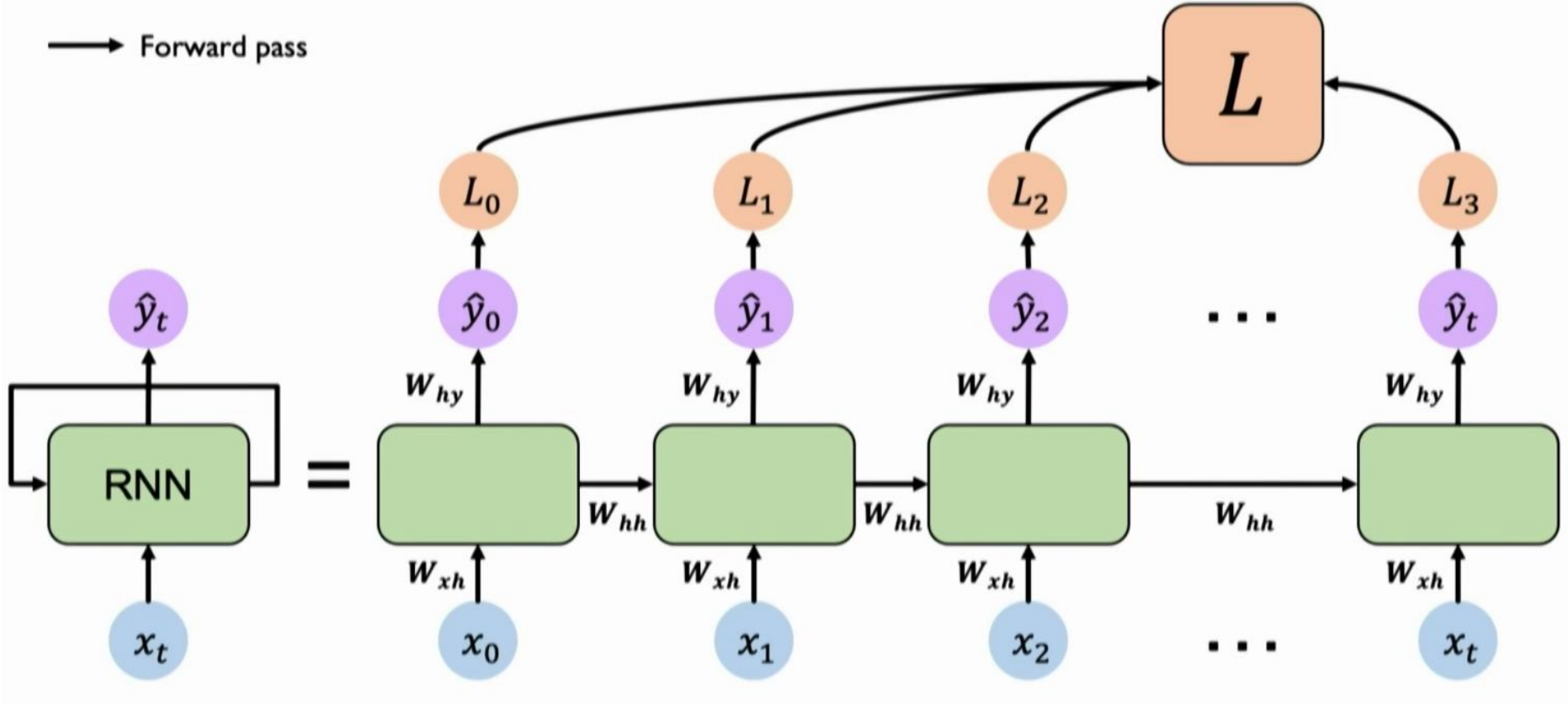
Training Recurrent Neural Networks

Training the NN: Backpropagation

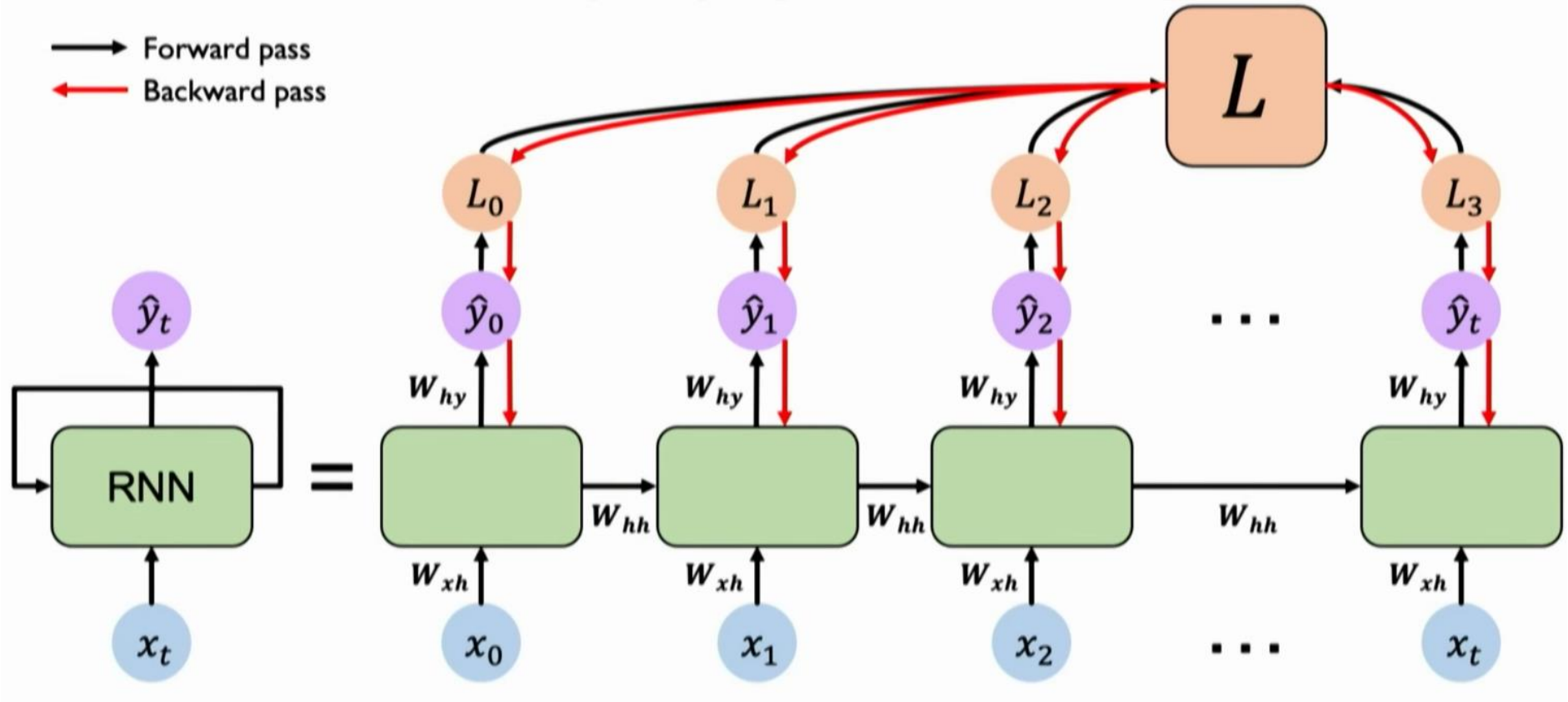




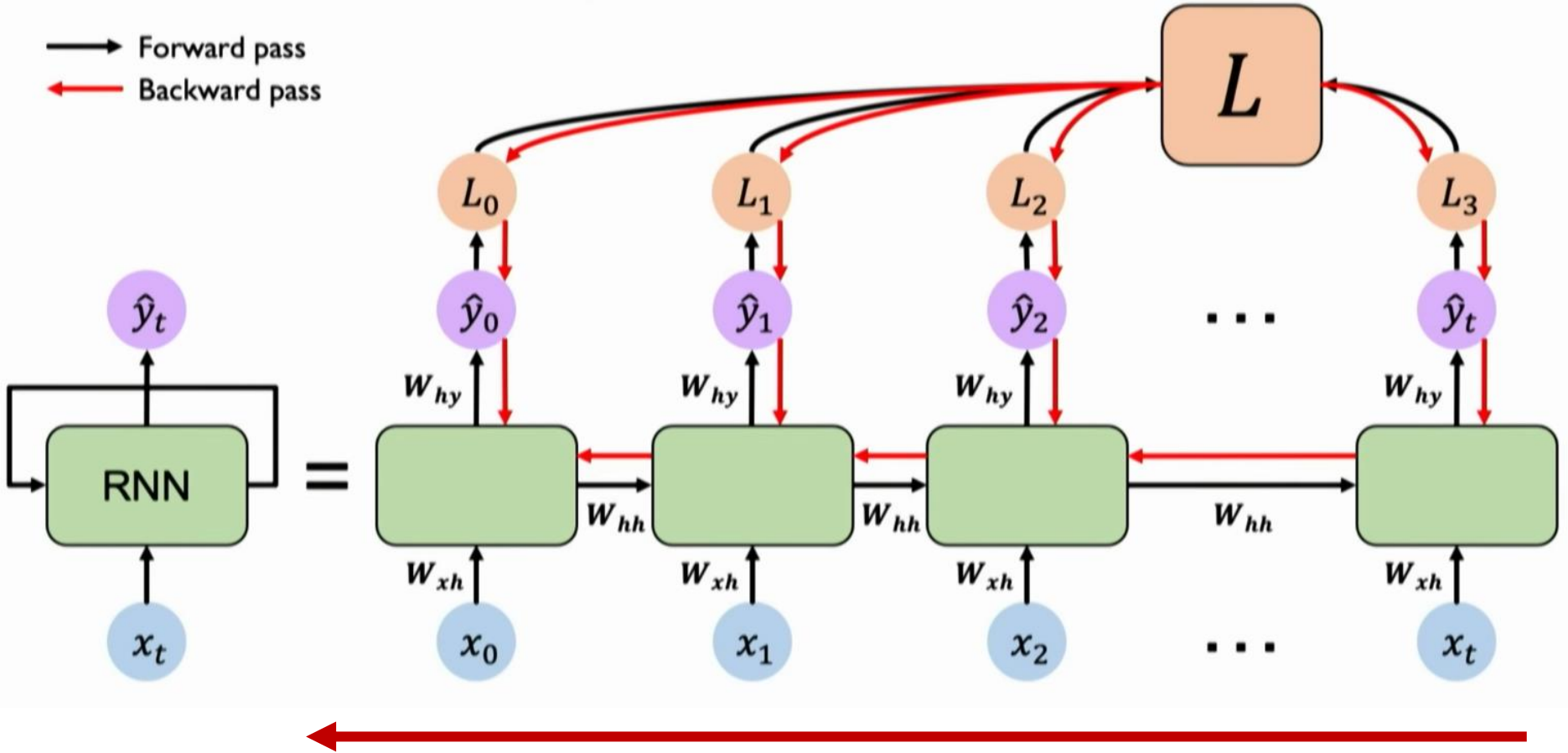
Training the RNN: Backpropagation through Time



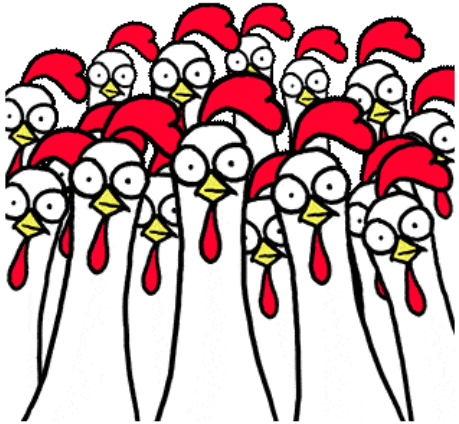
Training the RNN: Backpropagation through Time



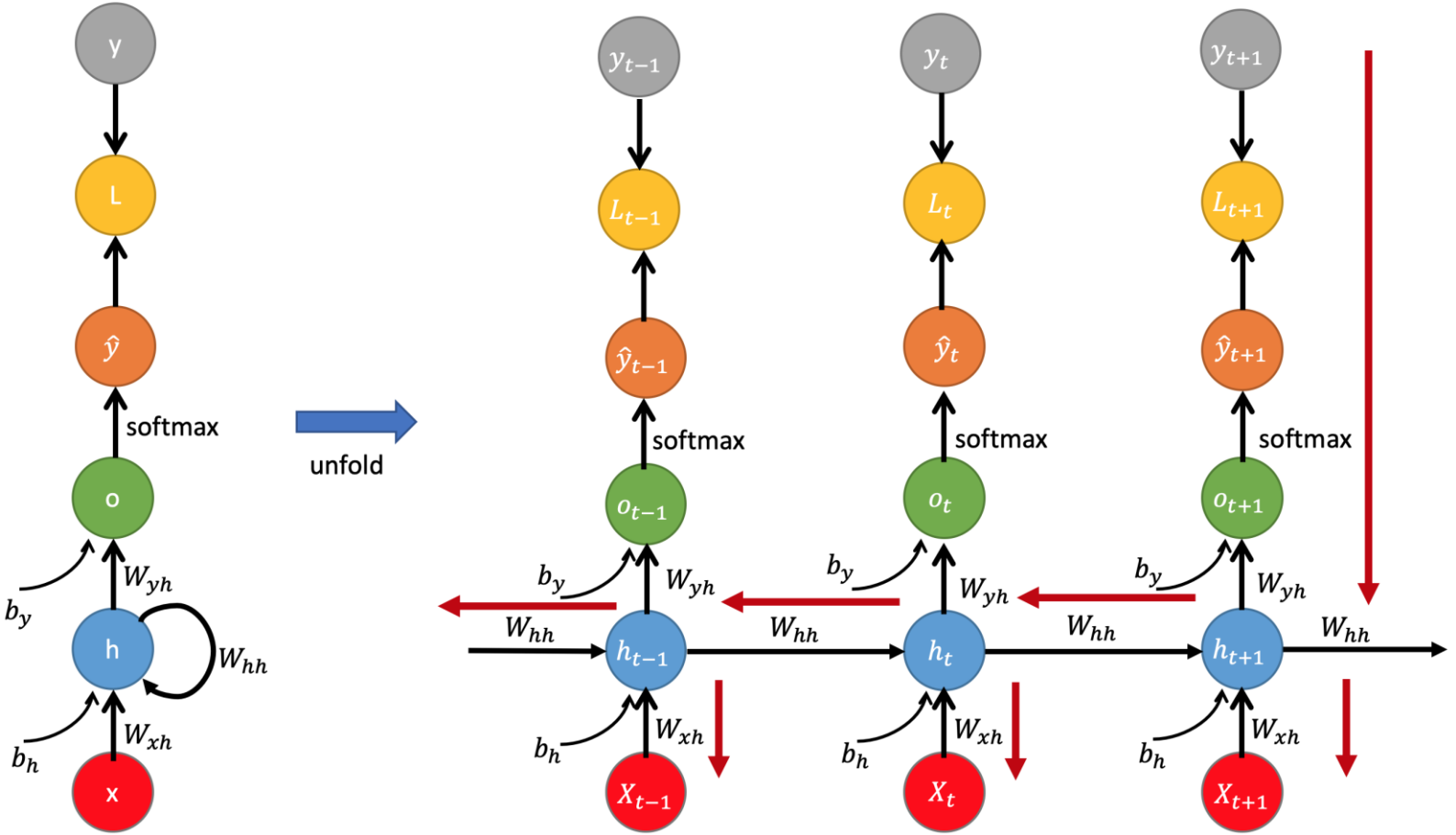
Training the RNN: Backpropagation through Time



Let's derive those gradients!



Detailed Derivation: BPTT



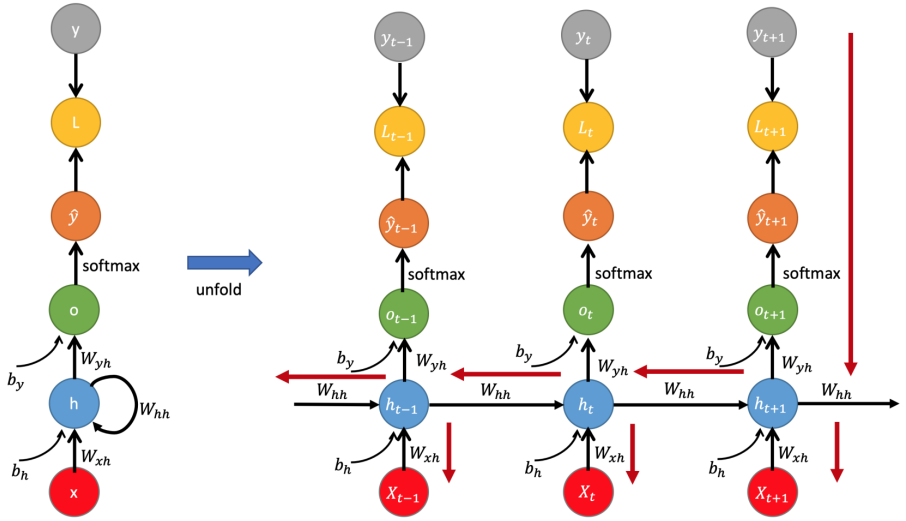
Backpropagation Through Time for Recurrent Neural Network: https://mmuratarat.github.io/2019-02-07/bptt-of-rnn

Detailed Derivation: BPTT

Forward Pass:

$$h_t = f_h(X_t, h_{t-1})$$
$$h_t = \phi_h\left(W_{xh}^T \cdot X_t + W_{hh}^T \cdot h_{t-1} + b_h\right)$$

$$\hat{y}_t = f_o(h_t)$$
$$o_t = W_{yh}^T \cdot h_t + b_y$$
$$\hat{y}_t = \phi_o(o_t)$$



where

- W_{xh}, W_{hh}, and W_{yh} are weight matrices for the input, recurrent connections, and the output, respectively.
- ϕ_h and ϕ_o are element-wise nonlinear functions e.g., sigmoid or a hyperbolic tangent function.
- Reusing same weight matrix W every time step reduces the number of parameters!

Backpropagation Through Time for Recurrent Neural Network: https://mmuratarat.github.io/2019-02-07/bptt-of-rnn

Detailed Derivation: BPTT

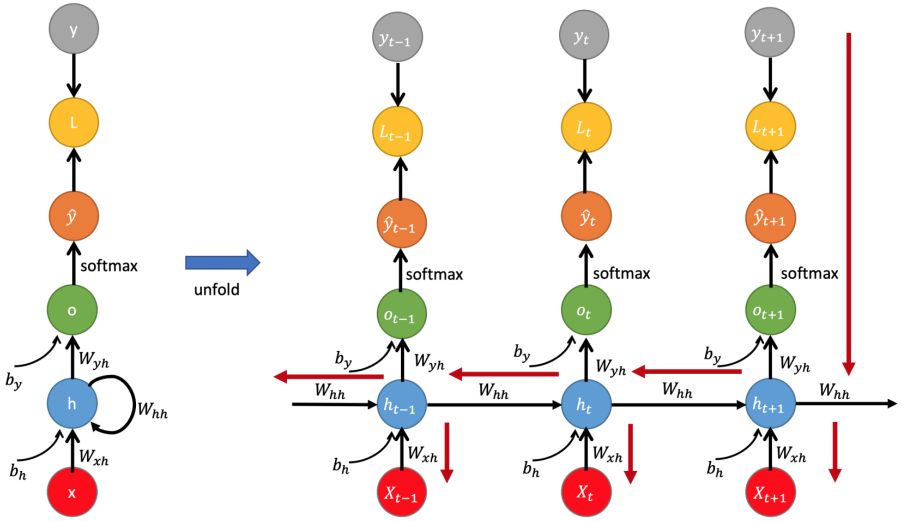
Backpropagation through time:

- The Loss function

$$L(\hat{y}, y) = \sum_{t=1}^{T} L_t(\hat{y}_t, y_t)$$

$$L(\hat{y}, y) = -\sum_{t}^{T} y_t log\hat{y}_t$$

$$L(\hat{y}, y) = -\sum_{t}^{T} y_t \log[softmax(o_t)]$$



Backpropagation Through Time for Recurrent Neural Network: https://mmuratarat.github.io/2019-02-07/bptt-of-rnn

Detailed Derivation: BPTT for W_{yh}

Backpropagation through time:

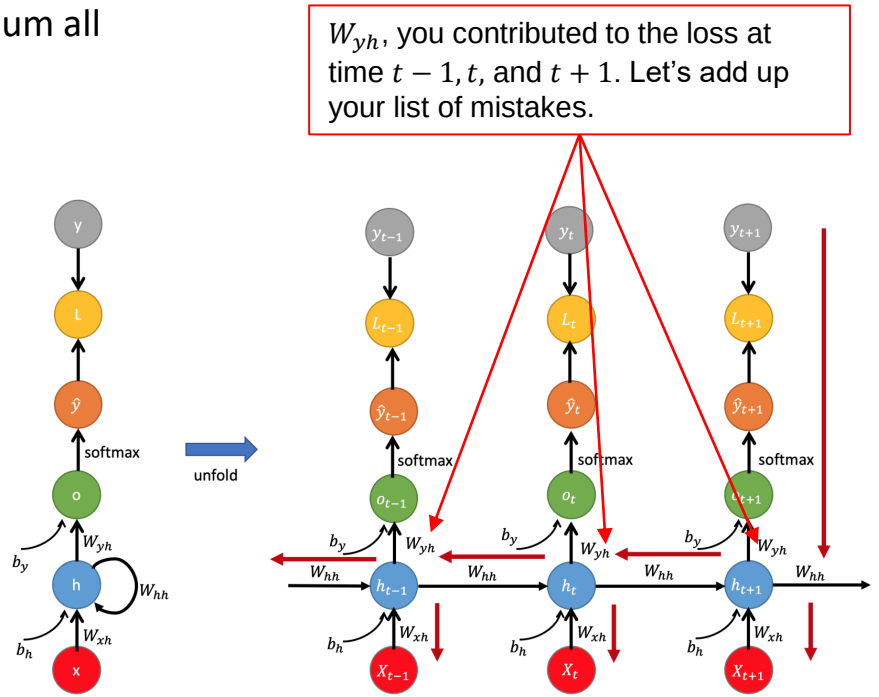
- Note that the weight W_{yh} is shared across all the time sequence.
- Therefore, we can differentiate it at each time step and sum all together:

$$\frac{\partial L}{\partial W_{yh}} = \sum_{t}^{T} \frac{\partial L_t}{\partial W_{yh}}$$

$$\frac{\partial L}{\partial W_{yh}} = \sum_{t}^{T} \frac{\partial L_t}{\partial \hat{y}} \frac{\partial \hat{y}_t}{\partial o_t} \frac{\partial o_t}{\partial W_{yh}}$$

- Similarly, we can get the gradient w.r.t. bias b_y:

$$\frac{\partial L}{\partial b_y} = \sum_{t}^{T} \frac{\partial L_t}{\partial \hat{y}} \frac{\partial \hat{y}_t}{\partial o_t} \frac{\partial o_t}{\partial b_y}$$



Backpropagation Through Time for Recurrent Neural Network: https://mmuratarat.github.io/2019-02-07/bptt-of-rnn

Detailed Derivation: BPTT for W_{hh}

- Let's use L_{t+1} to denote the output of the time-step $t+1$, $L_{t+1} = -y_{t+1} log \hat{y}_{t+1}$.
- The gradient with respect to W_{hh} considering time step $t+1$ only:

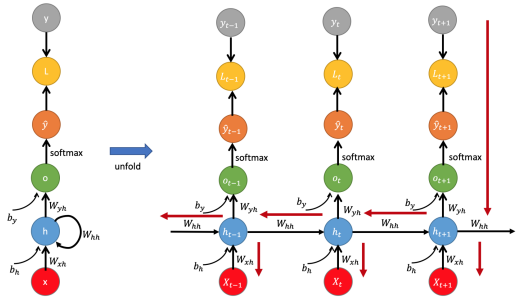
$$\frac{\partial L_{t+1}}{\partial W_{hh}} = \frac{\partial L_{t+1}}{\partial \hat{y}_{t+1}} \frac{\partial \hat{y}_{t+1}}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial W_{hh}}$$

- But, the hidden state h_{t+1} also **partially** depends on h_t according to the recursive formulation $h_t = \phi_h\left(W_{xh}^T \cdot X_t + W_{hh}^T \cdot h_{t-1} + b_h\right)$. Thus, at the time-step t to $t+1$:

$$\frac{\partial L_{t+1}}{\partial W_{hh}} = \frac{\partial L_{t+1}}{\partial \hat{y}_{t+1}} \frac{\partial \hat{y}_{t+1}}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial h_t} \frac{\partial h_t}{\partial W_{hh}}$$

- Therefore,

$$\frac{\partial L_{t+1}}{\partial W_{hh}} = \frac{\partial L_{t+1}}{\partial \hat{y}_{t+1}} \frac{\partial \hat{y}_{t+1}}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial W_{hh}} + \frac{\partial L_{t+1}}{\partial \hat{y}_{t+1}} \frac{\partial \hat{y}_{t+1}}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial h_t} \frac{\partial h_t}{\partial W_{hh}}$$



Backpropagation Through Time for Recurrent Neural Network: https://mmuratarat.github.io/2019-02-07/bptt-of-rnn

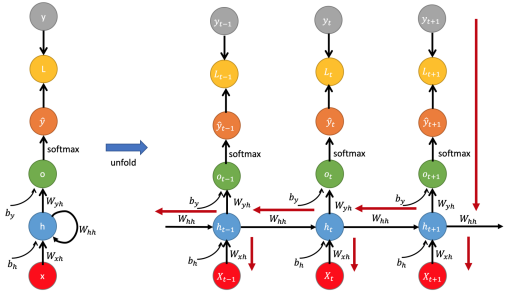
Detailed Derivation: BPTT for W_{hh}

- Thus, at the time-step $t+1$, we can compute the gradient and further use backpropagation through time from $t+1$ to 1 to compute the overall gradient with respect to W_{hh}

$$\frac{\partial L_{t+1}}{\partial W_{hh}} = \sum_{k=1}^{t+1} \frac{\partial L_{t+1}}{\partial \hat{y}_{t+1}} \frac{\partial \hat{y}_{t+1}}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial h_k} \frac{\partial h_k}{\partial W_{hh}}$$

- Note that $\frac{\partial h_{t+1}}{\partial h_k}$ is a chain rule in itself. For example, $\frac{\partial h_3}{\partial h_1} = \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1}$ Rewriting:

$$\frac{\partial L_{t+1}}{\partial W_{hh}} = \sum_{k=1}^{t+1} \frac{\partial L_{t+1}}{\partial \hat{y}_{t+1}} \frac{\partial \hat{y}_{t+1}}{\partial h_{t+1}} \left(\prod_{j=k}^{t} \frac{\partial h_{j+1}}{\partial h_j} \right) \frac{\partial h_k}{\partial W_{hh}}$$



- Where:

$$\prod_{j=k}^{t} \frac{\partial h_{j+1}}{\partial h_j} = \frac{\partial h_{j+1}}{\partial h_k} = \frac{\partial h_{t+1}}{\partial h_t} \frac{\partial h_t}{\partial h_{t-1}} \dots \frac{\partial h_{k+1}}{\partial h_k}$$

- Aggregate the gradients with respect to W_{hh} over the whole time-steps with backpropagation, we can finally yield the following gradient with respect to W_{hh}

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t}^{T} \sum_{k=1}^{t+1} \frac{\partial L_{t+1}}{\partial \hat{y}_{t+1}} \frac{\partial \hat{y}_{t+1}}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial h_k} \frac{\partial h_k}{\partial W_{hh}}$$

Backpropagation Through Time for Recurrent Neural Network: https://mmuratarat.github.io/2019-02-07/bptt-of-rnn

Detailed Derivation: BPTT for W_{xh}

- For W_{xh}, we consider the time-step $t+1$ (which gets only contribution from X_{t+1}) and calculate the gradients with respect to W_{xh} as follows:

$$\frac{\partial L_{t+1}}{\partial W_{xh}} = \frac{\partial L_{t+1}}{\partial \hat{y}_{t+1}} \frac{\partial \hat{y}_{t+1}}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial W_{xh}}$$

- Because h_t and X_{t+1} both make contribution to h_{t+1}, we need to backpropagate to h_t as well:

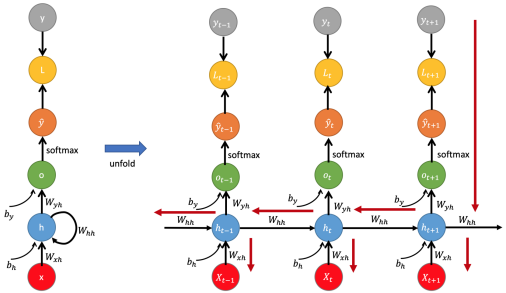
$$\frac{\partial L_{t+1}}{\partial W_{xh}} = \frac{\partial L_{t+1}}{\partial \hat{y}_{t+1}} \frac{\partial \hat{y}_{t+1}}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial W_{xh}} + \frac{\partial L_{t+1}}{\partial \hat{y}_{t+1}} \frac{\partial \hat{y}_{t+1}}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial h_t} \frac{\partial h_t}{\partial W_{xh}}$$

- Thus, summing up all the contributions from $t+1$ to 1 via Backpropagation, we can yield the gradient at the time-step $t+1$:

$$\frac{\partial L_{t+1}}{\partial W_{xh}} = \sum_{k=1}^{t+1} \frac{\partial L_{t+1}}{\partial \hat{y}_{t+1}} \frac{\partial \hat{y}_{t+1}}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial h_k} \frac{\partial h_k}{\partial W_{xh}}$$

- Further, we can take the derivative with respect to W_{xh} over the whole sequence as:

$$\frac{\partial L}{\partial W_{xh}} = \sum_{t}^{T} \sum_{k=1}^{t+1} \frac{\partial L_{t+1}}{\partial \hat{y}_{t+1}} \frac{\partial \hat{y}_{t+1}}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial h_k} \frac{\partial h_k}{\partial W_{xh}}$$



Backpropagation Through Time for Recurrent Neural Network: https://mmuratarat.github.io/2019-02-07/bptt-of-rnn

Problems with RNNs

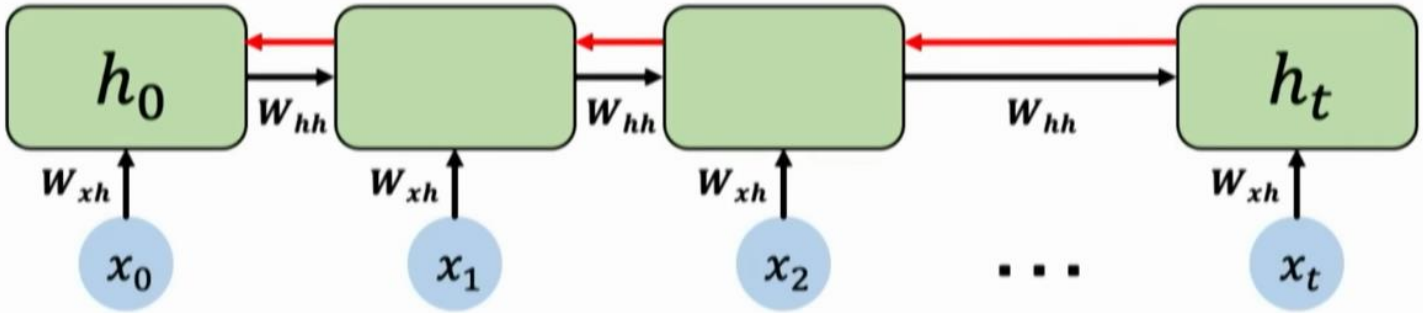
Unstable Gradients

- Just like a Neural Network with too many layers, RNNs can suffer from unstable gradients
  - Exploding Gradients
  - Vanishing Gradients

- Very long sequences can bring about these problems
  - Applying the Chain Rule hundreds of times in BPTT for a sequence that is the length of a newspaper or even longer
  - The **Sigmoid Activation Function** will always squish down numbers into a 0-1 range, hence leading to Vanishing Gradients
  - Finite amount of precision that computers can handle

Source: CS231n 2018, https://cs231n.github.io/rnn/

Exploding Gradients

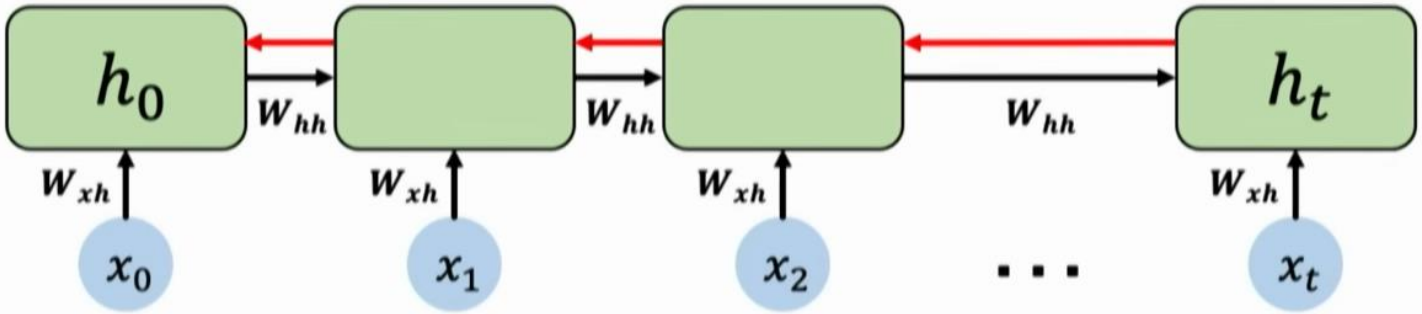


Computing the gradient wrt h_0 involves **many factors of W_{hh} + repeated gradient computation!**

Many values > 1:
exploding gradients

Gradient clipping to scale big gradients

Vanishing Gradients



Computing the gradient wrt h_0 involves **many factors of W_{hh} + repeated gradient computation!**

Many values > 1:
exploding gradients

Gradient clipping to scale big gradients

Many values < 1:
vanishing gradients

1. Activation function
2. Weight initialization
3. Network architecture

Possible Solutions

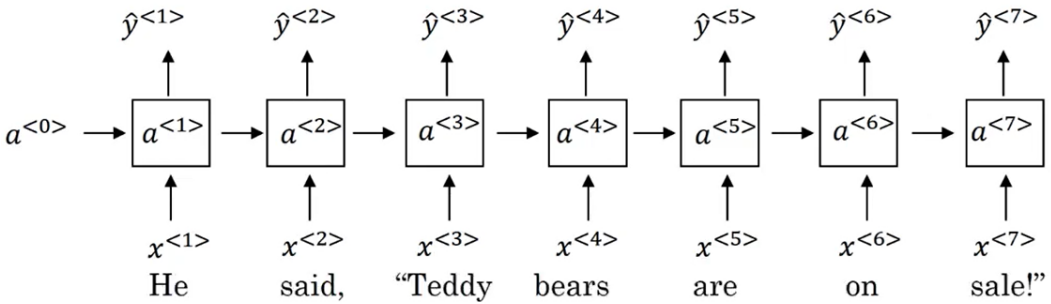
- Truncated BPTT
  - Only carry out the Chain Rule a handful of times, it's still a fixed number of parameters in the model
- Change the Activation Function
  - Instead of Sigmoid, we often use tanh (hyperbolic tangent) for RNNs
- Gradient Clipping
  - To prevent the gradients becoming too large, clip them when they go above a certain threshold

Source: A gentle introduction to exploding gradients in Neural Networks - https://machinelearningmastery.com/exploding-gradients-in-neural-networks/

Unidirectional Nature

He said, “Teddy bears are on sale!”

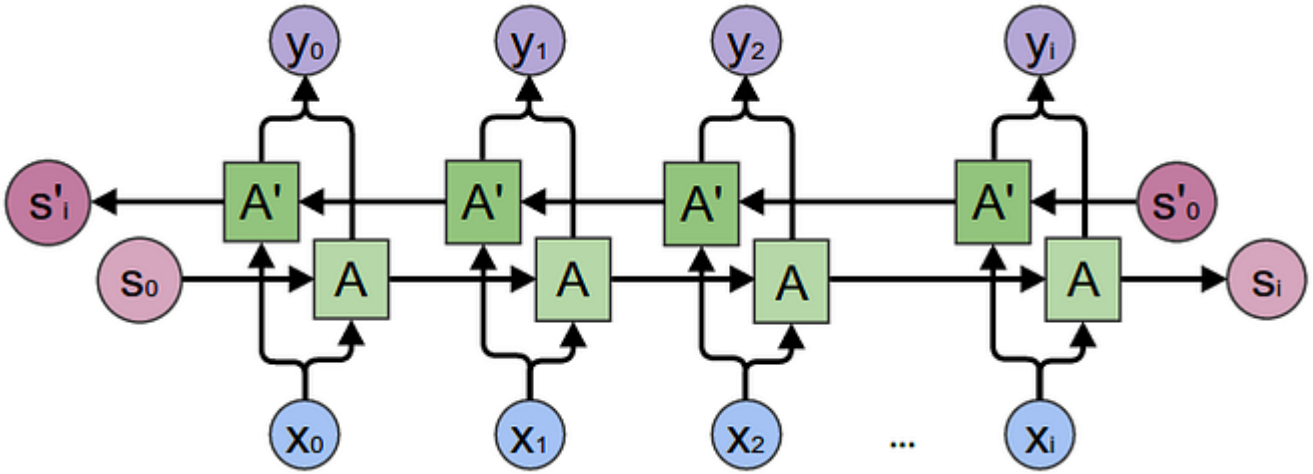
He said, “Teddy Roosevelt was a great President!”



- Plain RNNs are unable to “look in both directions” of the sequence
  - What is the model’s understanding of “Teddy” in the following sentences?
    1. “Teddy bears are on sale!”
    2. “Teddy Roosevelt was a great President!”

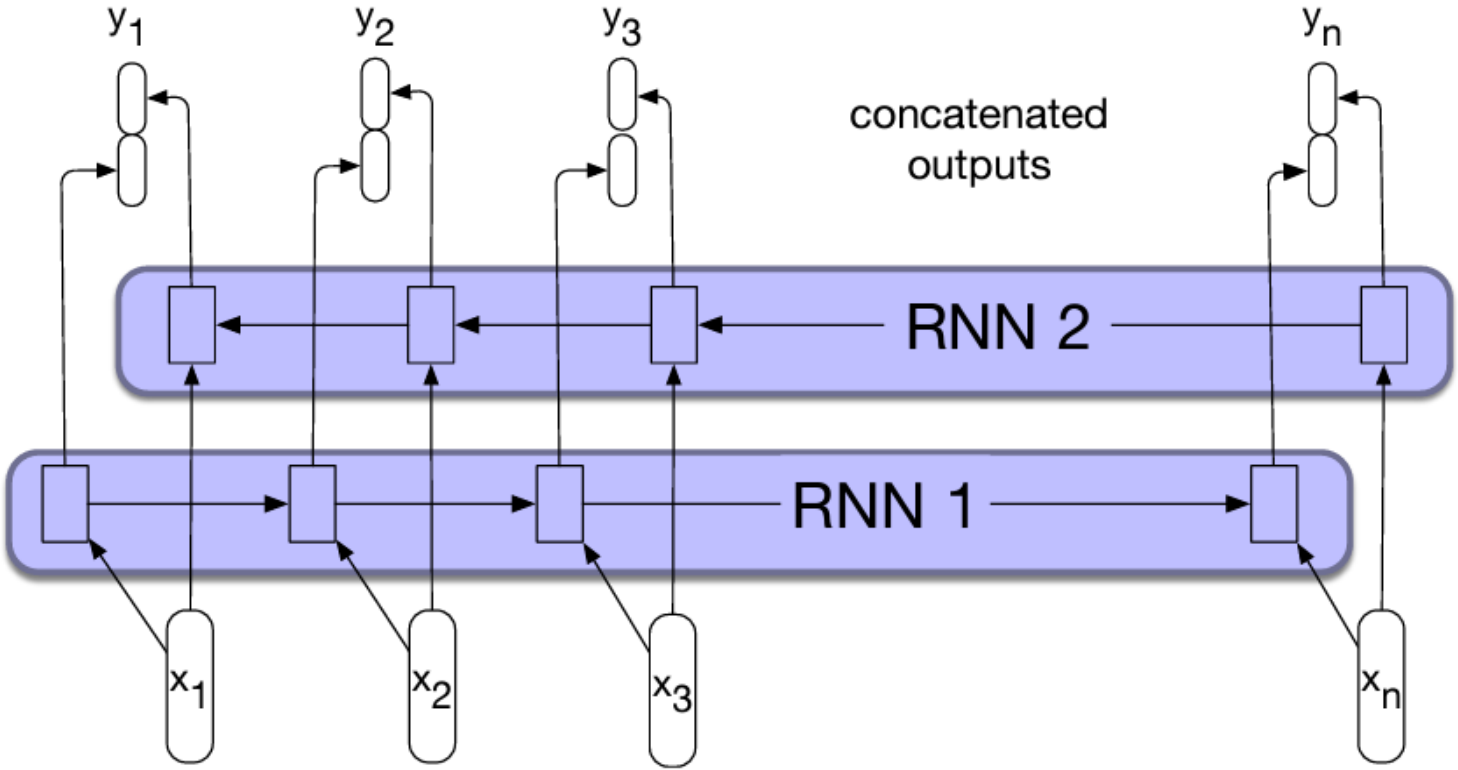
- It can be helpful to look in both directions for certain tasks
  - This is possible with a change to the architecture

Bidirectional RNNs



Source: Understanding Bidirectional RNNs in PyTorch - https://towardsdatascience.com/understanding-bidirectional-rnn-in-pytorch-5bd25a5dd66

Bidirectional RNNs



Source: Understanding Bidirectional RNNs in PyTorch - https://towardsdatascience.com/understanding-bidirectional-rnn-in-pytorch-5bd25a5dd66

Dealing with long-distance depencies

For problems with sufficiently long sequences:

- RNNs run into the problems of Vanishing and Exploding gradients, and
- RNNs often fail to capture long-term dependencies
  - Despite having access to the entire preceding sequence, the information encoded in hidden states tends to be fairly local, more relevant to the most recent parts of the input sequence and recent decisions.

"The flights the airline was cancelling were full."

Dealing with long-distance depencies

- One reason for the inability of RNNs to carry forward critical information is that the **hidden layers**, (and, by extension, the weights that determine the values in the hidden layer,) are being asked to perform two tasks simultaneously:

1. **Provide information** useful for the current decision and

2. **Updating and carrying forward information** required for future decisions.

Dealing with long-distance depencies

- To address these issues, more complex network architectures have been designed to explicitly manage the task of maintaining relevant context over time, by enabling the network:

- To learn to **forget** information that is no longer needed and
- To **remember** information required for decisions still to come.

The Long Short-term Memory (LSTM)

The Long Short-term Memory (LSTM)

- The LSTM network (Hochreiter and Schmidhuber, 1997) divides the context management problem into two subproblems:
  - **Removing information** no longer needed from the context, and
    - i.e., what to forget – the irrelevant information
  - **Retaining information** likely to be needed for later decision making.
    - i.e., what to remember – the long-term memory
- The network is trained via BPTT to learn what to remember and what to forget!

The Long Short-term Memory (LSTM)

- The key to solving both problems is to **learn** how to manage this context rather than hard-coding a strategy into the architecture.
- LSTMs accomplish this by:
  - Adding an explicit **context layer** to the architecture (in addition to the usual recurrent hidden layer), and
  - Using specialized **neural units that make use of gates** to control the flow of information into and out of the units that comprise the network layers.

The Long Short-term Memory (LSTM)

- The **gates** in an LSTM consist of
  - A feed forward layer,
  - Followed by a sigmoid activation function,
  - Followed by a pointwise multiplication with the layer being gated.
- The choice of the **sigmoid** as the activation function arises from its **tendency to push its outputs to either 0 or 1**.
- Combining this with a pointwise multiplication (Hadamard product) has an effect similar to that of a binary mask.
  - Values in the layer being gated that align with values near 1 in the mask are passed through nearly unchanged;
  - Values corresponding to lower values are essentially erased.

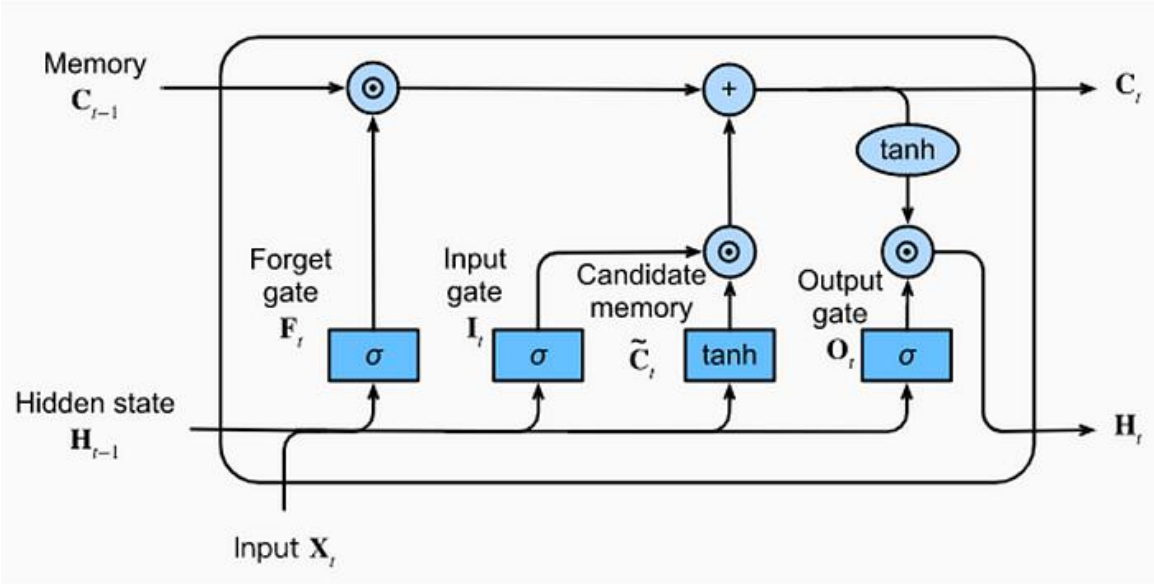
LSTM – Under the hood

LSTM Cell

- Previous context: C_{t-1}
- Previous hidden state: H_{t-1}
- Current input: X_t

The LSTM unit uses:

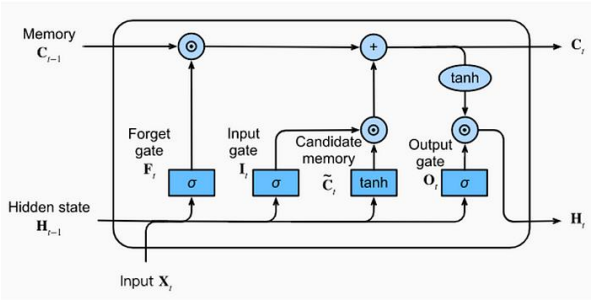
- Recent past information (the short-term memory, H)
- New information coming from the outside (the input vector, X)
  - To update the long-term memory (context, C).
- Finally, it uses the long-term memory (the context state, C) to update the short-term memory (the hidden state, H).



An Intuitive Explanation of LSTM: https://medium.com/@ottaviocalzone/an-intuitive-explanation-of-lstm-a035eb6ab42c, SLP3

Gates

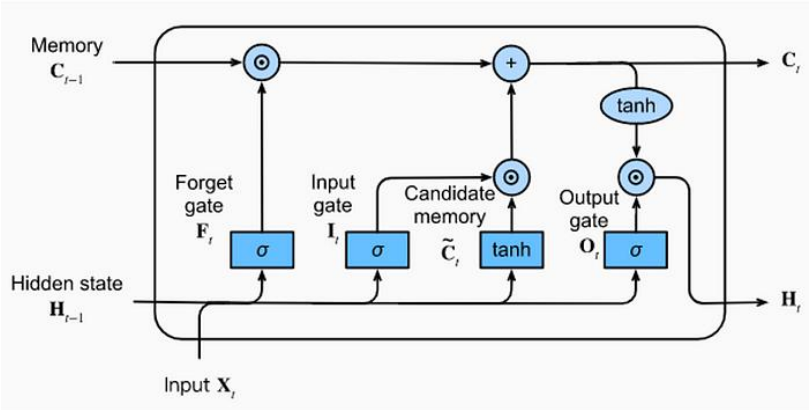
- The three gates (forget gate, input gate and output gate) are information selectors.
- Their task is to create **selector vectors**.
  - A vector with values between zero and one and near these two extremes.
  - A selector vector is created to be multiplied, element by element (Hadamard Product ⊙), by another vector of the same size.
  - This means that a position where the selector vector has a value equal to zero completely eliminates the information included in the same position in the other vector.
  - A position where the selector vector has a value equal to one leaves unchanged the information included in the same position in the other vector.
- All three gates are neural networks that use the sigmoid function as the activation function in the output layer.
- The sigmoid function is used to have, as output, a vector composed of values between zero and one and near these two extremes.
- All three gates use the input vector (X) and the hidden state vector coming from the previous instant (H_{t-1}) concatenated together in a single vector.
  - This vector is the input of all three gates.



An Intuitive Explanation of LSTM: https://medium.com/@ottaviocalzone/an-intuitive-explanation-of-lstm-a035eb6ab42c, SLP3

The Gates of LSTM

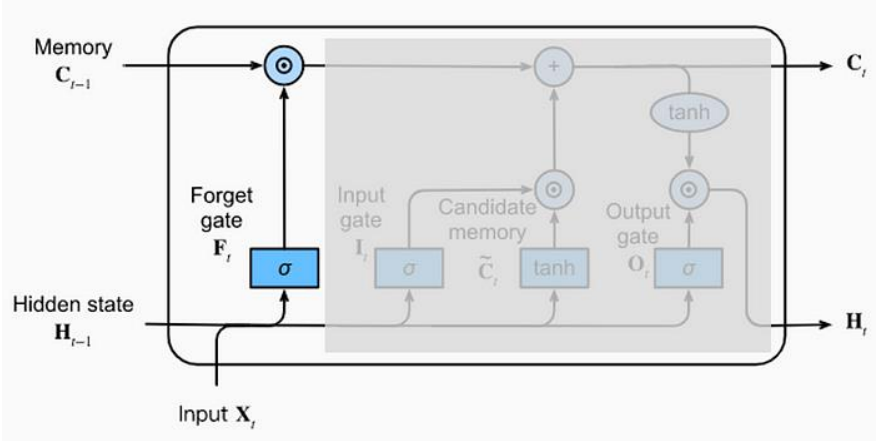
- Each cell is composed of three gates
  1. The **Forget Gate** that tells the cell which information to "forget" from the cell state (the context, C)
  2. The **Input Gate** that tells the cell which information to store in the cell state (the context, C)
  3. The **Output Gate** which is what the cell finally outputs (the hidden state, H)



Long-term context

Short-term context

Forget Gate



- The forget gate decides (based on X_t and H_{t-1} vectors) what information to remove from the context vector coming from time $t-1$. The outcome of this decision is a **selector vector**.

$$f_t = s(U_f H_{t-1} + W_f X_t)$$

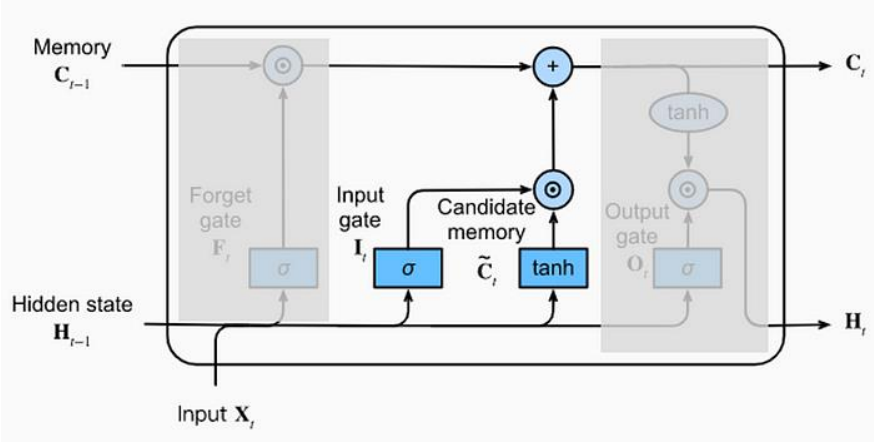
- The selector vector is multiplied element by element with the context vector to decide what to forget and what to keep.

$$k_t = C_{t-1} \odot f_t$$

An Intuitive Explanation of LSTM: https://medium.com/@ottaviocalzone/an-intuitive-explanation-of-lstm-a035eb6ab42c, SLP3

Input Gate

- After removing some of the information from the cell state received in input (C_{t-1}), we can insert a new one.
- This activity is carried out by two independent neural networks: The candidate memory and the input gate.
  - Their input are the vectors X_t and H_{t-1}, concatenated together into a single vector.
  - The **candidate memory** is responsible for the generation of a candidate vector: a vector of information that is candidate to be added to the cell state.
    - Candidate memory output neurons use tanh to ensure that all values of the candidate vector are between -1 and 1.
  - The input gate is responsible for the generation of a **selector vector** which will be multiplied element by element with the candidate vector.
  - The result of the multiplication between the candidate vector and the selector vector is added to the context vector.
    - This adds new information to the context.



Candidate Memory:

$$g_t = tanh(U_g H_{t-1} + W_g X_t)$$

Selector Vector and selection:

$$i_t = \sigma(U_i H_{t-1} + W_i X_t)$$

$$j_t = g_t \odot i_t$$

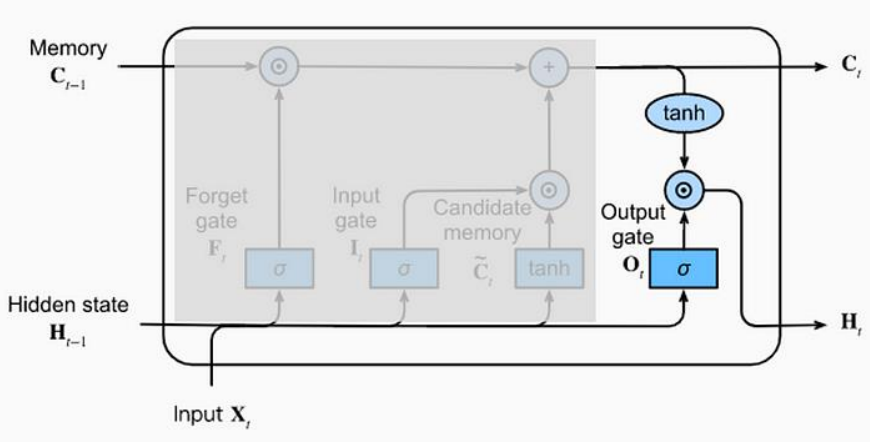
New Context Vector:

$$C_t = j_t + k_t$$

An Intuitive Explanation of LSTM: https://medium.com/@ottaviocalzone/an-intuitive-explanation-of-lstm-a035eb6ab42c, SLP3

Output Gate

- The output gate determines the value of the hidden state outputted by the LSTM (in instant t) and received by the LSTM in the next instant ($t + 1$) input set.
- Output generation also works with a multiplication between a selector vector and a candidate vector.
  - The candidate vector is obtained by using the $\tanh$ on the context vector.
  - The selector vector is generated from the output gate based on the values of X_t and H_{t-1}.



Candidate Vector:

$$q_t = tanh(C_t)$$

Selector Vector

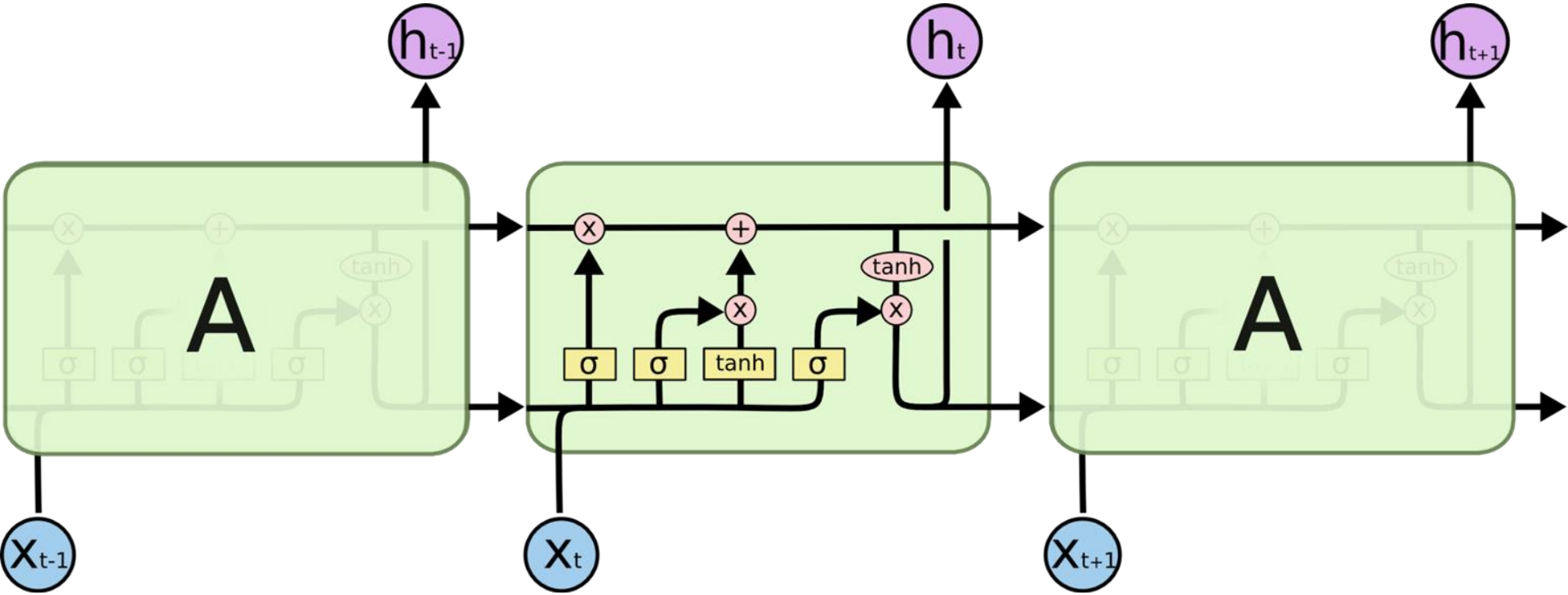
$$o_t = \sigma(U_o H_{t-1} + W_o X_t)$$

New Hidden State output

$$H_t = o_t \odot q_t$$

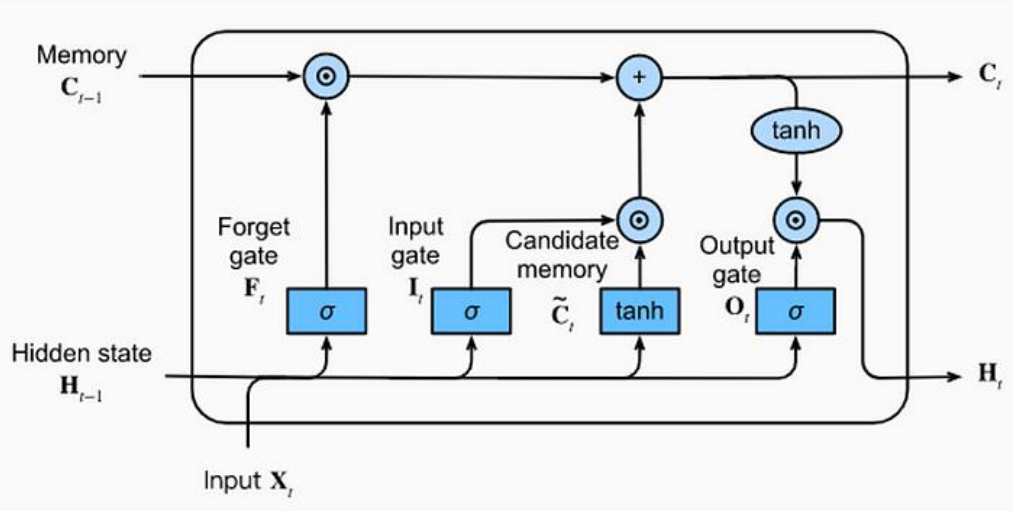
An Intuitive Explanation of LSTM: https://medium.com/@ottaviocalzone/an-intuitive-explanation-of-lstm-a035eb6ab42c, SLP3

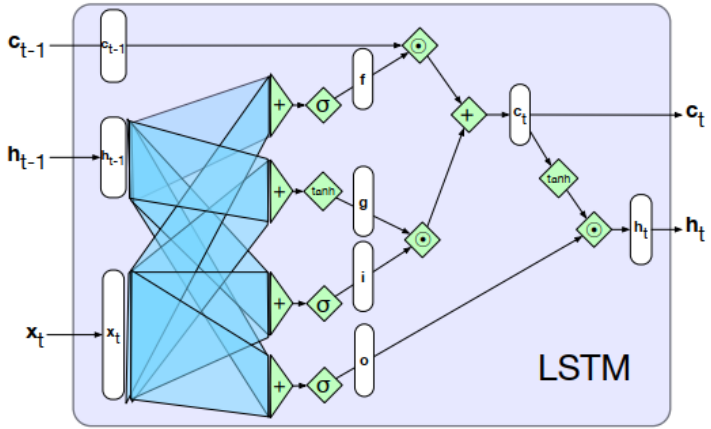
LSTM – Under the hood



Source: Chris Olah, *Understanding LSTM Networks*, https://colah.github.io/posts/2015-08-Understanding-LSTMs/

LSTM – Under the hood

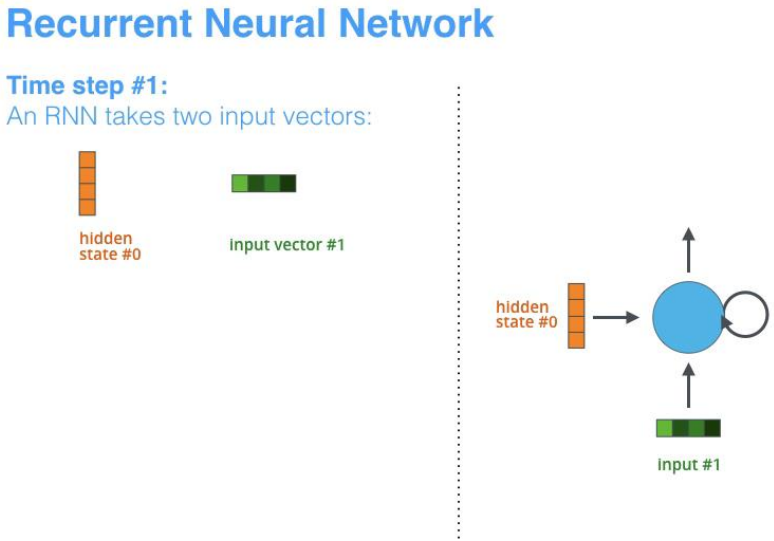




An Intuitive Explanation of LSTM: https://medium.com/@ottaviocalzone/an-intuitive-explanation-of-lstm-a035eb6ab42c, SLP3


Machine → Translation → and → <EOS> → Attention

RNN



Visualizing A Neural Machine Translation Model (Mechanics of Seq2seq Models With Attention): https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/

Missed me?

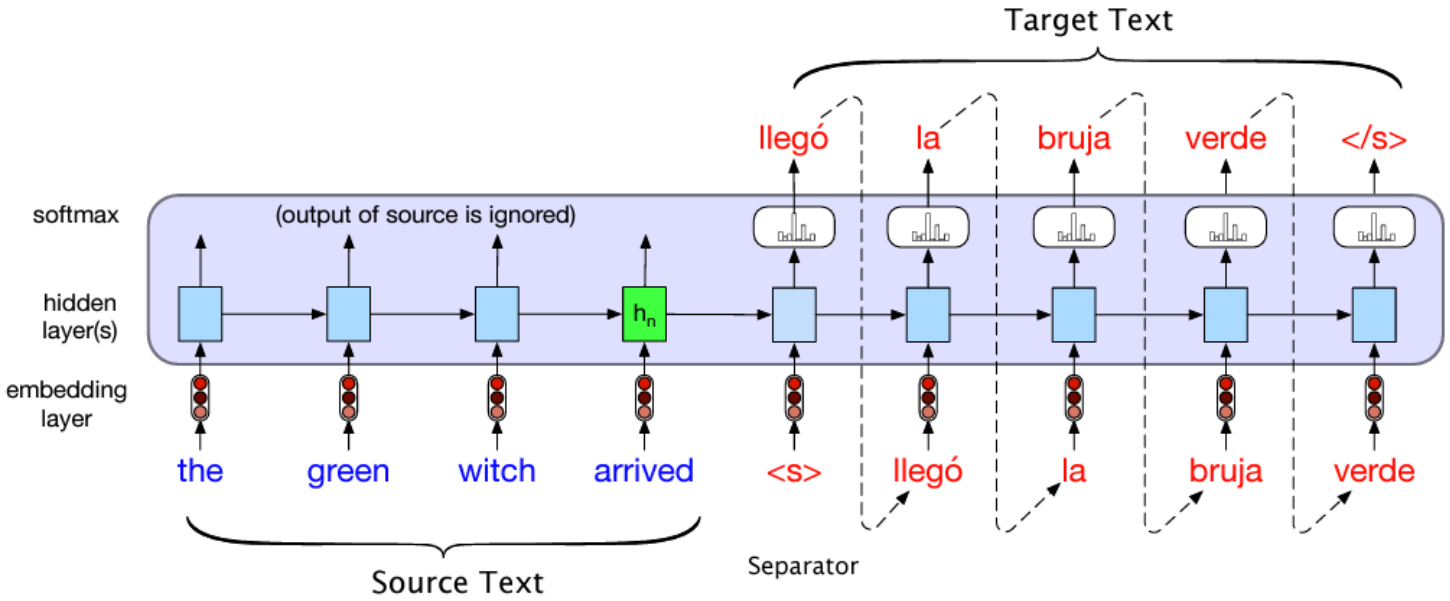


Image Reference: SLP3, https://web.stanford.edu/~jurafsky/slp3/

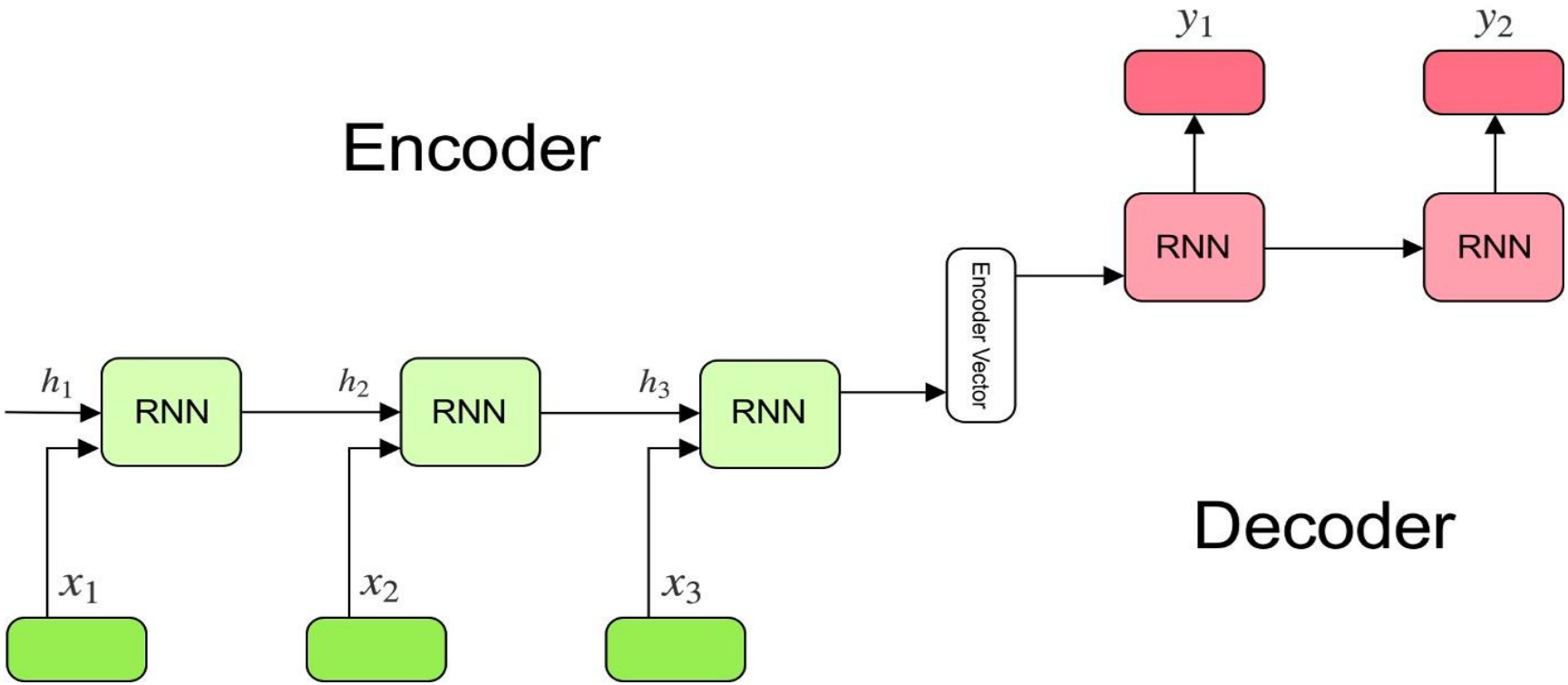
What is Machine Translation?

- Can be represented as a “many to many” problem
  - These are often called Seq2Seq problems

- Setting up the task
  - Model Input: text in one language (e.g. English)
  - Model Output: text generated in another language (e.g. Urdu)

- In Seq2Seq problems, the output is generated in an “autoregressive” fashion
  - This means that the model feeds it’s generated output at timestep *t* as input during timestep (*t*+1)
  - At training time, the correct label (rather than the last predicted label) is fed to each decoder module, and we also stop generating at the correct output length. This is called Teacher Forcing.

Encoder-Decoder Framework



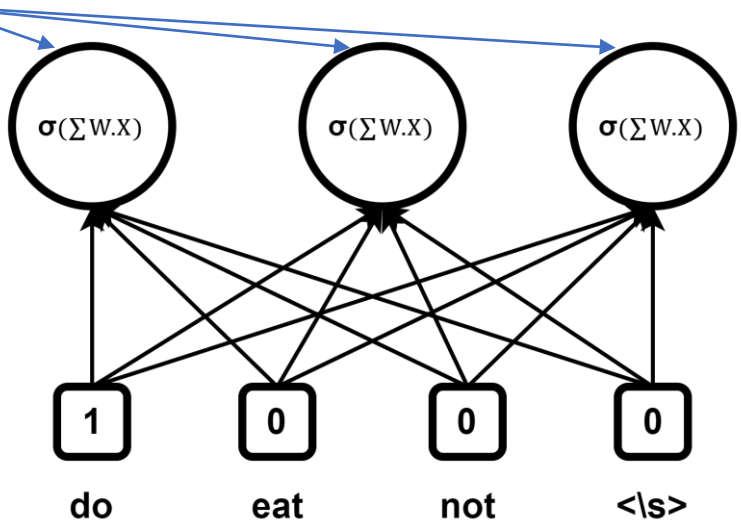
Source: Understanding Encoder Decoder Sequence-to-Sequence Models - https://towardsdatascience.com/understanding-encoder-decoder-sequence-to-sequence-model-679e04af4346

Let's Fix this! But first… You need to know the Truth

- Remember our **"smoothie"** that preserves the taste of each word w_t and also keeps the flavor of some of the previous words ($w_{t-1}, w_{t-2}, \ldots$)?
  - Its **actually the embedding vector** for w_t generated by the Neural Network!

Trained embeddings for "do"

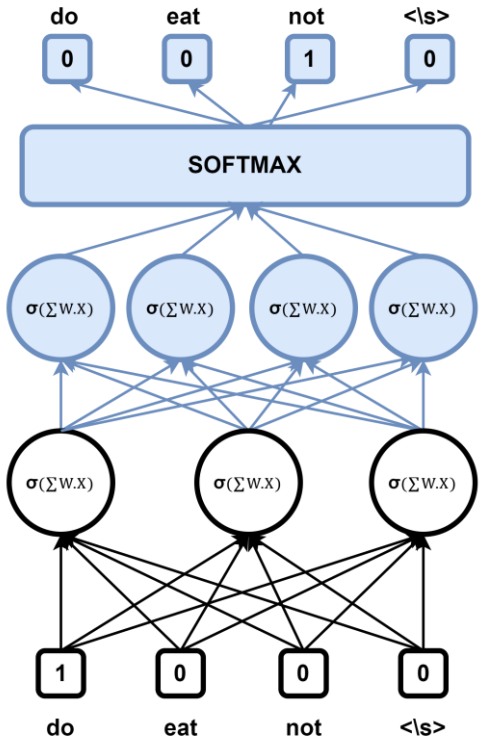
- We are free to choose the number of embeddings.



Stat Quest: Transformer Neural Networks, ChatGPT's foundation, Clearly Explained!!!

Let's Fix this! But first… You need to know the Truth

- We can always go from the "**smoothie" (Embeddings)** to the predicted output word by mapping back to the number of neurons equal to |V|, and doing a softmax



Stat Quest: Transformer Neural Networks, ChatGPT's foundation, Clearly Explained!!!

Now look at this again!

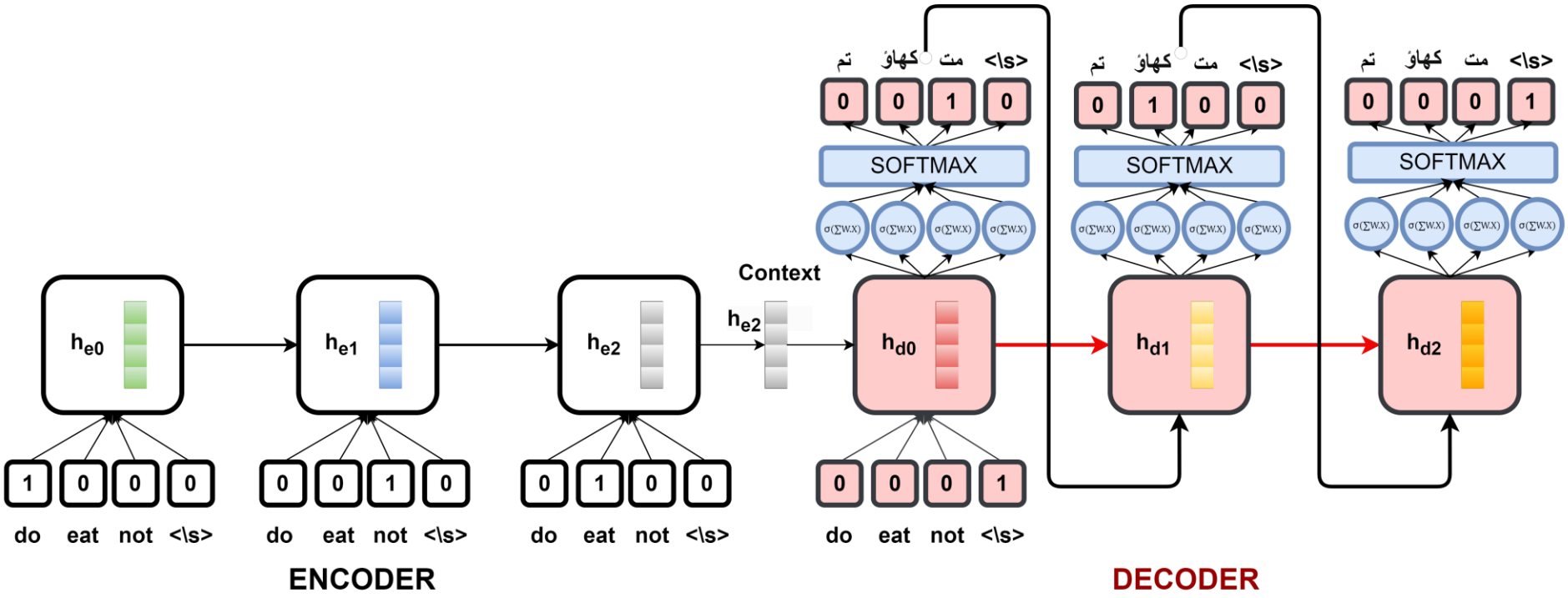


Image Reference: SLP3, https://web.stanford.edu/~jurafsky/slp3/

A better version

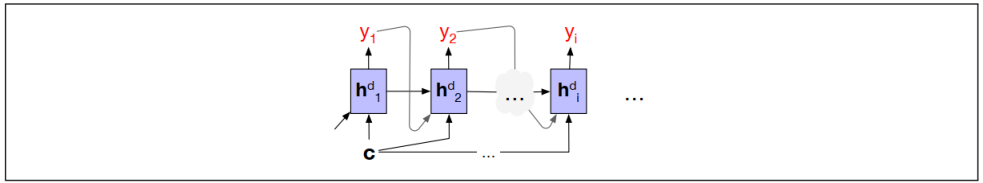


Figure 9.19 Allowing every hidden state of the decoder (not just the first decoder state) to be influenced by the context c produced by the encoder.

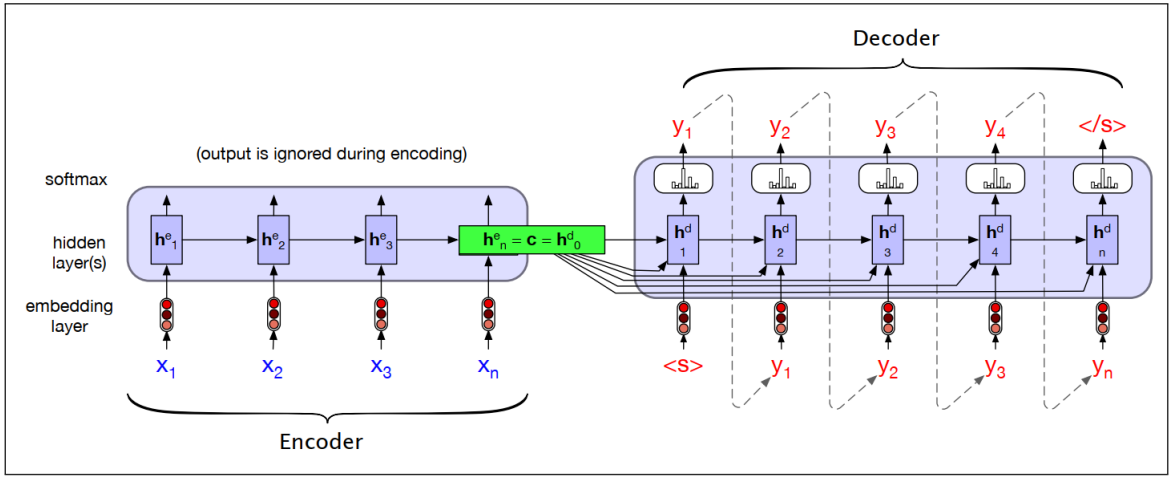


Figure 9.18 A more formal version of translating a sentence at inference time in the basic RNN-based encoder-decoder architecture. The final hidden state of the encoder RNN, h_n^e, serves as the context for the decoder in its role as h_0^d in the decoder RNN.

Image Reference: SLP3, https://web.stanford.edu/~jurafsky/slp3/

Teacher Forcing Approach for Training

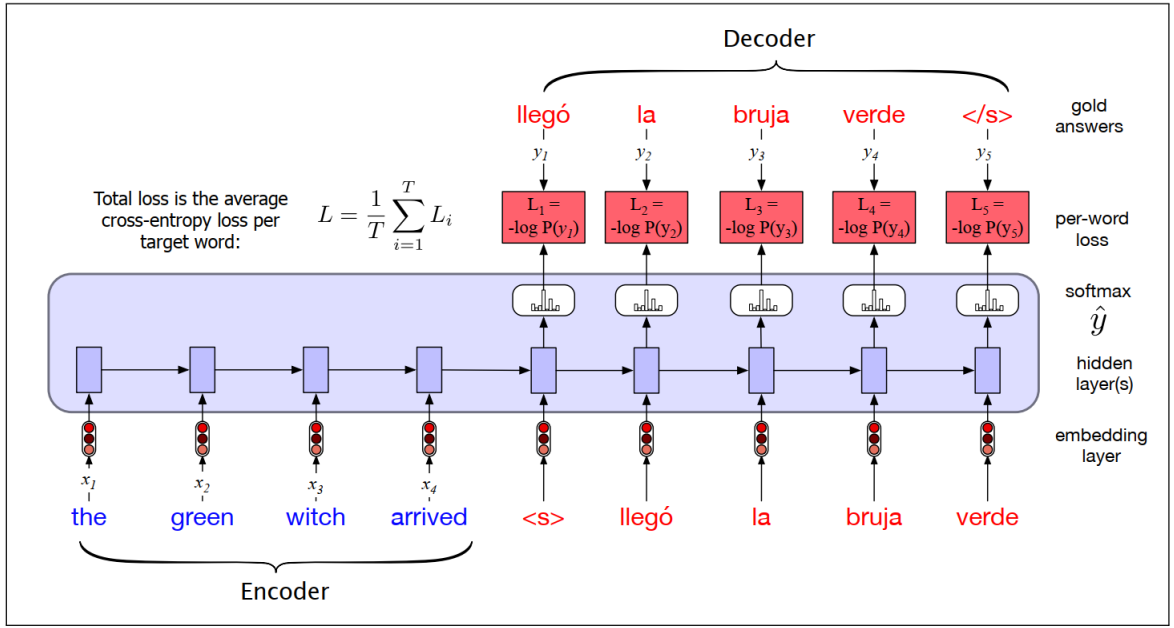


Figure 9.20 Training the basic RNN encoder-decoder approach to machine translation. Note that in the decoder we usually don't propagate the model's softmax outputs $\hat{y}_t$, but use **teacher forcing** to force each input to the correct gold value for training. We compute the softmax output distribution over $\hat{y}$ in the decoder in order to compute the loss at each token, which can then be averaged to compute a loss for the sentence.

Image Reference: SLP3, https://web.stanford.edu/~jurafsky/slp3/

Possible Issues

- Compiling all the knowledge into a single vector allows the model to get a bigger picture before it starts generating tokens
  - May work fine in cases where words at the end of the sentence in language 1, would make more sense to show up at the start in the generated sentence in language 2
- The model technically only glances at the sentence once, and forces itself to generate these tokens
  - This creates an **Information Bottleneck** in the model
  - This is the human equivalent of reading a 500 word passage in English once, and trying to translate it from memory (not very sensible)
  - Translate: Don't try to do good on your exams by cheating!
  - The flavor of "Don't" fades away in the context vector even with LSTMs

Source: DeepLearning.AI, *Sequence Models*, Coursera

Fixing the Problem

- The bottleneck is caused by the Encoder passing only a single Hidden State (also called the “encoder vector”, or “context vector”) to the Decoder
  - Simple fix: pass **all** the hidden states to the Decoder

- What’s the best way to deal with so many Hidden States?
  - *Lame solution:* Could just give all of them equal importance (similar to taking an average), but that doesn’t do anything special
  - *Better solution:* Could **learn how to give attention** to these different states (similar to a weighted average, but the model has to learn the weights)

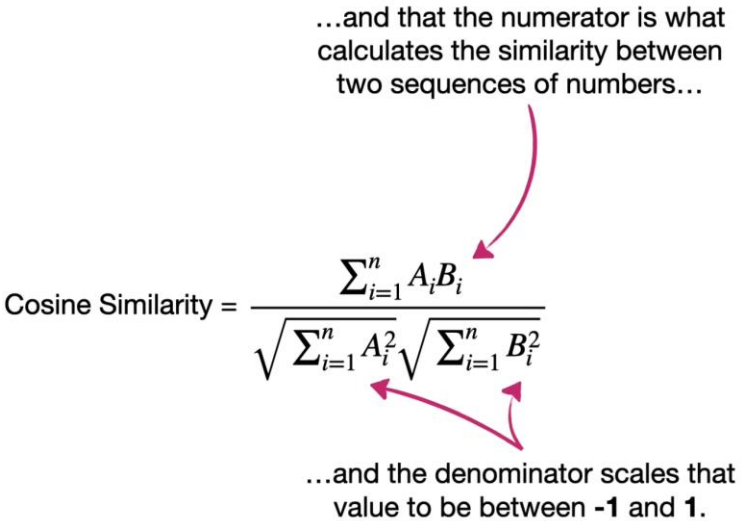
Source: DeepLearning.AI, *Sequence Models*, Coursera

Attention

- The **attention mechanism** is a solution to the bottleneck problem, a way of allowing the decoder to get information from *all* the hidden states of the encoder, not just the last hidden state.
  - The idea of attention is to create the single fixed-length vector *c* by taking a weighted sum of all the encoder hidden states.
  - The weights focus on ('attend to') a particular part of the source text that is relevant for the token the decoder is currently producing.
  - Attention thus replaces the static context vector with one that is dynamically derived from the encoder hidden states, different for each token in decoding.

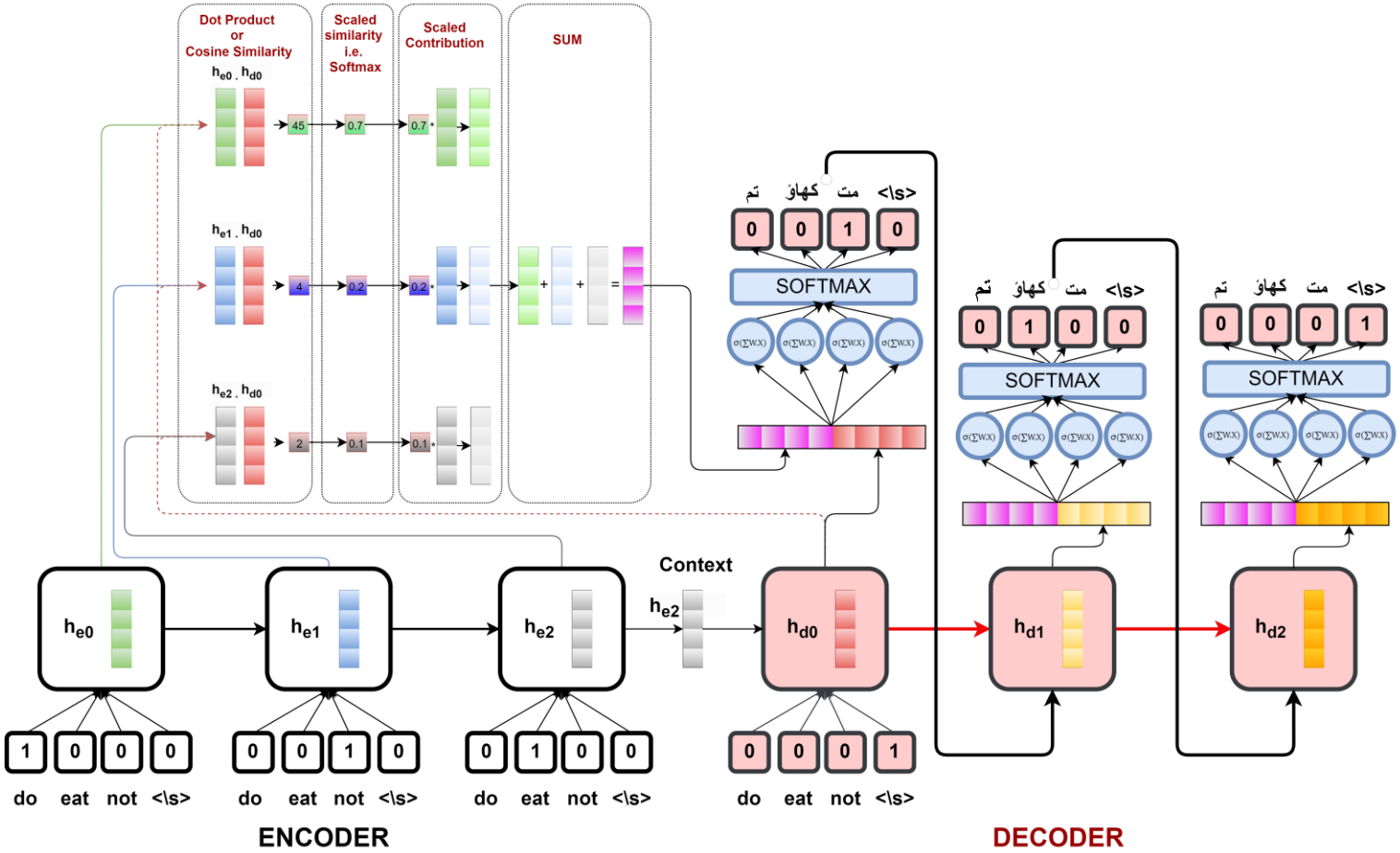
Similarity

- **Attention** is calculated based on the similarity between each hidden state from the encoder and each hidden state produced at the decoder.

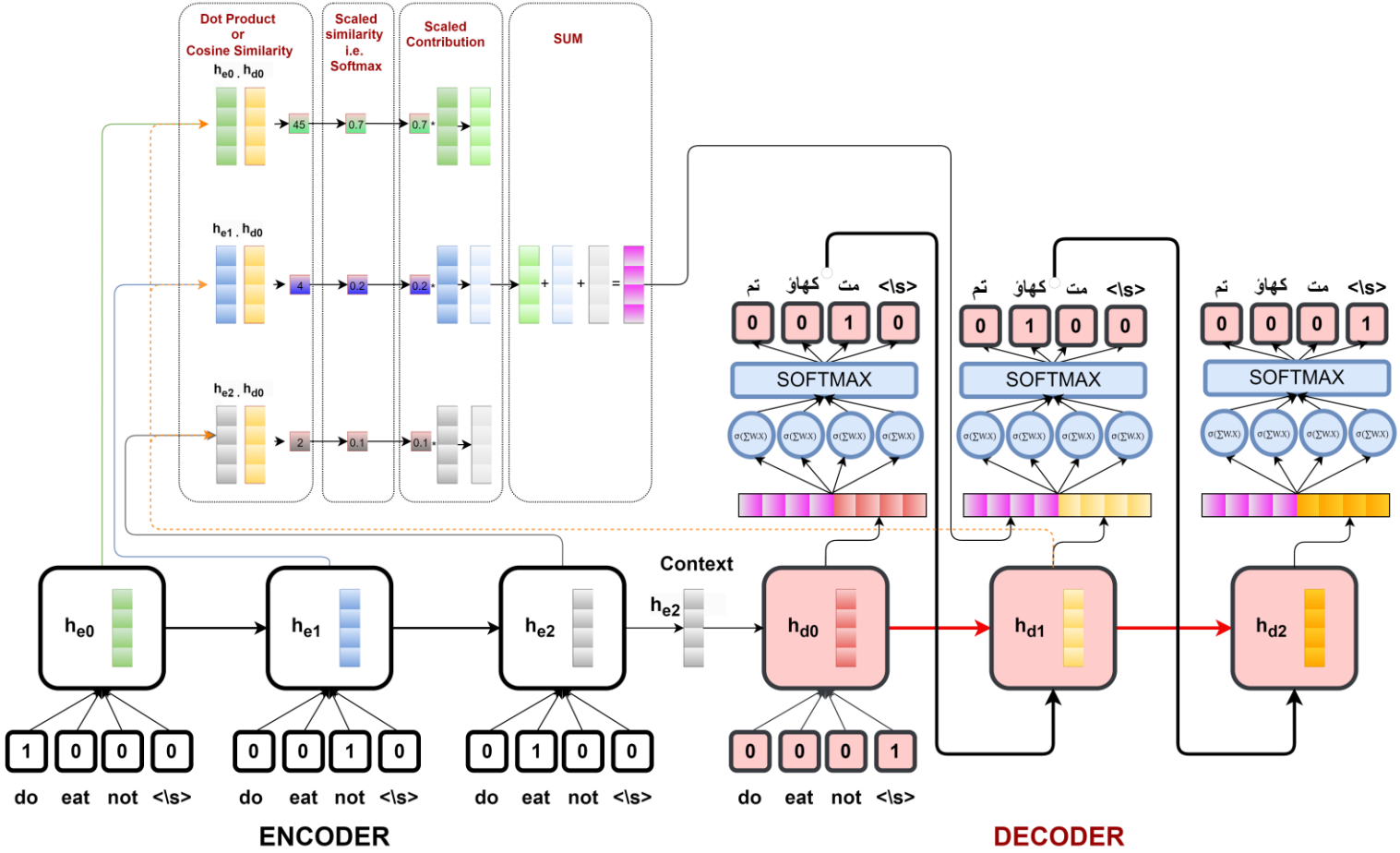


Attention for Neural Networks, Clearly Explained!!! https://www.youtube.com/watch?v=PSs6nxngL6k

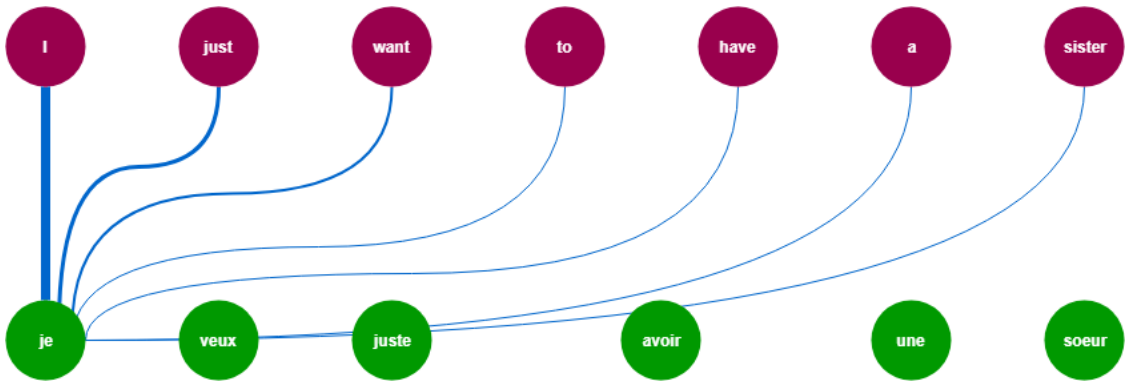
Encoder-Decoder with Attention



Encoder-Decoder with Attention

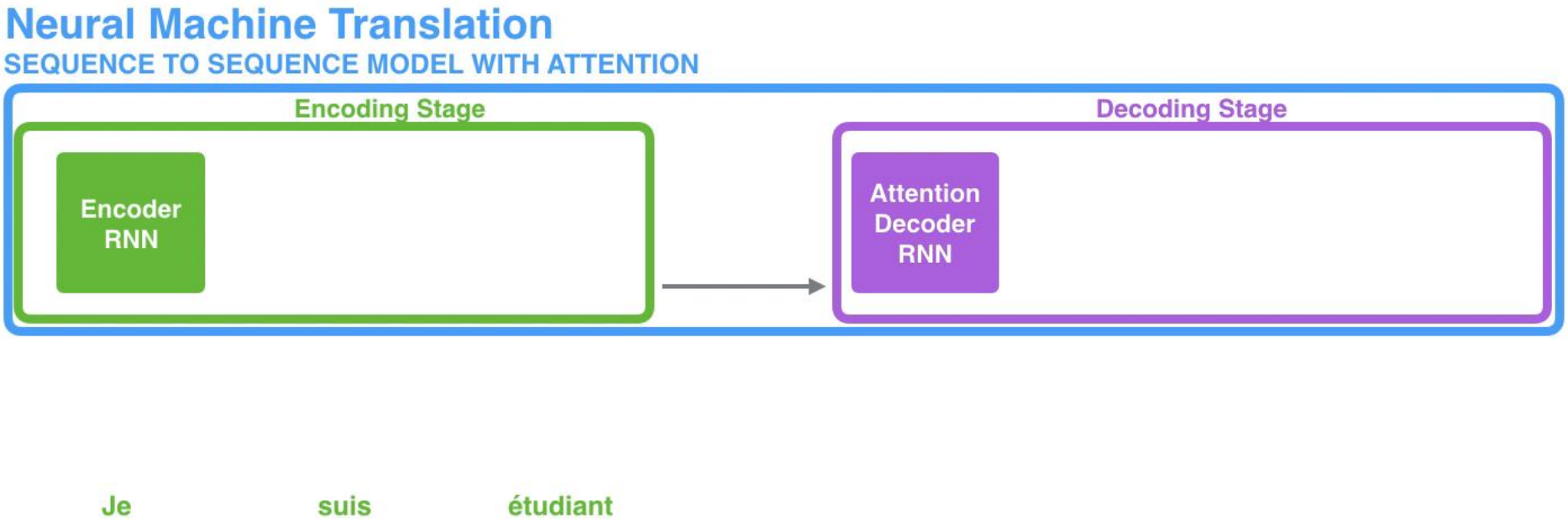


Attending to the inputs that matter



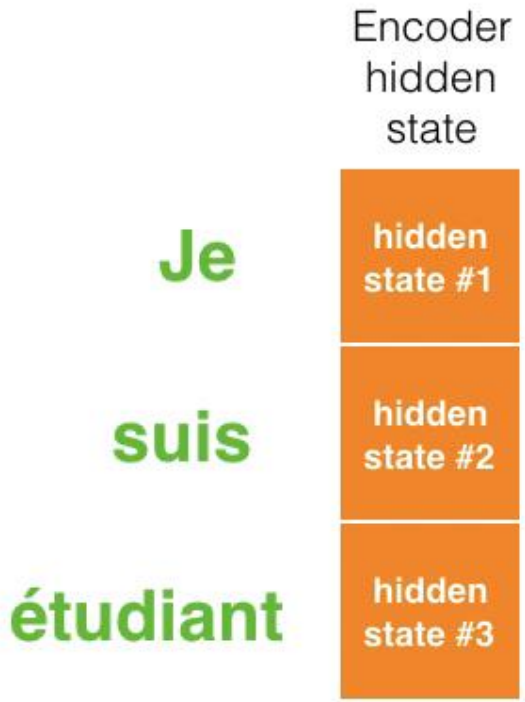
An Explanation of Attention Based Encoder-Decoder Deep Learning Networks: https://www.linkedin.com/pulse/explanation-attention-based-encoder-decoder-deep-keshav-bhandari/

Seq2Seq with Attention



Visualizing A Neural Machine Translation Model (Mechanics of Seq2seq Models With Attention): https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/

Seq2Seq with Attention

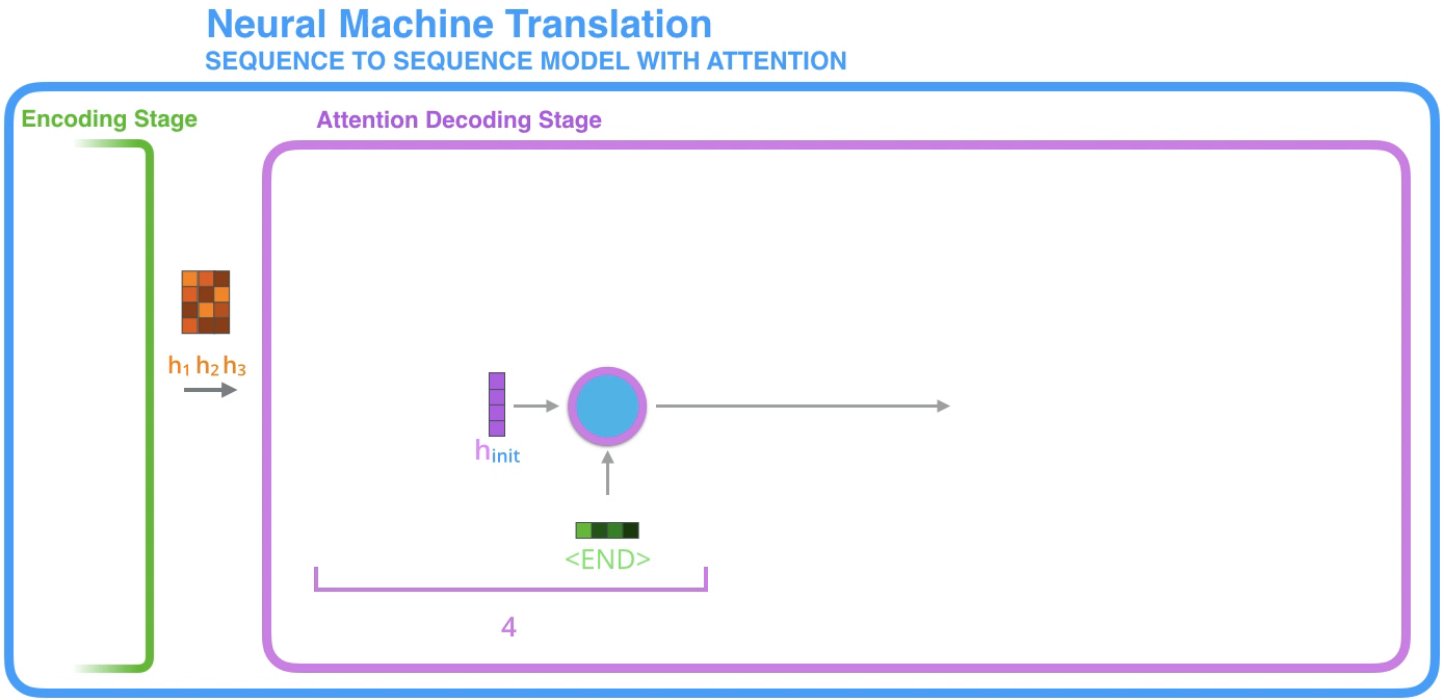


Visualizing A Neural Machine Translation Model (Mechanics of Seq2seq Models With Attention): https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/

Giving Attention to the Right Inputs

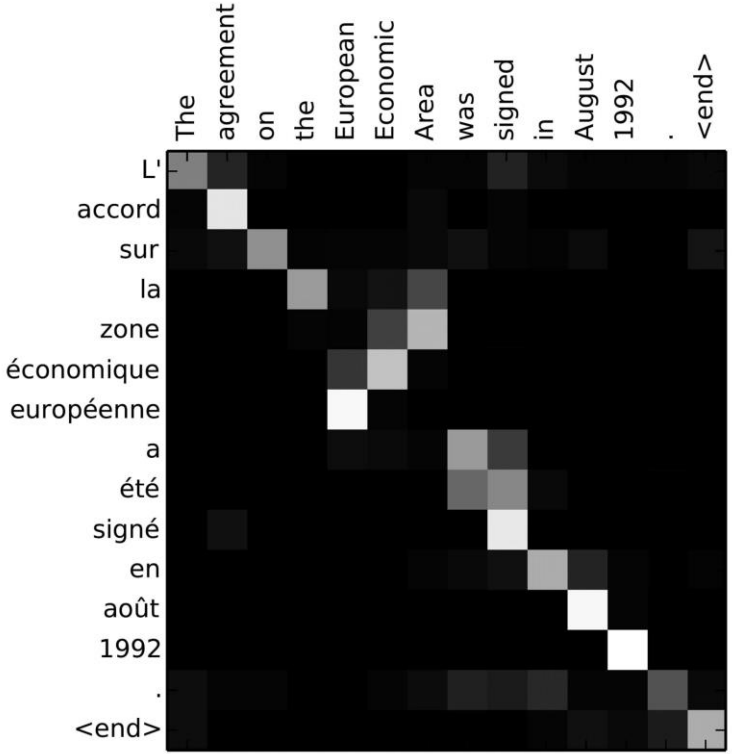
Visualizing A Neural Machine Translation Model (Mechanics of Seq2seq Models With Attention): https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/

Encoder-Decoder with Attention



Visualizing A Neural Machine Translation Model (Mechanics of Seq2seq Models With Attention): https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/

The Attention Mechanism



Source: Bahdanau et al., *Neural Machine Translation by Jointly Learning to Align and Translate*, https://arxiv.org/abs/1409.0473

A Swiss Army Knife

The Attention Mechanism is a very versatile tool, with many variations:

- Cross Attention in Machine Translation (as seen in the previous slides)
- Coordinate Attention for Computer Vision
- Self Attention (coming up!)

Issues with Sequence Models

- **Sequential Processing**
  - To encode the fifth word in the sentence, you need to have computed the Hidden States for the previous four words
  - RNNs and LSTMs are not easily parallelizable

- Struggles with very long dependencies
  - For sufficiently long sequences, even LSTMs can fail to capture relations between the start and end tokens
  - Finite context window due to linear interaction distance

- If we do not pass on the final hidden state from the encoder, then the ordering of the input gets lots despite the ross-attention
  - Cross attention alone cannot differentiate between “mr. white be careful of the black elephant” and “mr. black be careful of the white elephant”.

CS535/EE514

Machine Learning

Fall 2023

Agha Ali Raza

Transformers

Wild thoughts…

- Let’s get rid of the loop… let's not pass any information from time t to $t + 1$ of the RNN!
  - Okay… so how do we preserve the context?
  - How do we maintain ordering information?
  - What happens to our word *Smoooothie*!
- Answer:
  - …
  - …
  - … How about **self-attention** and **time-stamps**!

Positional Encodings and Self-Attention

Ordering Information

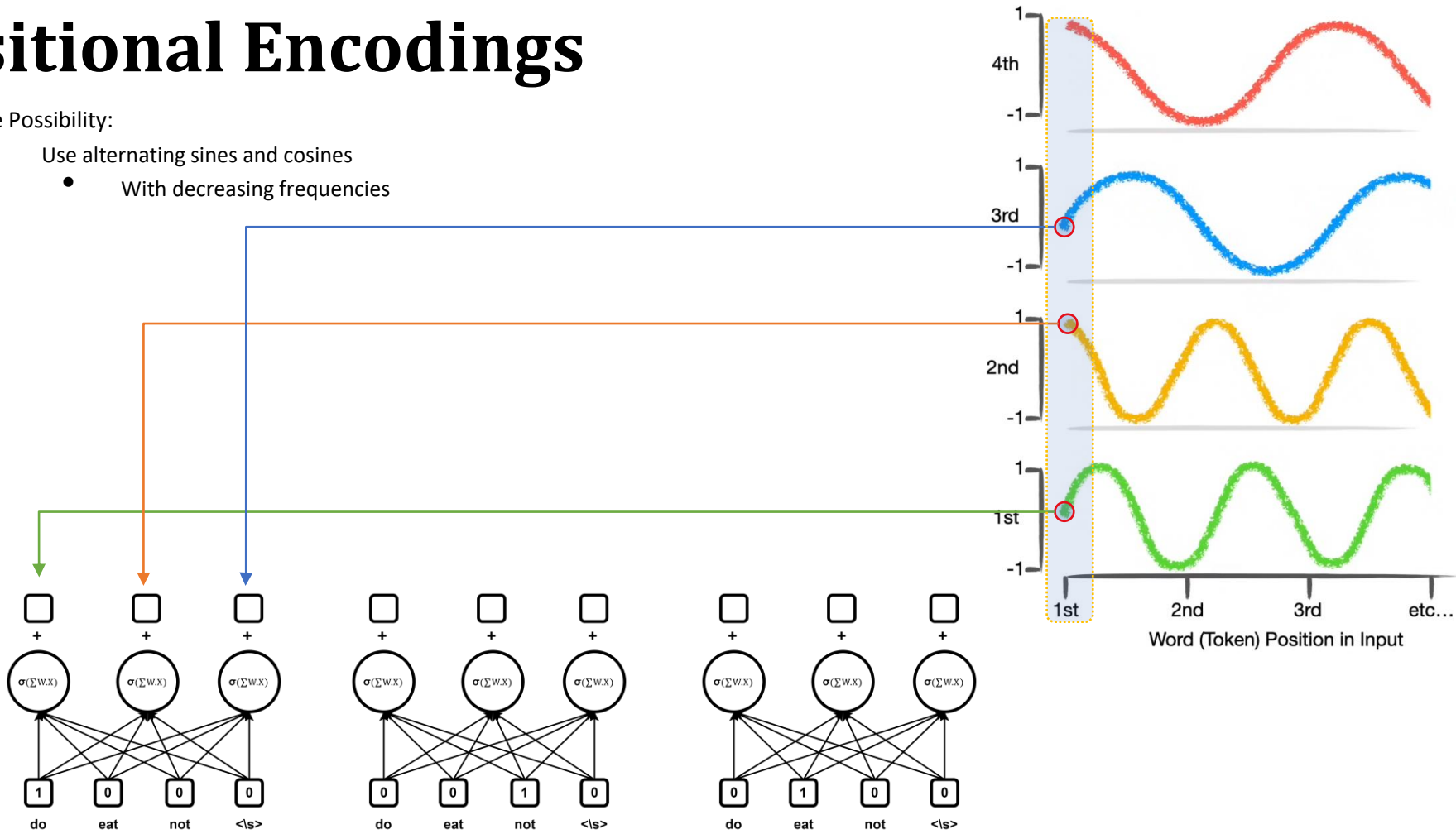
- How does a bank deal with people who do not wait in a queue
  - Tickets with time printed on them
- What if the tickets just show the value of the seconds from the clock?
  - We cannot differentiate between people who entered more than a minute apart!
    - So, we add minutes
      - And we add hours for people who entered more than an hour apart
        - Should we add days? Months? Years? Decades? Century? Millinea?
          - Well, yes if the bank is a bit slow

- All of the above are just rotating circles
  - Values that repeat after certain unique entities

- So, what kind of tickets can we give to each word in a sequence to preserve short and long-term distances?

Positional Encodings

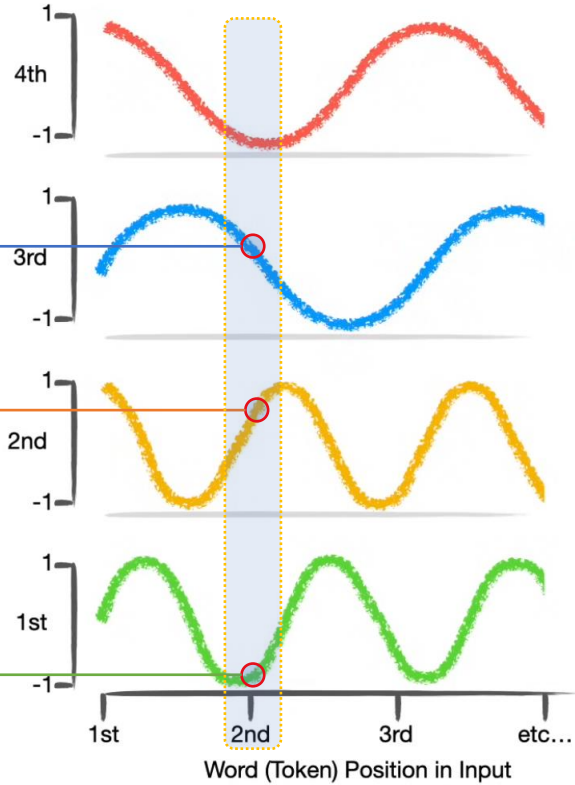
- One Possibility:
  - Use alternating sines and cosines
    - With decreasing frequencies

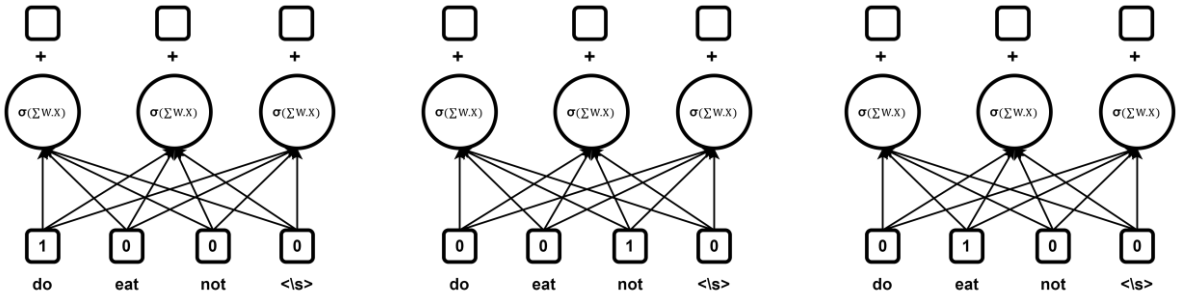


Attention for Neural Networks, Clearly Explained!!! https://www.youtube.com/watch?v=PSs6nxngL6k

Positional Encodings

- One Possibility:
  - Use alternating sines and cosines
    - With decreasing frequencies

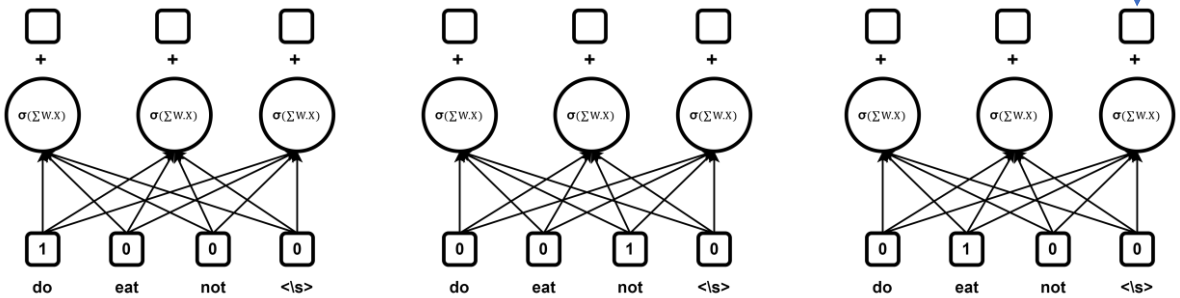


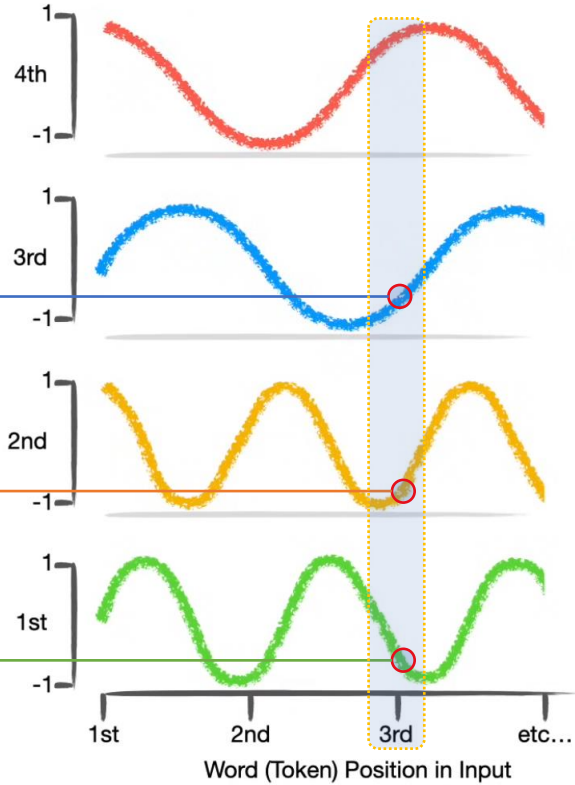


Attention for Neural Networks, Clearly Explained!!! https://www.youtube.com/watch?v=PSs6nxngL6k

Positional Encodings

- One Possibility:
  - Use alternating sines and cosines
    - With decreasing frequencies





Attention for Neural Networks, Clearly Explained!!! https://www.youtube.com/watch?v=PSs6nxngL6k

Sequence Order: Positional Encodings

- Self-Attention is natively *Permutation Invariant*: this means it is technically an operations on sets (where there is no order to the elements)
  - To incorporate *order information* to this operation, we need to explicitly add **Positional Encodings**
  - While there are multiple forms of these encodings, the 2017 paper uses *Sinusoidal Positional Encodings* (continuous unique representations for each position)
  - Periodicity of these encodings show "absolute position" isn't very important

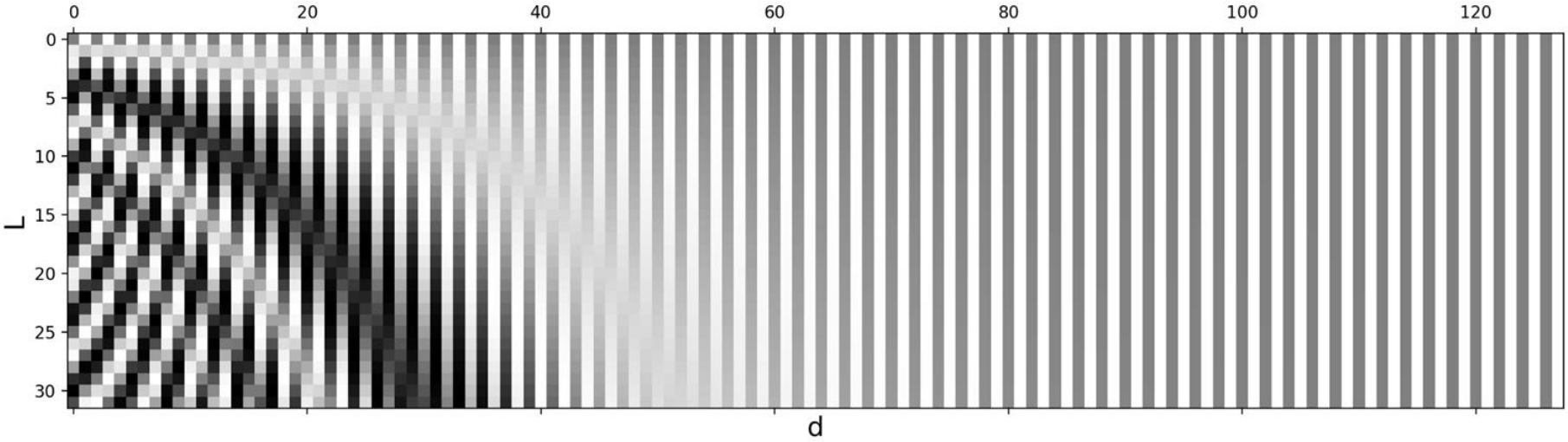
$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

  - Where d is the dimentionality of the model

Source: Lilian Weng, The Transformer Family - https://lilianweng.github.io/posts/2023-01-27-the-transformer-family-v2/

Sequence Order: Positional Encodings

Encodings for each token position (*y*-axis) and dimension (*x*-axis)



Source: Lilian Weng, The Transformer Family - https://lilianweng.github.io/posts/2023-01-27-the-transformer-family-v2/

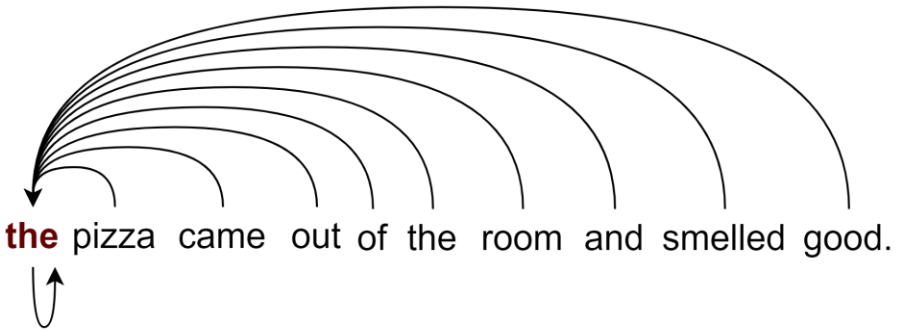
A Need for Context

- Take the sentence, “The chicken didn’t cross the street because *it* was too tired”
  - What does *it* in this sentence refer to? The chicken or the street?
  - I hate *tired streets* by the way.
- Associating an individual word with others is something very natural for humans, but difficult for machines.
  - **Self-Attention** is the mechanism with which Transformers can look at other positions in the input sequence for clues that can help lead to a better encoding for a specific word.
- RNNs use their hidden state to incorporate representation of previous words into the one it is currently processing.
  - **Self-Attention** is what Transformers use.

Source: Jay Alammar, The Illustrated Transformer - http://jalammar.github.io/illustrated-transformer/

Self-Attention

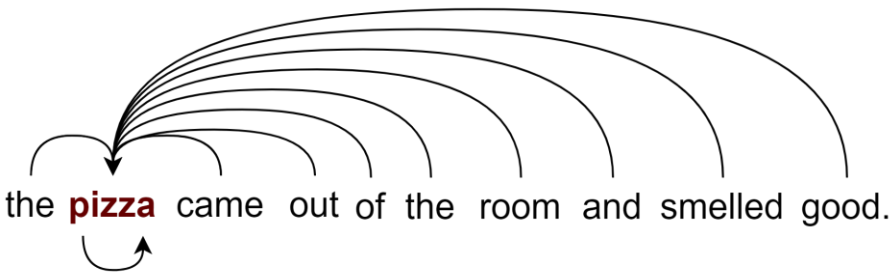
- For all of the words
  - In parallel



Attention for Neural Networks, Clearly Explained!!! https://www.youtube.com/watch?v=PSs6nxngL6k

Self-Attention

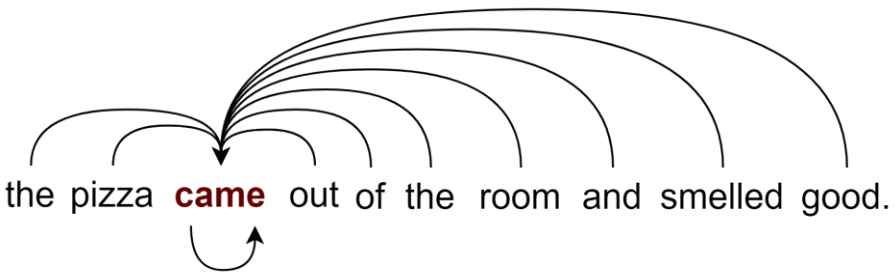
- For all of the words
  - In parallel



Attention for Neural Networks, Clearly Explained!!! https://www.youtube.com/watch?v=PSs6nxngL6k

Self-Attention

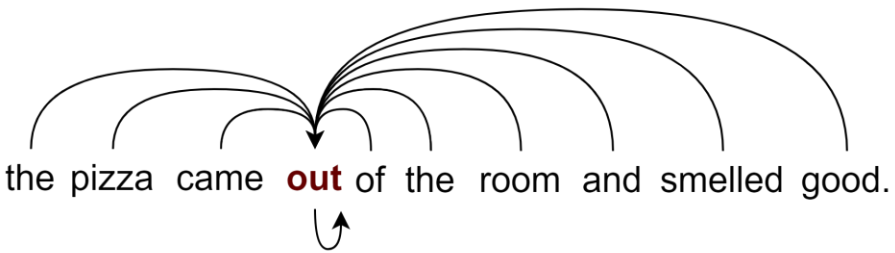
- For all of the words
  - In parallel



Attention for Neural Networks, Clearly Explained!!! https://www.youtube.com/watch?v=PSs6nxngL6k

Self-Attention

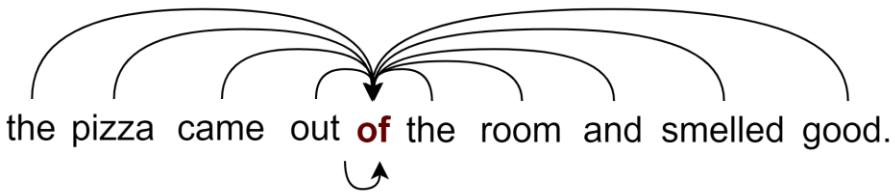
- For all of the words
  - In parallel



Attention for Neural Networks, Clearly Explained!!! https://www.youtube.com/watch?v=PSs6nxngL6k

Self-Attention

- For all of the words
  - In parallel



Attention for Neural Networks, Clearly Explained!!! https://www.youtube.com/watch?v=PSs6nxngL6k

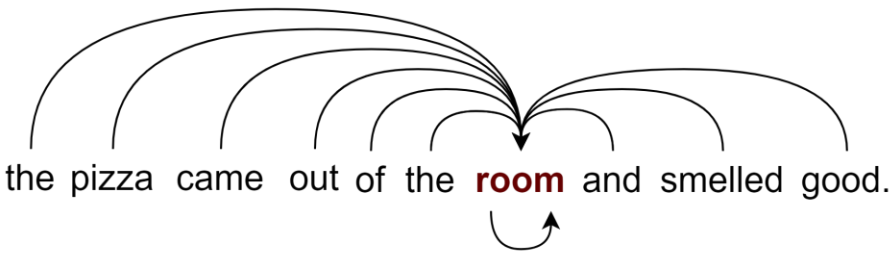
Self-Attention

- For all of the words
  - In parallel

Attention for Neural Networks, Clearly Explained!!! https://www.youtube.com/watch?v=PSs6nxngL6k

Self-Attention

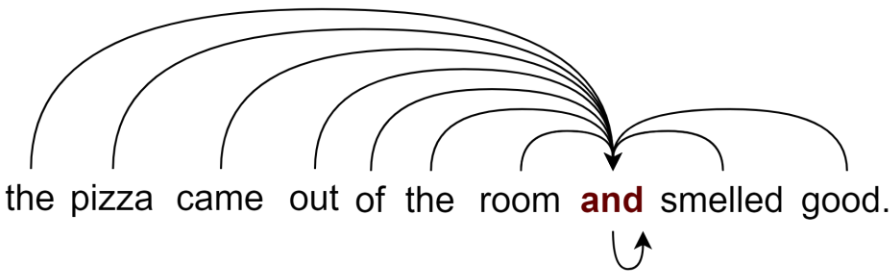
- For all of the words
  - In parallel



Attention for Neural Networks, Clearly Explained!!! https://www.youtube.com/watch?v=PSs6nxngL6k

Self-Attention

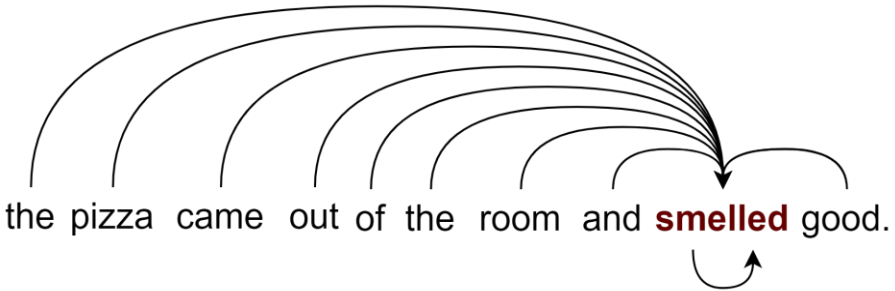
- For all of the words
  - In parallel



Attention for Neural Networks, Clearly Explained!!! https://www.youtube.com/watch?v=PSs6nxngL6k

Self-Attention

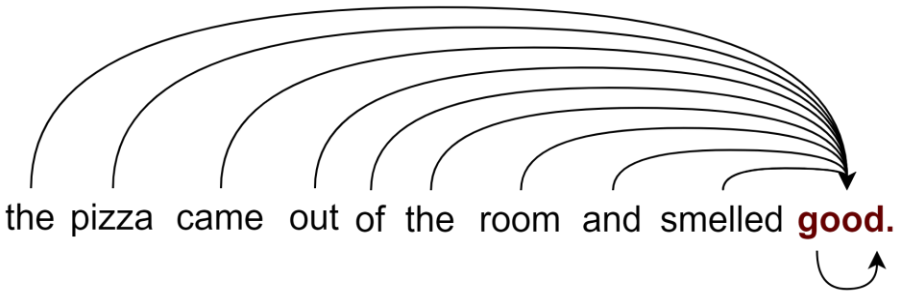
- For all of the words
  - In parallel



Attention for Neural Networks, Clearly Explained!!! https://www.youtube.com/watch?v=PSs6nxngL6k

Self-Attention

- For all of the words
  - In parallel



Attention for Neural Networks, Clearly Explained!!! https://www.youtube.com/watch?v=PSs6nxngL6k

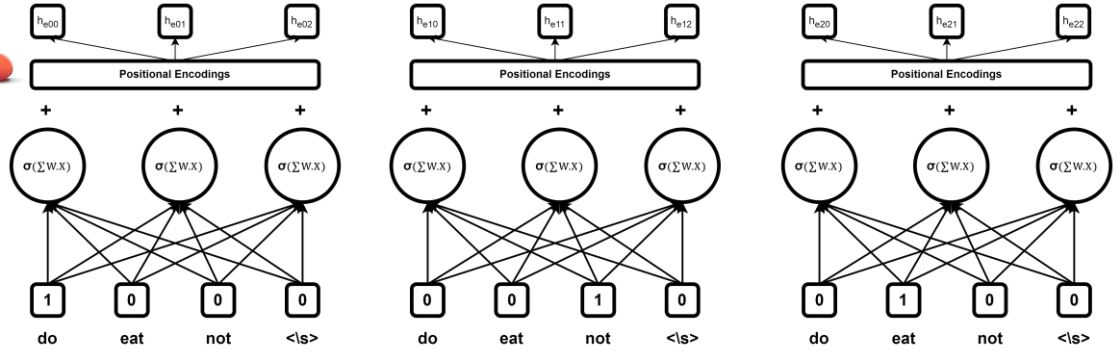
Time to match up!

I wonder if we could learn it!!!

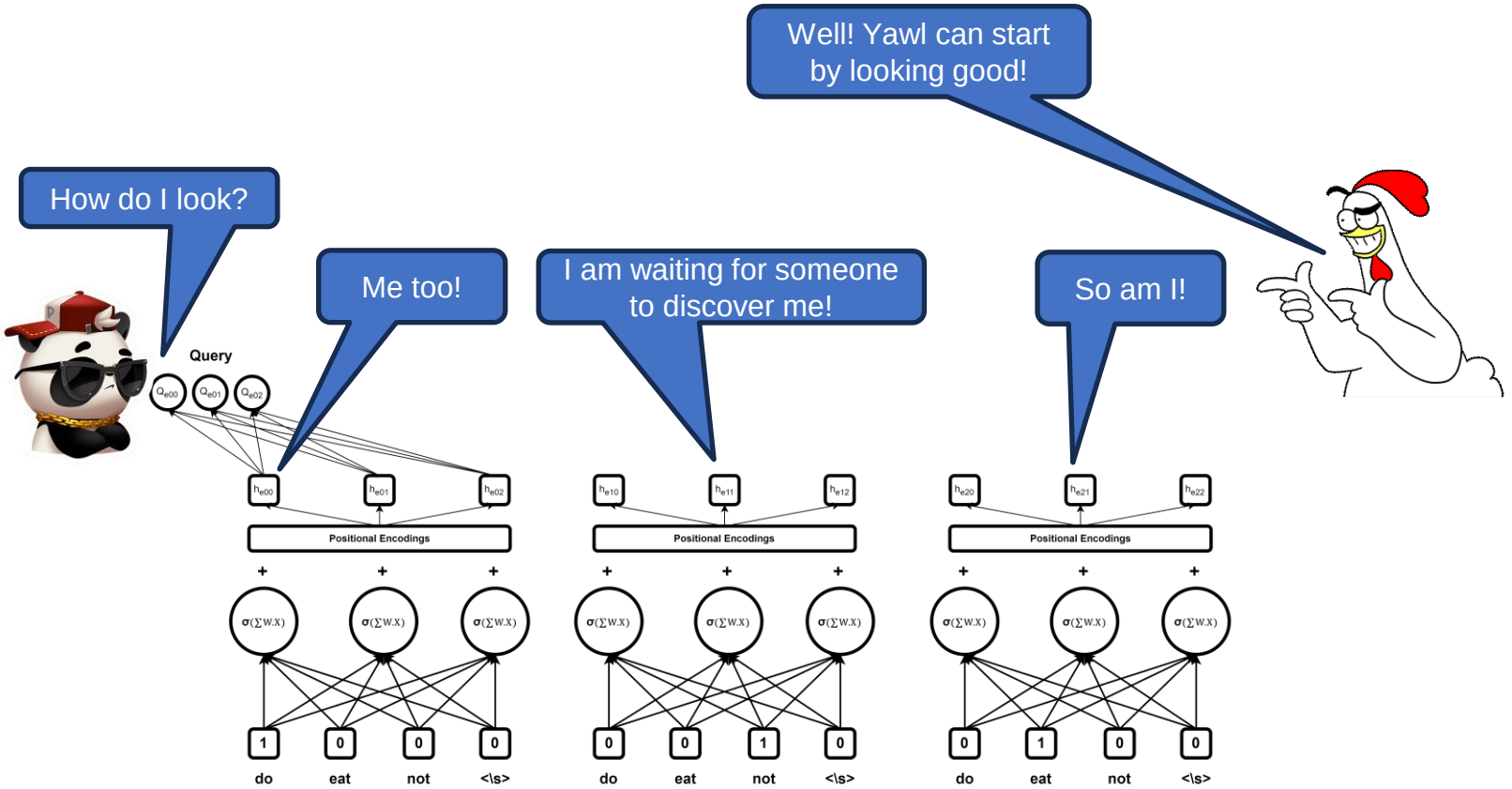
Oh no! I look terrible! What should I wear?

And *that's* what you are wearing?

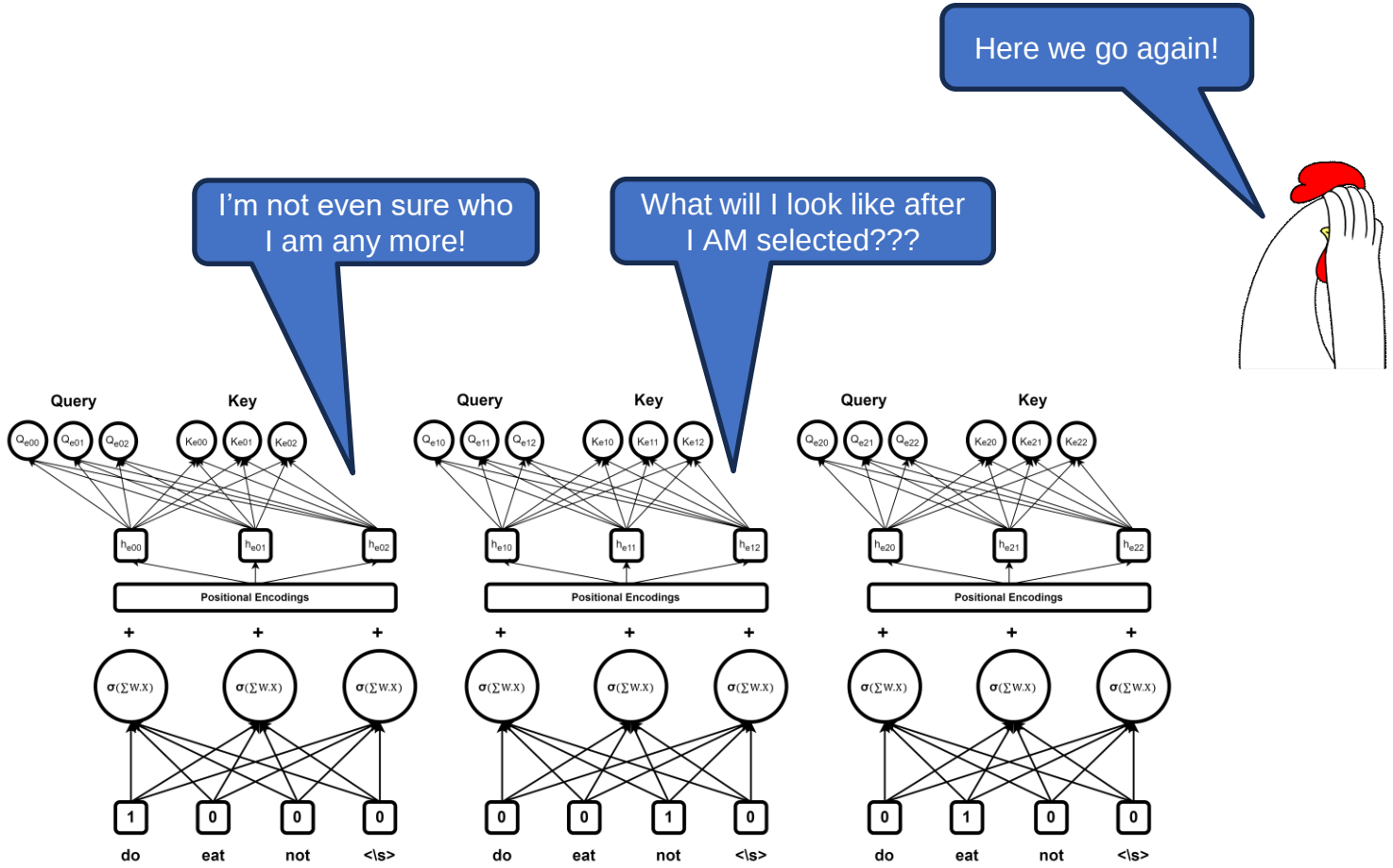
I'm looking for a match!



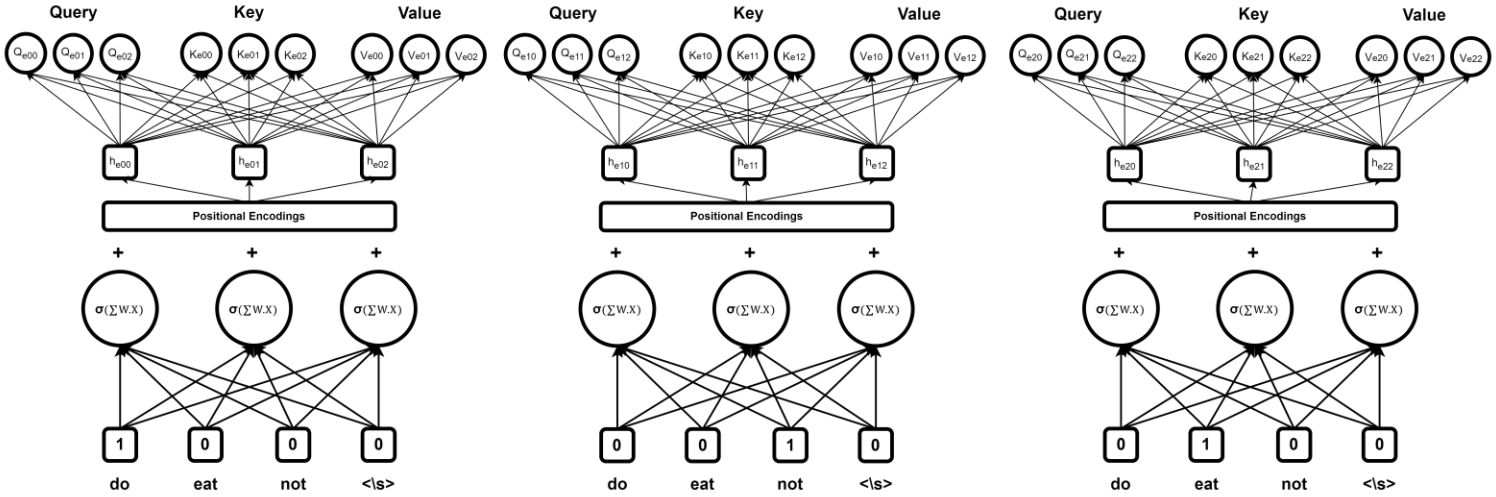
Time to match up!



Time to match up!



Time to match up!

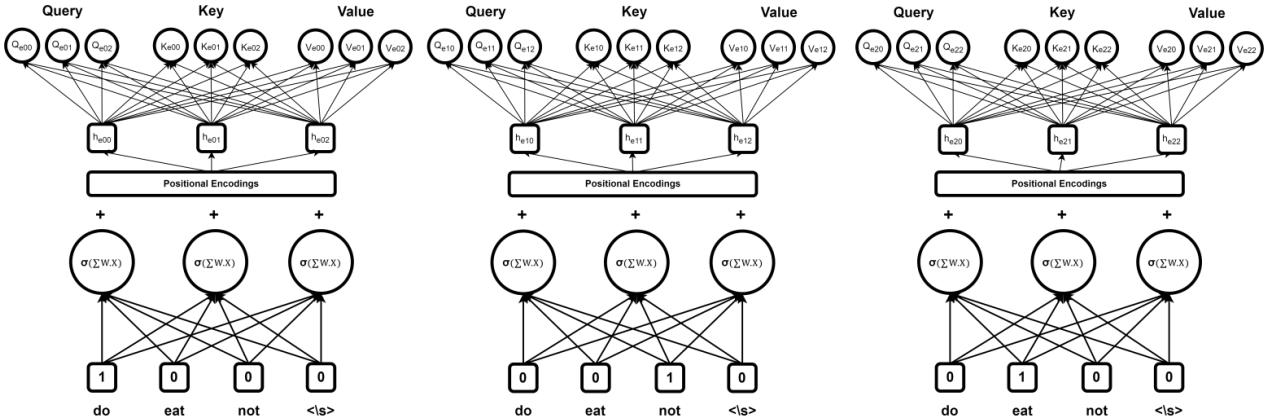


Time to match up!

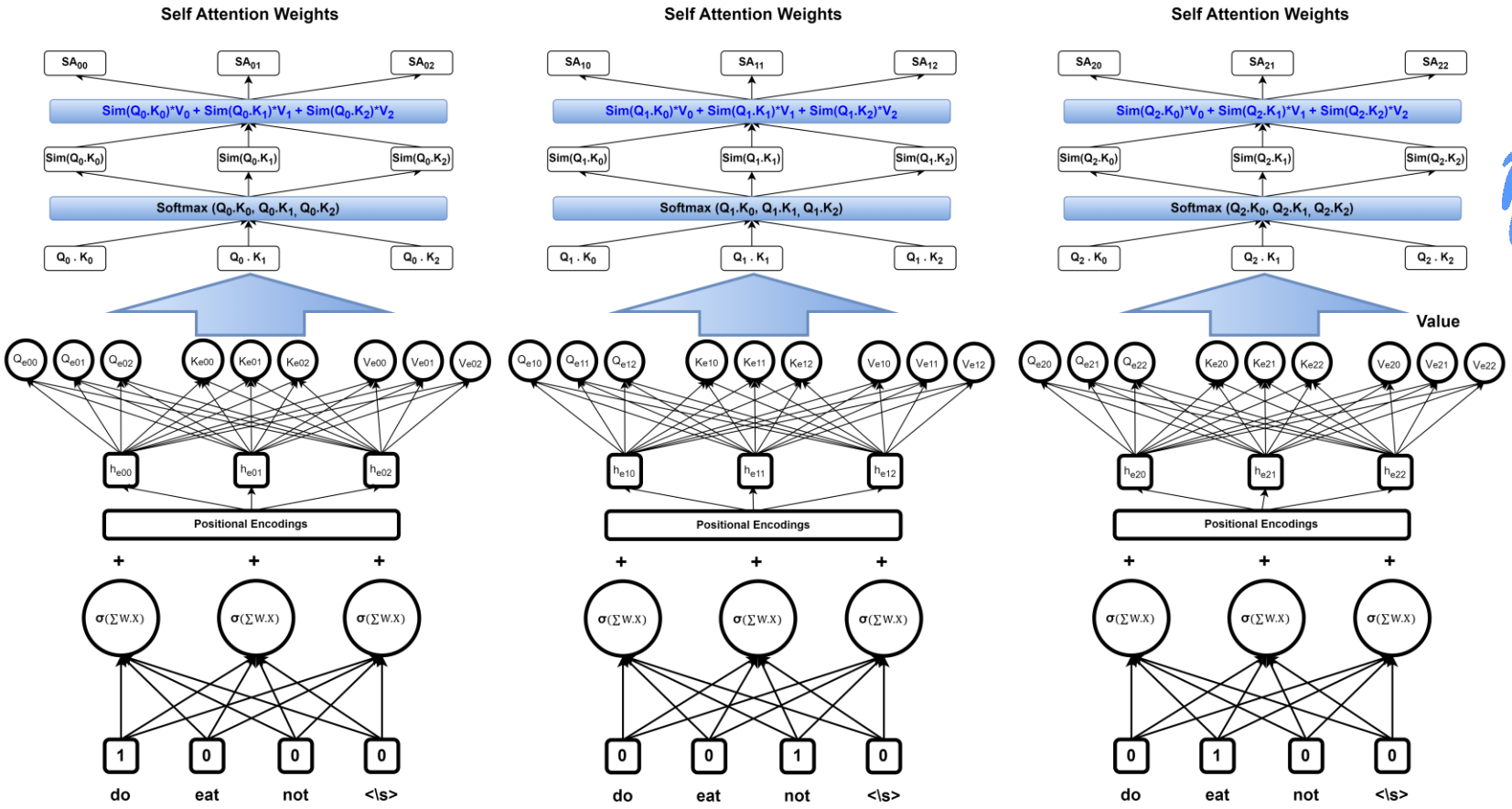
- There is **just one** of each $Weights_{Query}$, $Weights\_key$, and $Weights_{value}$ matrices for the encoder that are shared for all the words.
- The Query vector for the word will also be matched with its own Key to measure the self similarity.
- The next few steps are very similar to what we have seen for the encoder-decoder attention:

1. Find similarities using a **dot product** or **cosine similarity** – Between the query of each target words and all keys to get a **score** for each key.
2. Scale the scores for each target word using the **Soft Max** function – which preserves order and scales the weights so that they sum to 1.
3. Now, scale each key using its soft max contribution and add these all up to form the **Self Attention** vector.

- **The whole process happens in parallel for the entire encoder!**

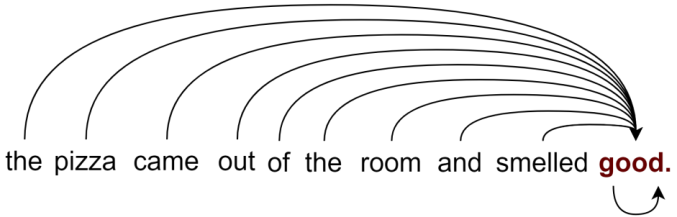


Checkpoint: Self-Attention Vectors



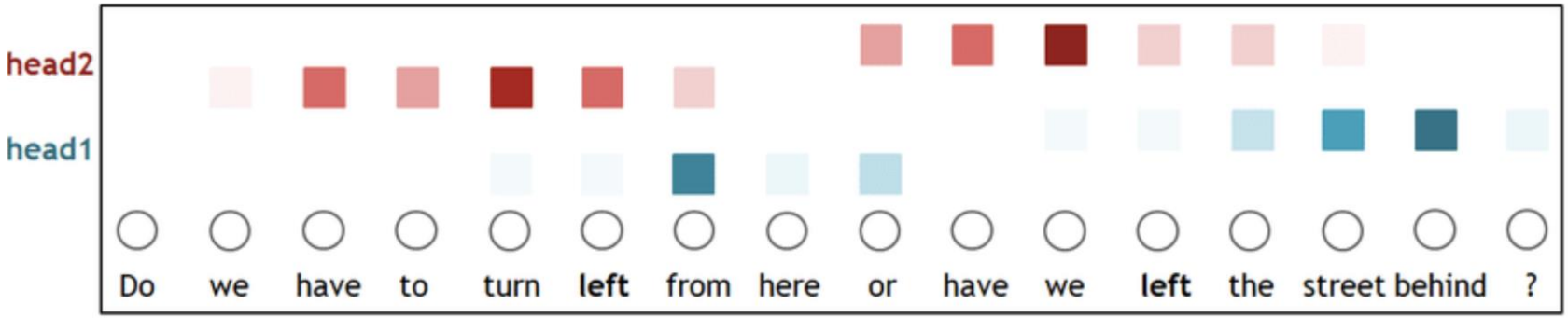
The bigger picture

- These self-attention weights establish the relationships among words in a sentence.



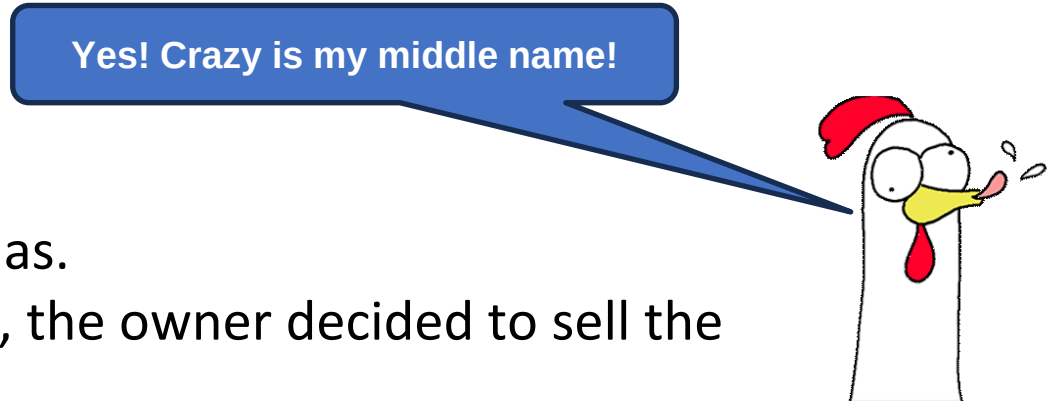
- For a complex paragraph or article where multiple layers of relationships exist among words, it makes sense to replicate these **self-attention cells** or **attention heads**
  - To capture different relationships among the words
- This is known as **multi-head attention**
- In the first paper on transformers 8 attention heads were used
  - Why 8? May be, it is their lucky number!

Multiple Attention Heads to deal with Multiple Parses



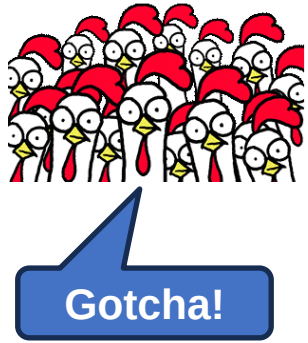
Ready to go crazy?

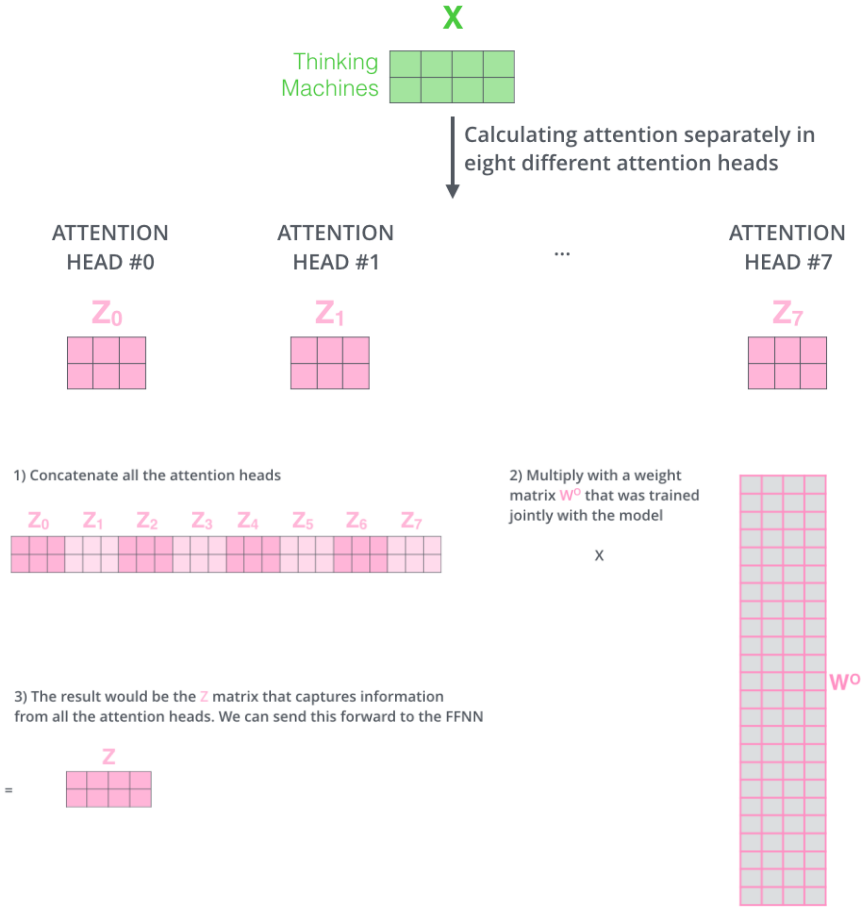
- Time flies like an arrow.
- Old men and women.
- I shot an elephant in my pajamas.
- Locked in a vault for fifty years, the owner decided to sell the jewels.
- Plunging a thousand feet into the gorge, we saw Yosemite falls.

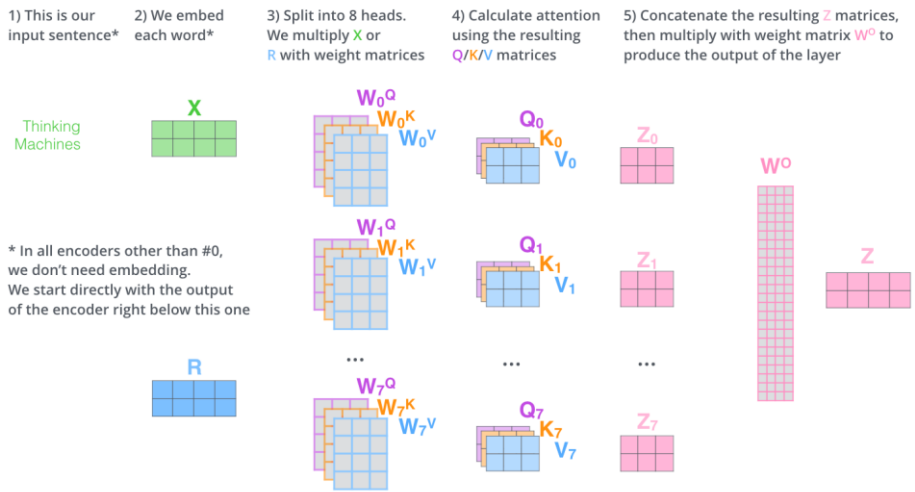


Transformers in Time-series Analysis: A Tutorial: https://www.researchgate.net/publication/360353665_Transformers_in_Time-series_Analysis_A_Tutorial

Multi-head Attention

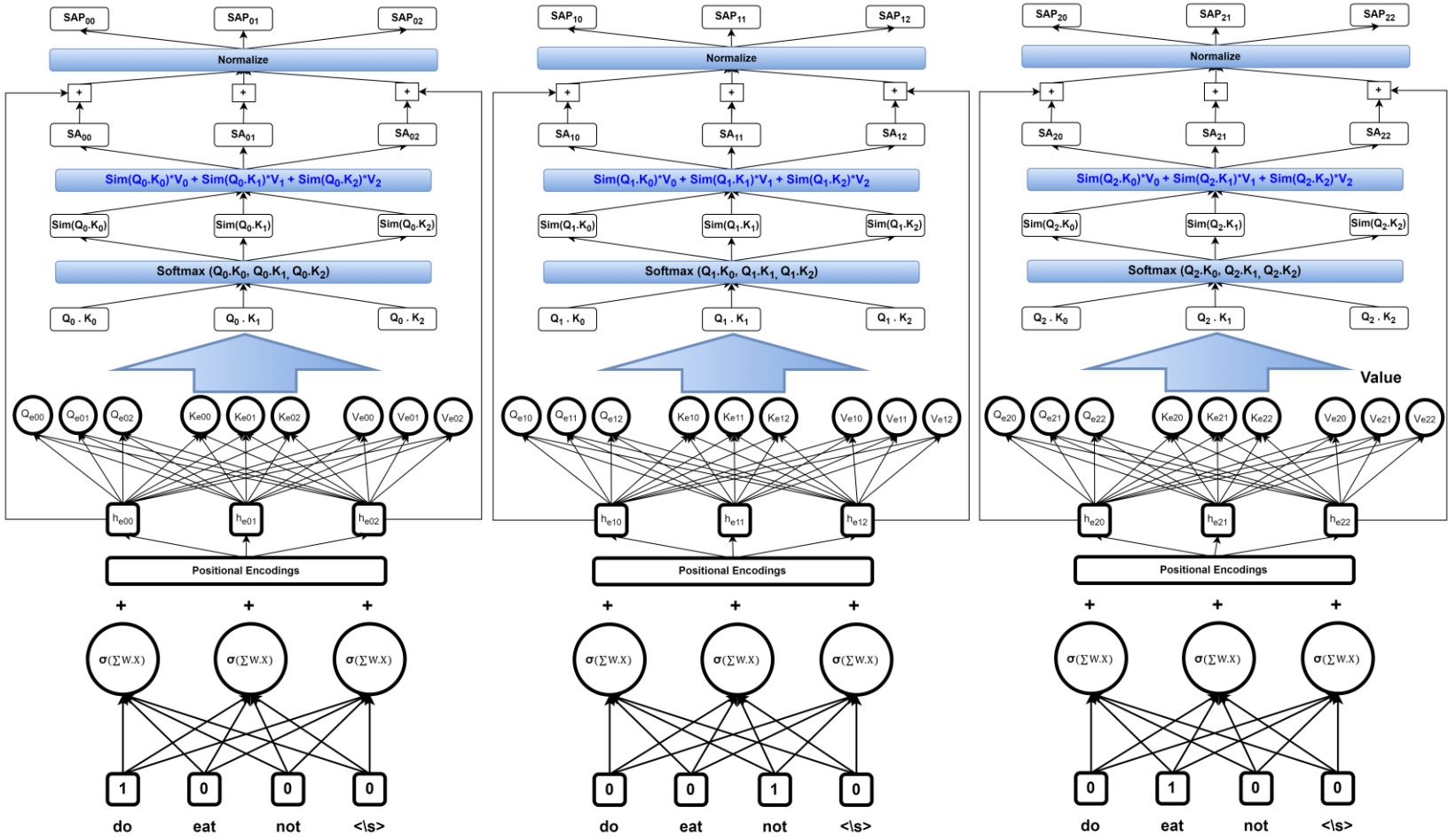






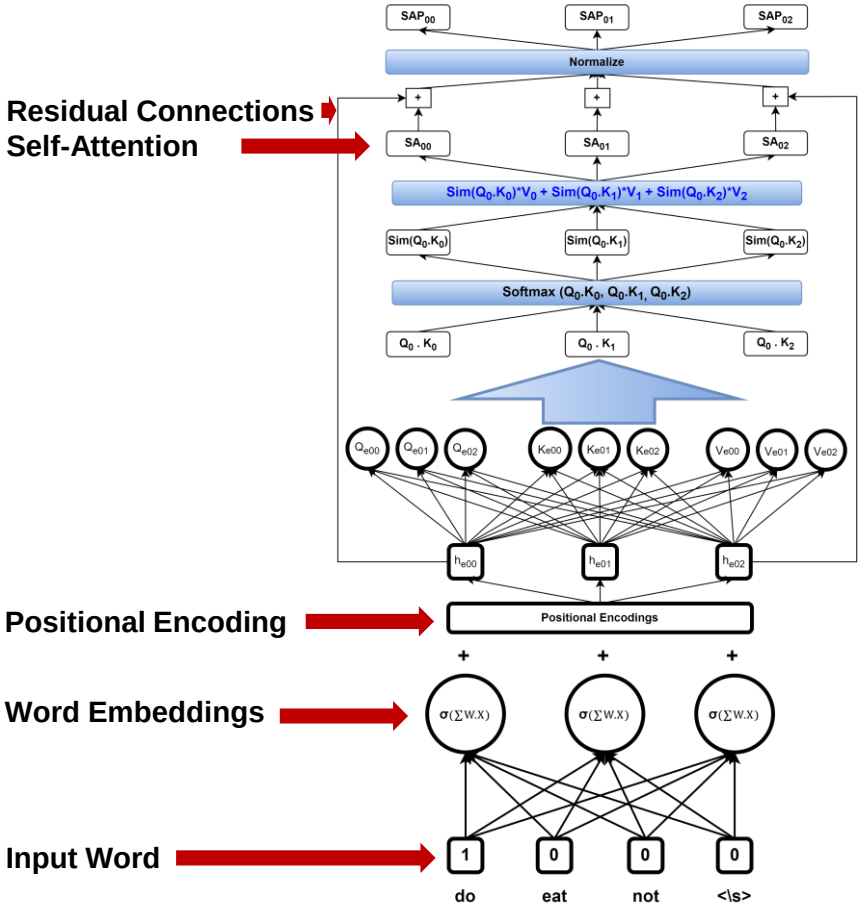
Transformers Explained Visually (Part 3): Multi-head Attention, deep dive: https://towardsdatascience.com/transformers-explained-visually-part-3-multi-head-attention-deep-dive-1c1ff1024853. The Illustrated Transformer: https://jalammar.github.io/illustrated-transformer/

The Residual Connections from Positional Embeddings

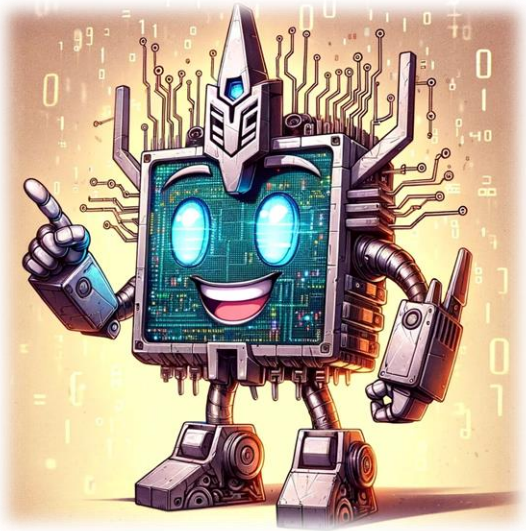


Summary

- So, the encoded values now have:
  - Word information using the word embeddings
    - Words to numbers
  - Positional Information based on Positional Embeddings
    - Encoding order
  - Contextual information based on Self-Attention
    - Encoding the relationships among words
- And, we get the ability to train in parallel



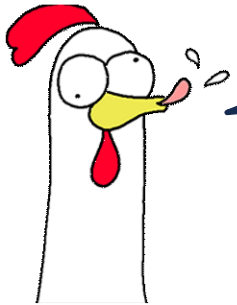
Its time to Decode!



Decoders

- Decoders have their own Self-Attention *Query, Key* and *Value* Matrices
  - As the output sentence begins, it only has itself for self-attention as we will be doing auto-regressive generation
- This self-attention will remain one-directional, as only the previous decoded states and current states have been generated at time t
  - Such one-directional behavior, when adopted by choice is called "masked self-attention".
    - Examples include training of the decoder where, in addition to **teacher forcing**, only the previous states are allowed to take part in the decode process, even though the future gold labels are also available
    - This also occurs in both inference and training of decode-only transformer architectures like GPT.
- Decoders also have a new set of *Query, Key, and Value* Matrices for Encoder-Decoder Attention

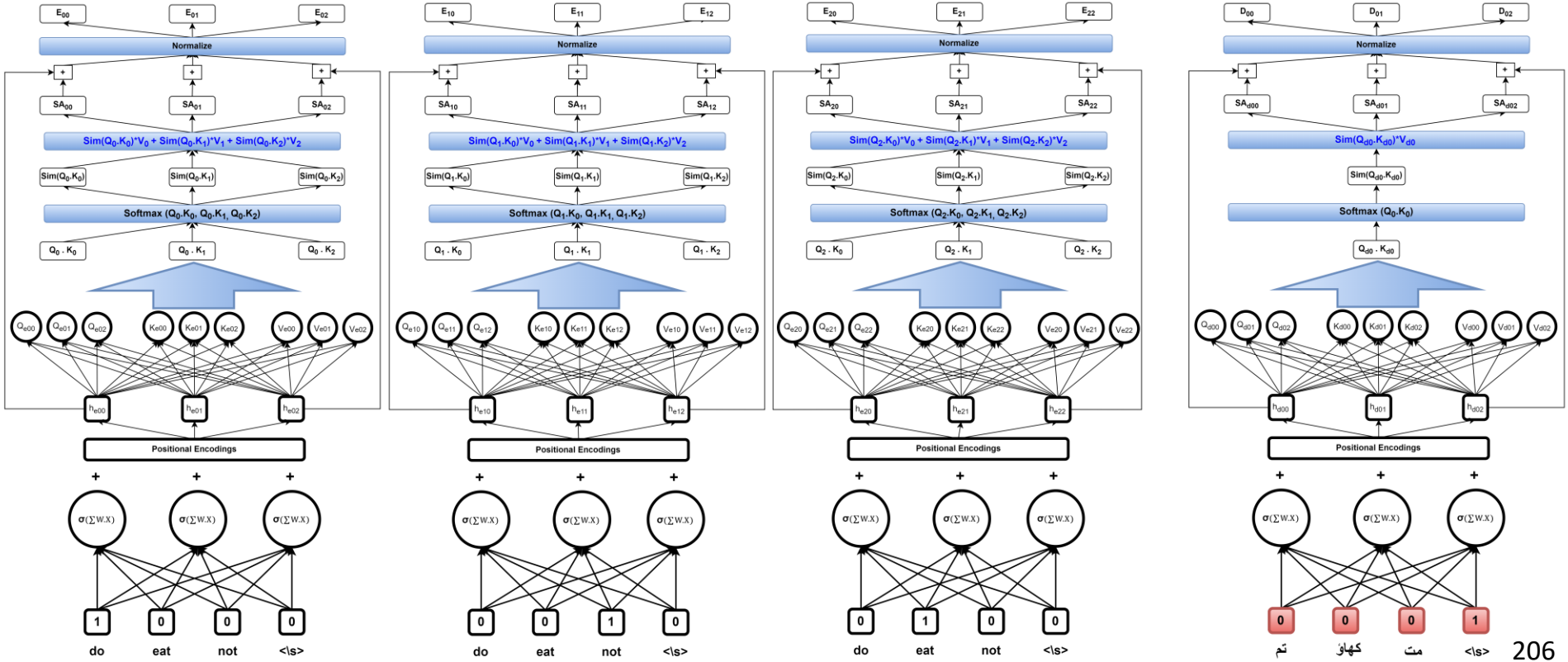
Simple… right?



Yes! Very simple!

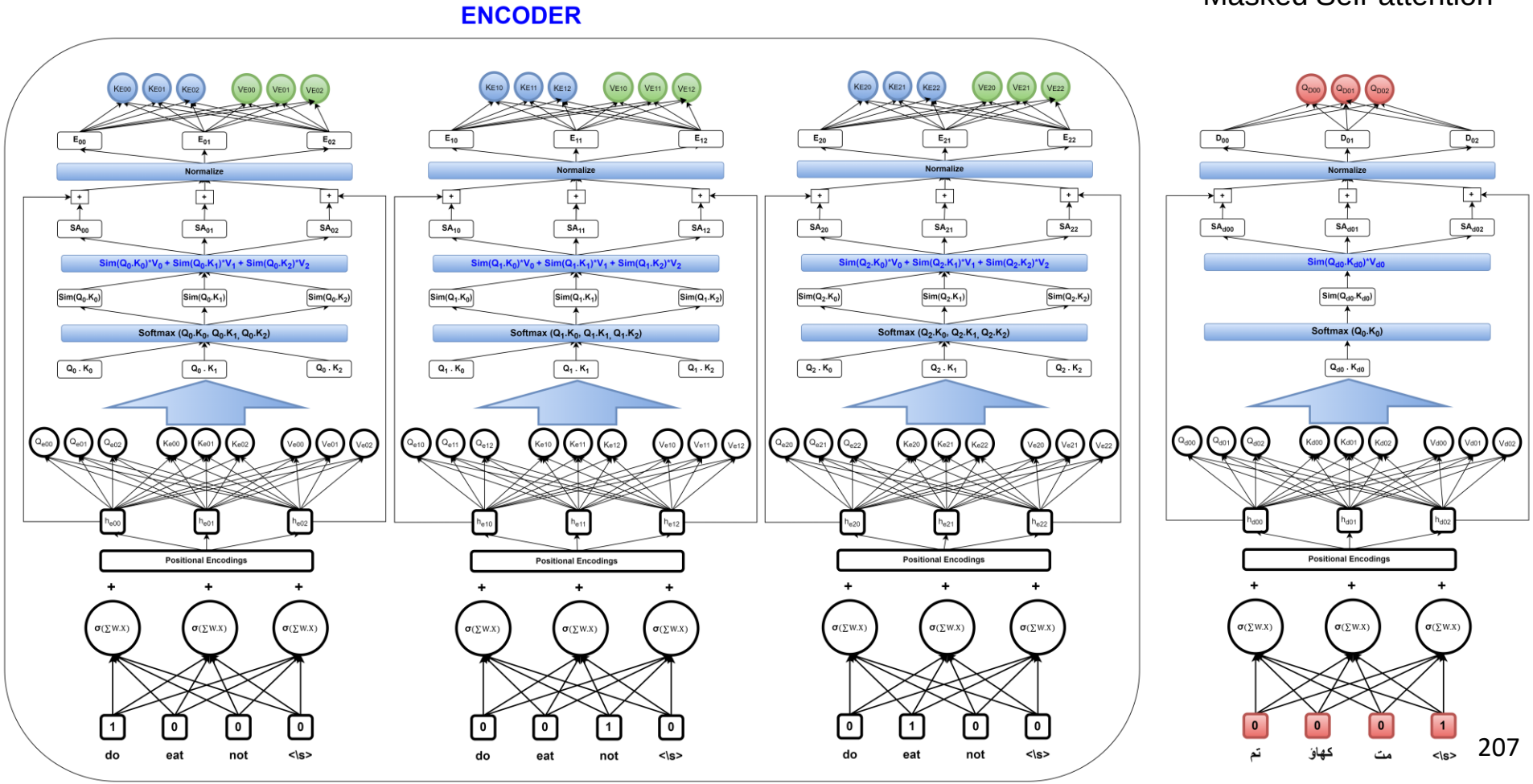
Though, I am having a hard time recalling my name now.

And… The beginning of the End!

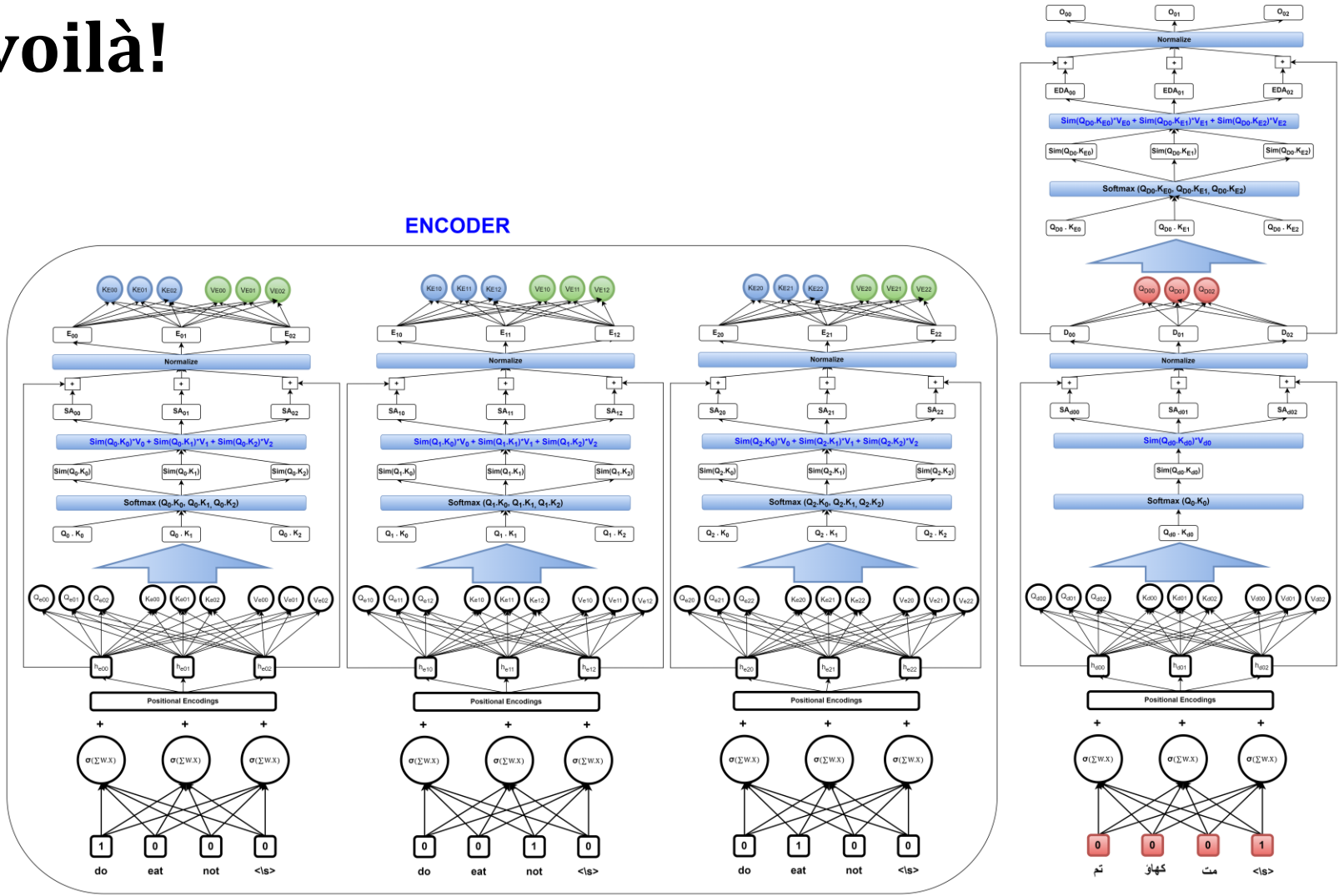


Adding in Encoder-Decoder Attention

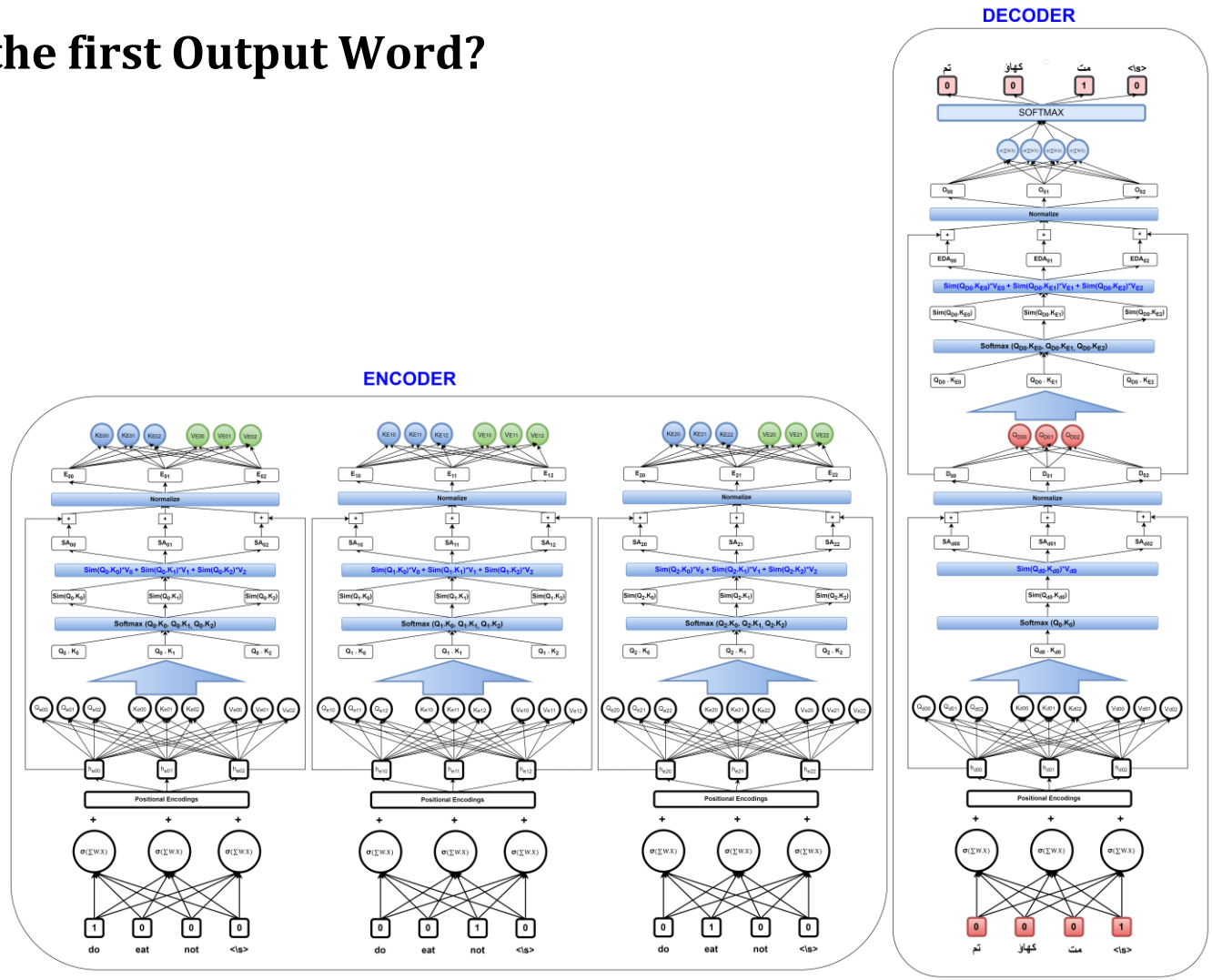
Masked Self-attention



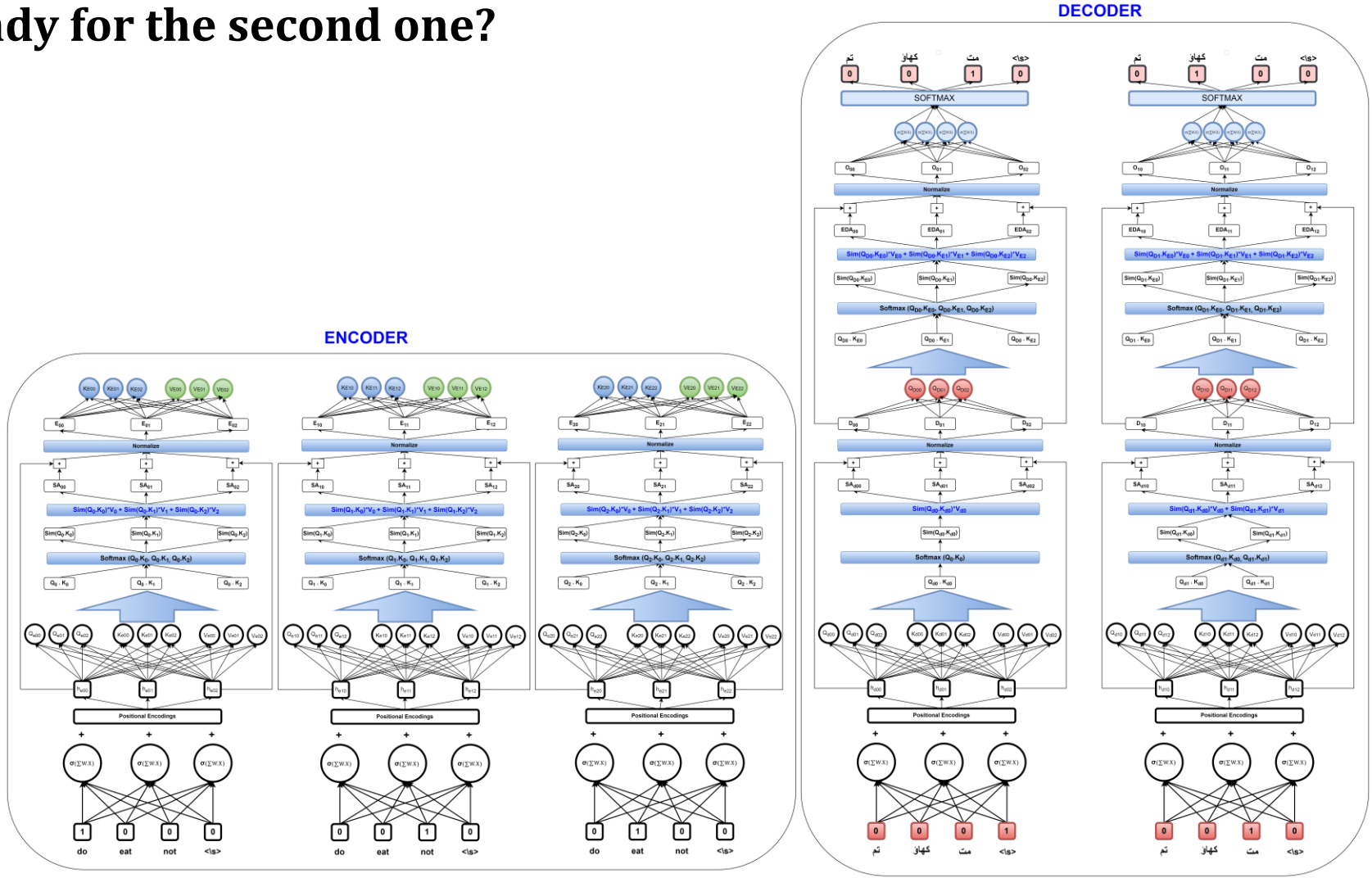
Et voilà!


ENCODER
Normalize
Positional Encodings
Softmax (Q0.K0, Q0.K1, Q0.K2)
Softmax (Q1.K0, Q1.K1, Q1.K2)
Softmax (Q2.K0, Q2.K1, Q2.K2)
Softmax (QD0.KE0, QD0.KE1, QD0.KE2)
Softmax (Qd0.Kd0)
do
eat
not
<\s>
تم
کھاو
مت
<\s>

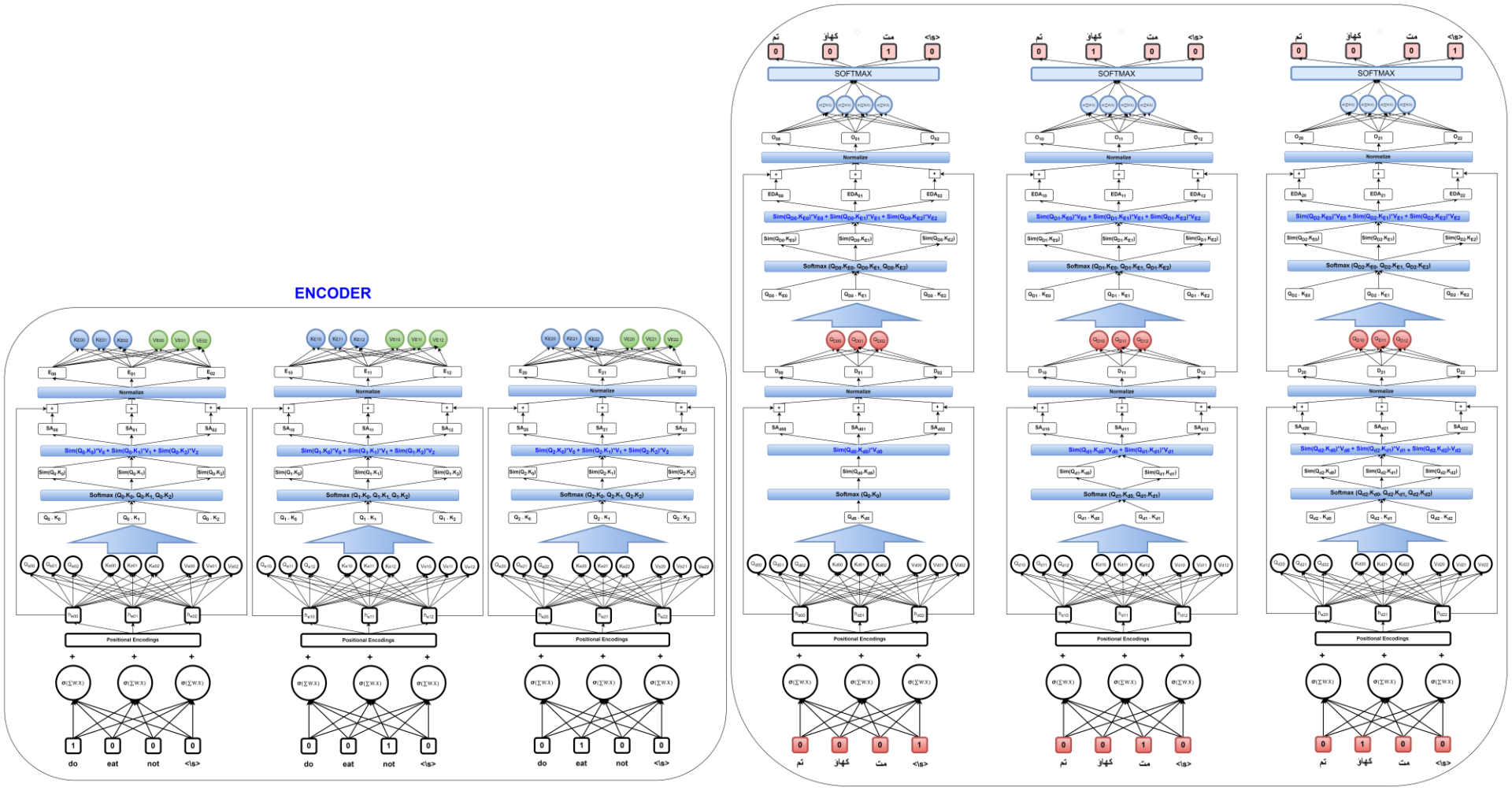
Ready for the first Output Word?



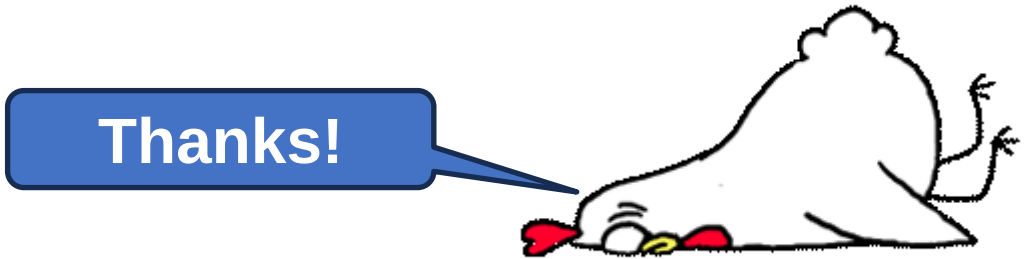
Ready for the second one?



Ready for the third one?



Congratulations! ☺



Now some small details and explanations

Notes: Queries, Keys, and Values

- In Self-Attention, we extract three types of vectors/encodings from each token in the sequence. These three have different roles:
  1. The **Query** which says “Here’s what I’m interested in…”
  2. The **Key** which says “This is what I have…”
  3. The **Value** which says “If you find me interesting, here’s what I will communicate to you…”

- Can be thought of as a search engine
  - “When you search for videos on Youtube, the search engine will map your **query** (text in the search bar) against a set of **keys** (video title, description, etc.) associated with candidate videos in their database, then present you the best matched videos (**values**)”

Source: What exactly are Keys, Queries, and Values in Attention? - https://stats.stackexchange.com/questions/421935/what-exactly-are-keys-queries-and-values-in-attention-mechanisms

Notes: Self-Attention: Getting Q, K, and V

- Recall that we usually represent words in the form of word embeddings:

$$x_1, x_2, \ldots, x_T \text{ where } x_i \in \mathbb{R}^d$$

- The three vectors are extracted by (learnable) projections:

$$k_i = W^K x_i, W^K \in \mathbb{R}^{d \times d}$$
$$q_i = W^Q x_i, W^Q \in \mathbb{R}^{d \times d}$$
$$v_i = W^V x_i, W^V \in \mathbb{R}^{d \times d}$$

Source: Lilian Weng, The Transformer Family - https://lilianweng.github.io/posts/2023-01-27-the-transformer-family-v2/

Notes: Self-Attention: Dot-Product

- Similar to the search engine analogy, when we are looking for a good match, we focus on the **query** and the **key**
  - It only makes sense to look through *all the key vectors* when dealing with an *individual query vector* (imagine if Google only searched through one webpage for your query!)
  - Since we are dealing with vectors, we can compute a compatibility/affinity score with *dot products*

$$q_1 \cdot k_1$$
$$q_1 \cdot k_2$$
$$\vdots$$
$$q_1 \cdot k_T$$

Source: Lilian Weng, The Transformer Family - https://lilianweng.github.io/posts/2023-01-27-the-transformer-family-v2/

Notes: Self-Attention: Vectorizing our Approach

- Before diving deeper, this would be a good point to shift from individual tokens to dealing with the sequences as a whole
  - Vectorizing the approach makes the equations much cleaner

- Instead of dealing with individual input vectors, we can concatenate all of them to get one large matrix; similarly, we could get Query, Key, and Value matrices in one clean expression

$$X = [x_1; \ldots; x_T] \in \mathbb{R}^{T \times d}$$

$$K = XW^K, W^K \in \mathbb{R}^{d \times d}$$
$$Q = XW^Q, W^Q \in \mathbb{R}^{d \times d}$$
$$V = XW^V, W^V \in \mathbb{R}^{d \times d}$$

Source: Lilian Weng, The Transformer Family - https://lilianweng.github.io/posts/2023-01-27-the-transformer-family-v2/

Notes: Self-Attention: Vectorizing our Approach II

- Now that we have Query, Key, and Value matrices, we can get the affinity scores for all pairs in one go with a matrix multiplication

$$QK^T \in \mathbb{R}^{T \times T}$$

Source: Lilian Weng, The Transformer Family - https://lilianweng.github.io/posts/2023-01-27-the-transformer-family-v2/

Self-Attention: Scores to Weights

- Since we have (affinity) scores for each of the keys to this given query, they should be normalized to be interpreted as weights (for a weighted average later)
  - Recall for Classification, we pass the logits of a Neural Network through a Softmax to get probabilities
  - Extra step: divide the scores by the dimensionality (# embedding dimensions) of the key vector (helps with stable gradients, otherwise the Softmax input would be very high)

$$\text{softmax}(\frac{QK^T}{\sqrt{d_k}})$$

Source: Lilian Weng, The Transformer Family - https://lilianweng.github.io/posts/2023-01-27-the-transformer-family-v2/

Notes: Self-Attention: Multiply with Values

- Now with the scores normalized to a sensible range, we can begin to *aggregate* information from the other tokens into our query token, based off the scores we found
  - This is just a matrix multiplication with the Value matrix
  - Note the shape of this result being the same as that of the input matrix

$$H = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V \in \mathbb{R}^{T \times d}$$

Source: Lilian Weng, The Transformer Family - https://lilianweng.github.io/posts/2023-01-27-the-transformer-family-v2/

A few extras

- The original paper had 37,000 tokens
  - And longer inputs and outputs
- So, to make it work, you have to normalize often
  - The original paper normalizes after positional encodings and after self-attention.

- Also, to give the Transformer more weights and biases to deal with complicated data, they added additional neural networks with hidden layers to both the encoders and decoders.

Let’s Go over it all again!

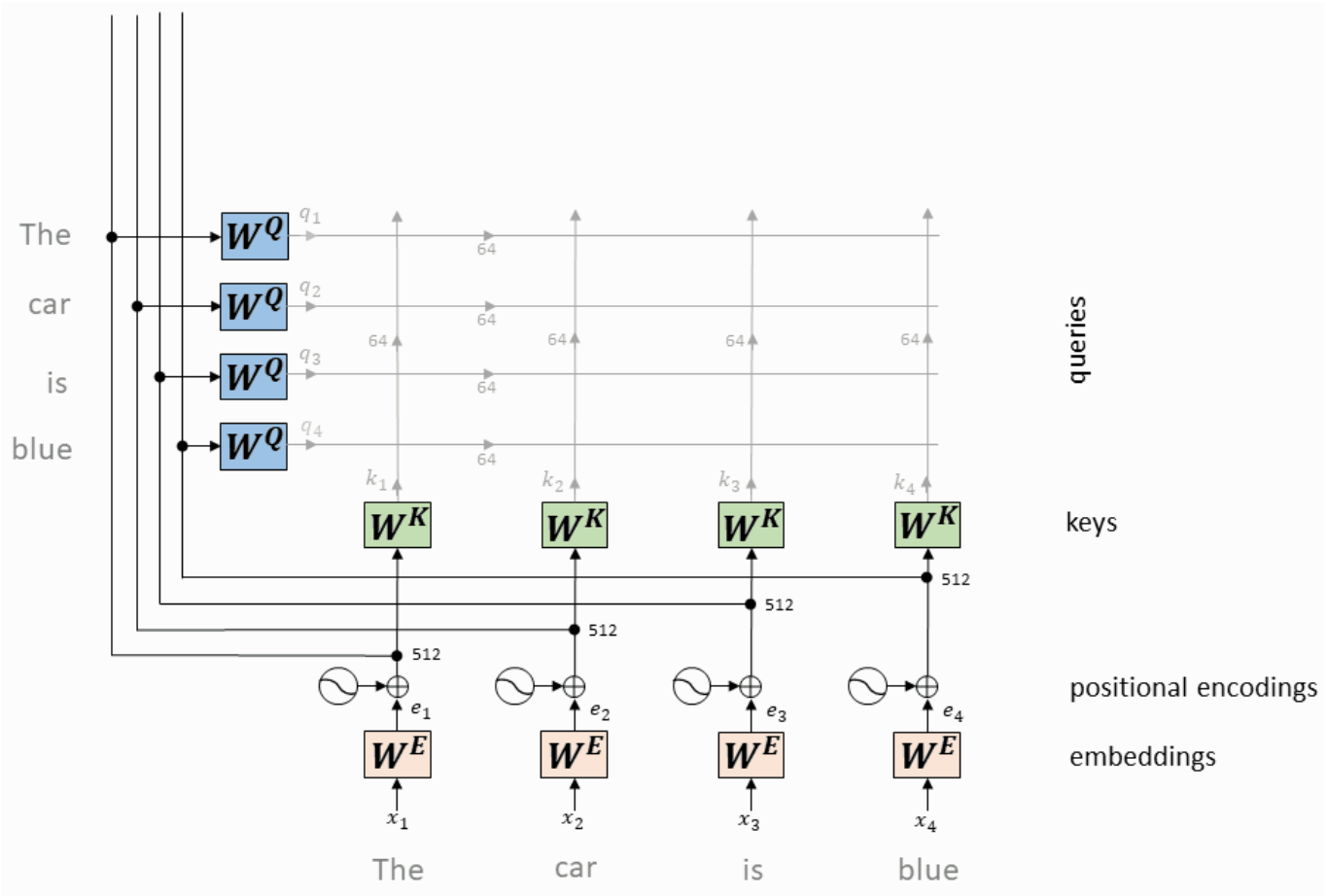
In animations

From the Input Sentence to Queries and Keys

The

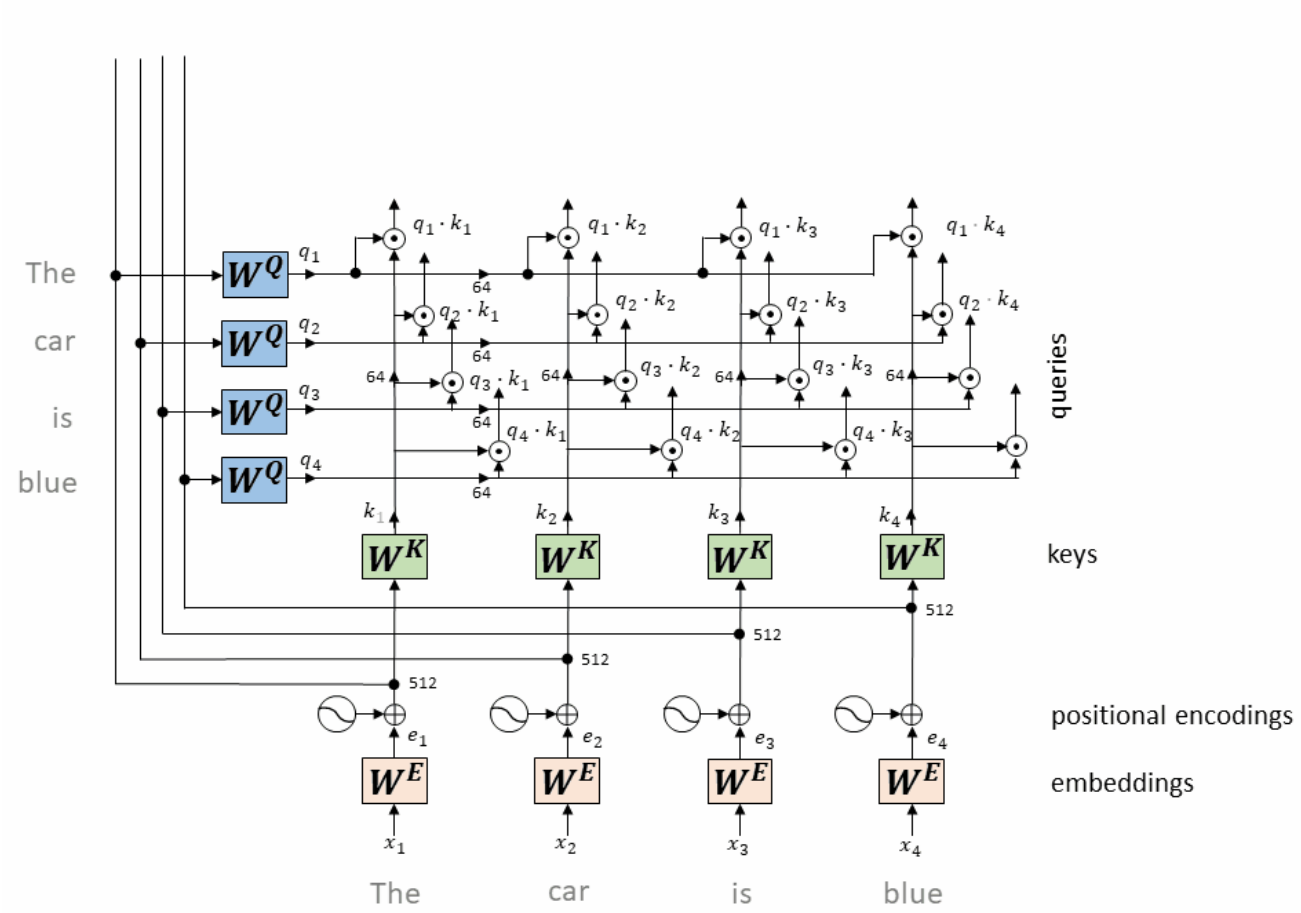
Drawing the Transformer Network from Scratch (Part 1): https://towardsdatascience.com/drawing-the-transformer-network-from-scratch-part-1-9269ed9a2c5e

Dot products of Queries and Keys



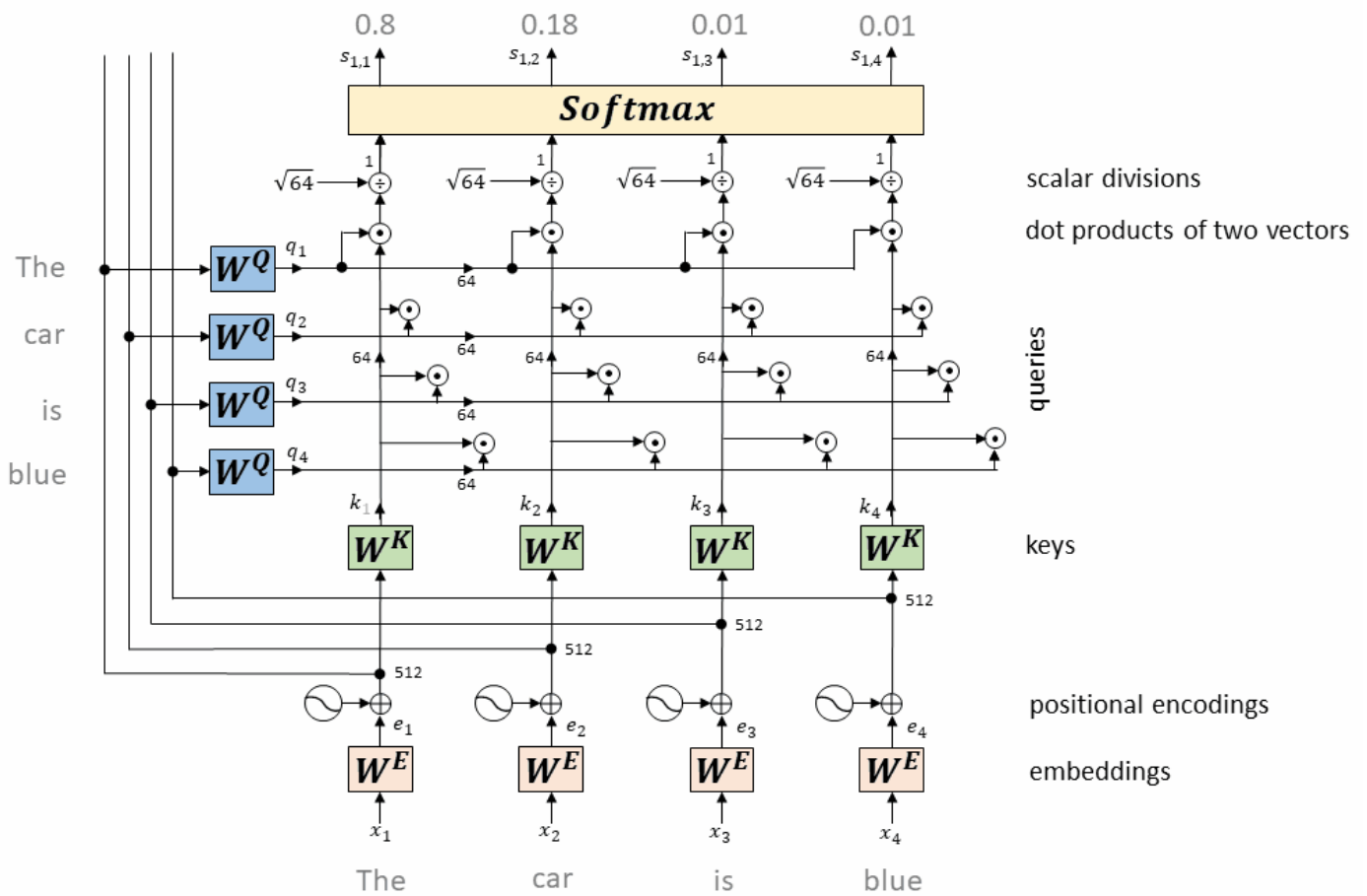
Drawing the Transformer Network from Scratch (Part 1): https://towardsdatascience.com/drawing-the-transformer-network-from-scratch-part-1-9269ed9a2c5e

Scaling and Softmax



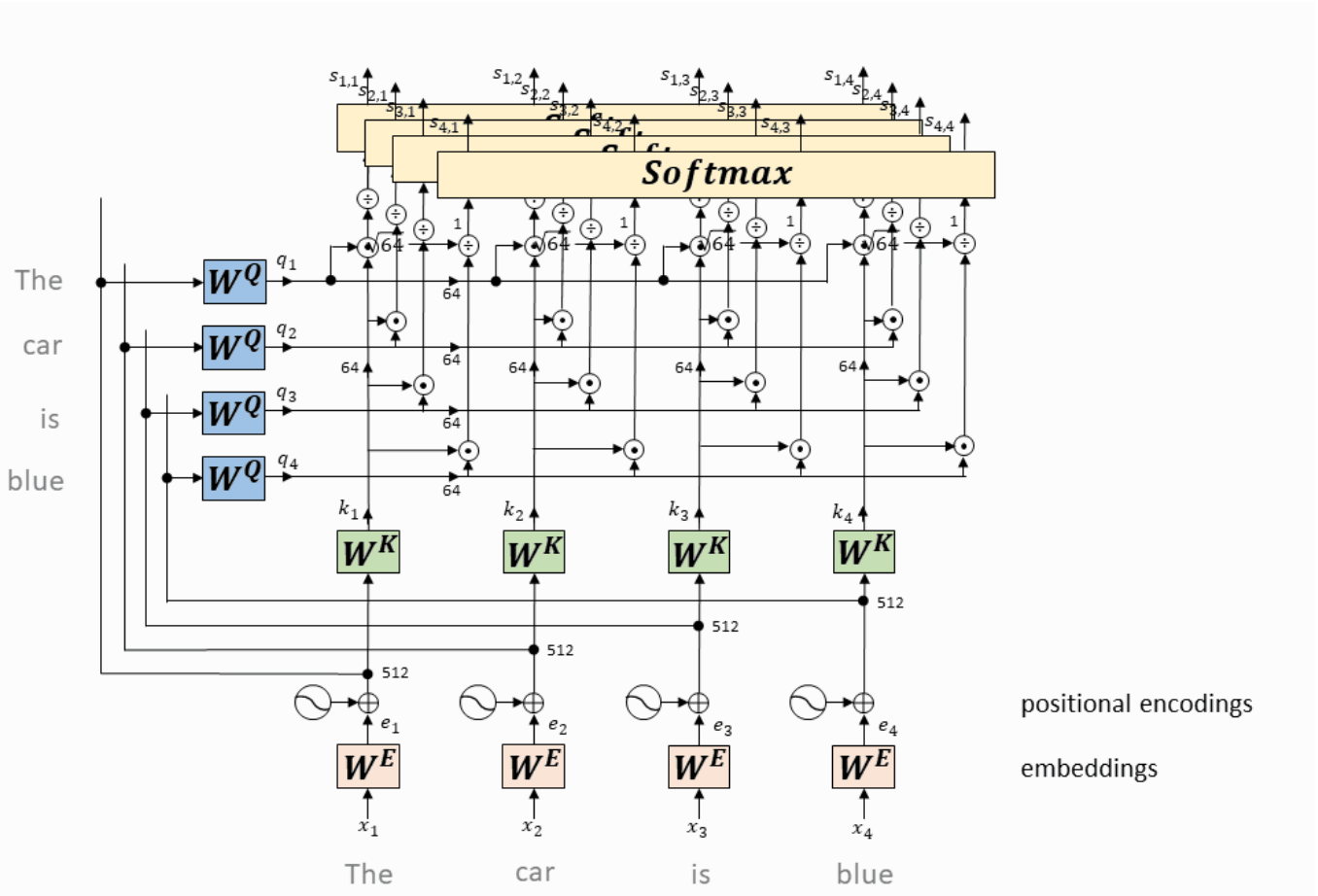
Drawing the Transformer Network from Scratch (Part 1): https://towardsdatascience.com/drawing-the-transformer-network-from-scratch-part-1-9269ed9a2c5e

Scaling and softmax of the weights belonging to the remaining words: “car”, “is” and “blue”



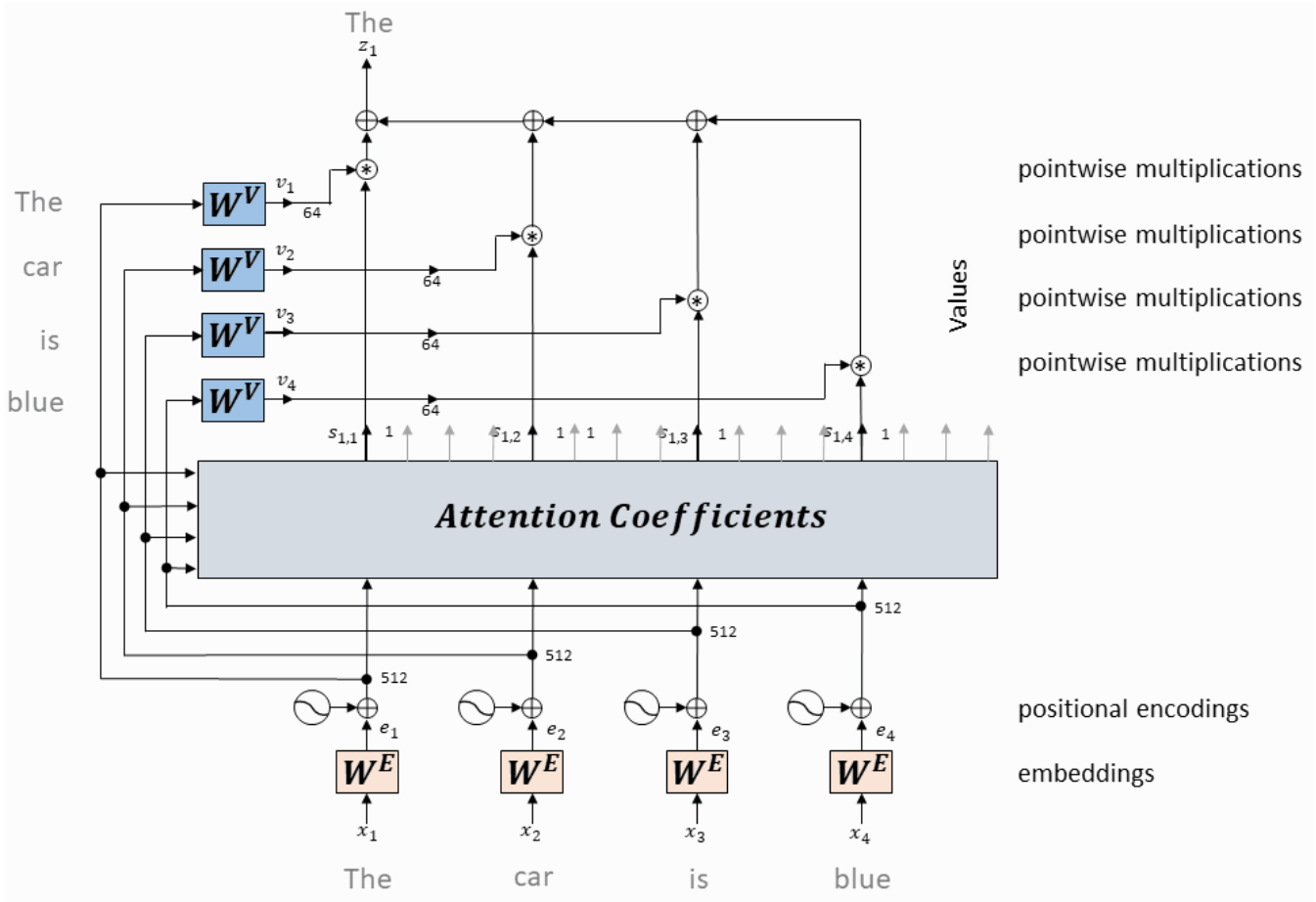
Drawing the Transformer Network from Scratch (Part 1): https://towardsdatascience.com/drawing-the-transformer-network-from-scratch-part-1-9269ed9a2c5e

Values, Weighting and Summation for the first word “The”



Drawing the Transformer Network from Scratch (Part 1): https://towardsdatascience.com/drawing-the-transformer-network-from-scratch-part-1-9269ed9a2c5e

Values, Weighting and Summation for the remaining words: “car”, “is” and “blue”



Drawing the Transformer Network from Scratch (Part 1): https://towardsdatascience.com/drawing-the-transformer-network-from-scratch-part-1-9269ed9a2c5e

First Stop

- That concludes the self-attention calculation. The output of the self-attention layer can be considered as a context enriched word embedding. Depending on the context, a word can have different meanings:
  - I like fresh, crisp **fall** weather.
  - Don’t **fall** on your way to the tram.

Drawing the Transformer Network from Scratch (Part 1): https://towardsdatascience.com/drawing-the-transformer-network-from-scratch-part-1-9269ed9a2c5e

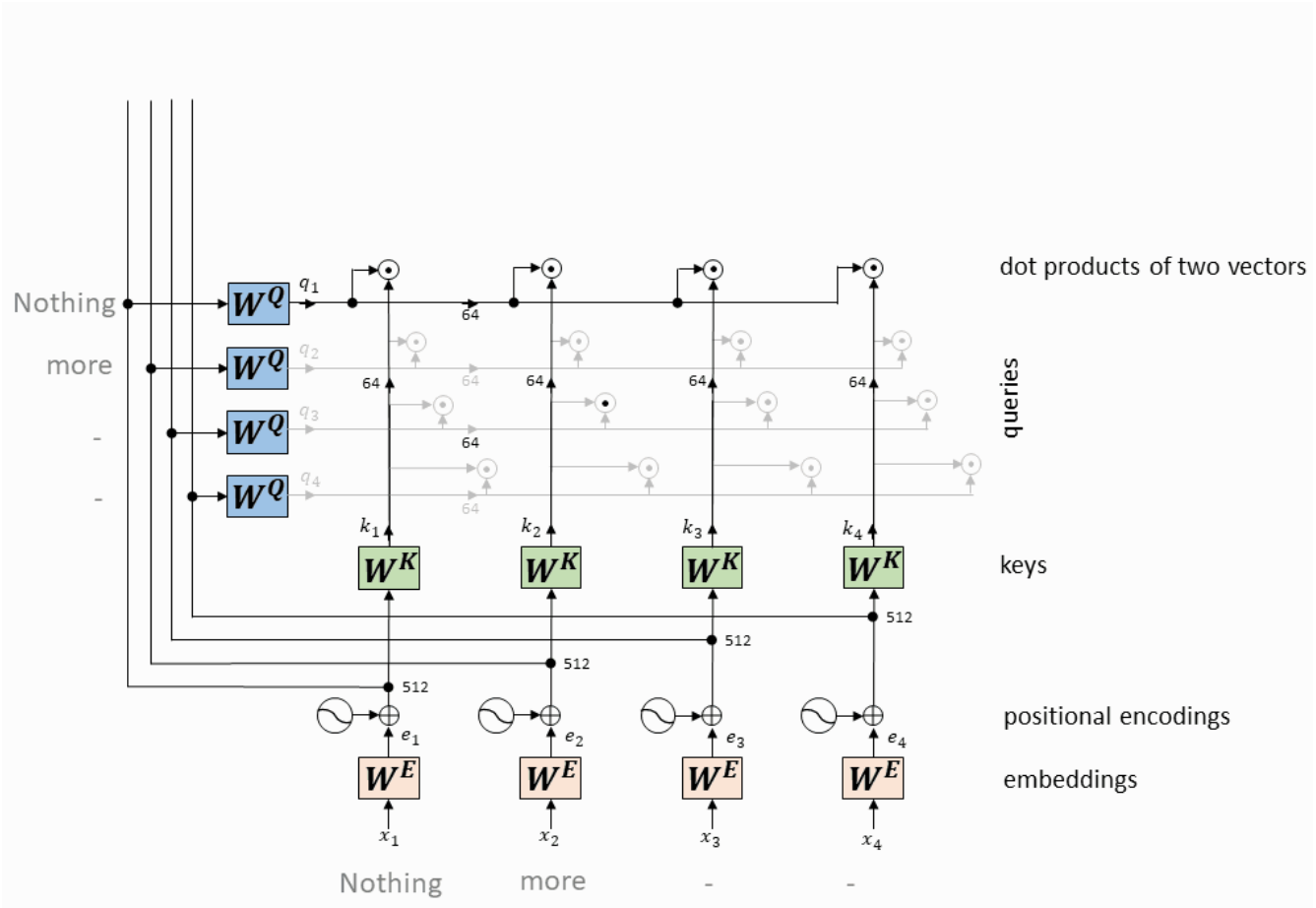
Shorter Sentences

- The length of the input sequence is supposed to be fixed in length
  - Basically, it is the length of the longest sentence in training dataset.
- Hence, a parameter defines the maximum length of a sequence that the Transformer can accept.
  - Sequences that are greater in length are just truncated.
  - Shorter sequences are padded with zeros.
  - However, padded words are not supposed to contribute to the self-attention calculation.
    - This is avoided by masking the corresponding words (setting them to -inf) before the softmax step in the self-attention calculation.
    - This in fact sets their weight factors to zero.

$$a_{t,t'} = \frac{\exp(e_{t,t'})}{\sum_{t'=1}^{T} \exp(e_{t,t'})}$$

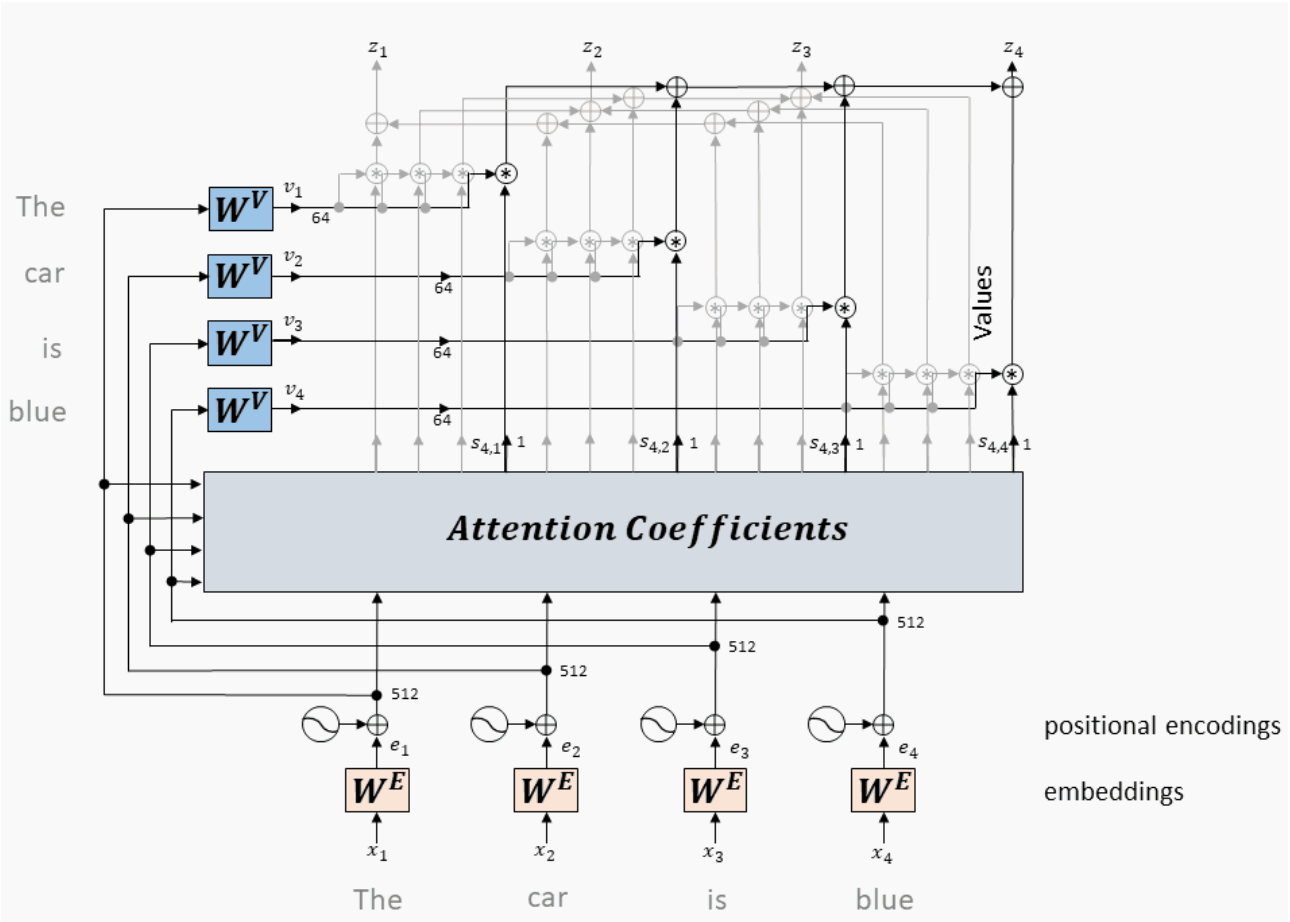
Drawing the Transformer Network from Scratch (Part 1): https://towardsdatascience.com/drawing-the-transformer-network-from-scratch-part-1-9269ed9a2c5e

Masking out the words-to-be-truncated



Drawing the Transformer Network from Scratch (Part 1): https://towardsdatascience.com/drawing-the-transformer-network-from-scratch-part-1-9269ed9a2c5e

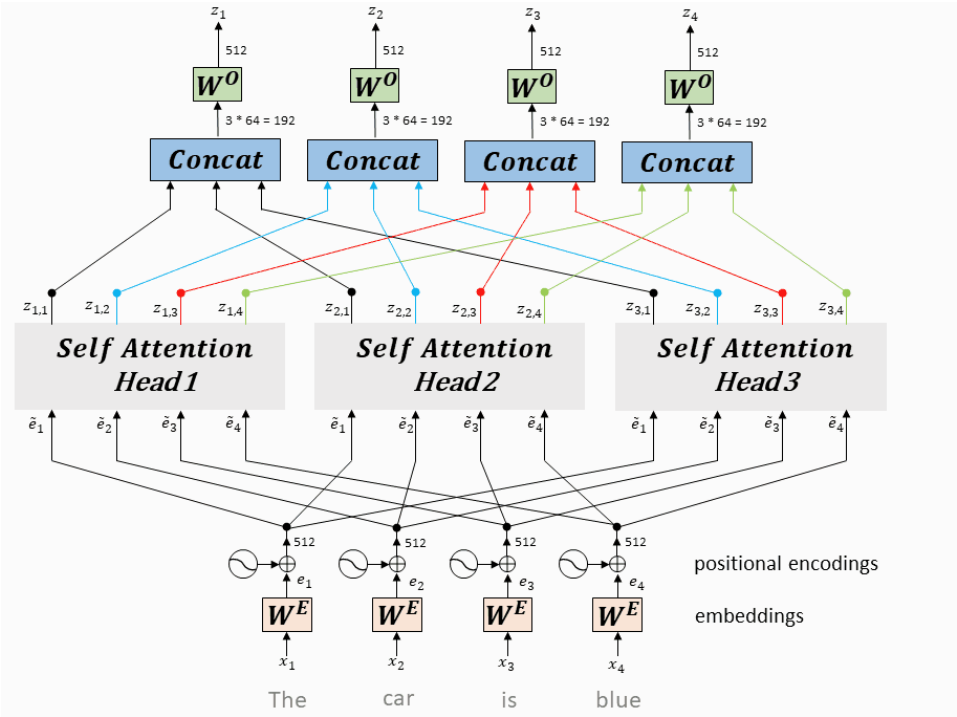
Multi-Head Self-Attention with 3 heads, original paper uses 8 heads



Drawing the Transformer Network from Scratch (Part 1): https://towardsdatascience.com/drawing-the-transformer-network-from-scratch-part-1-9269ed9a2c5e

Add and Normalize

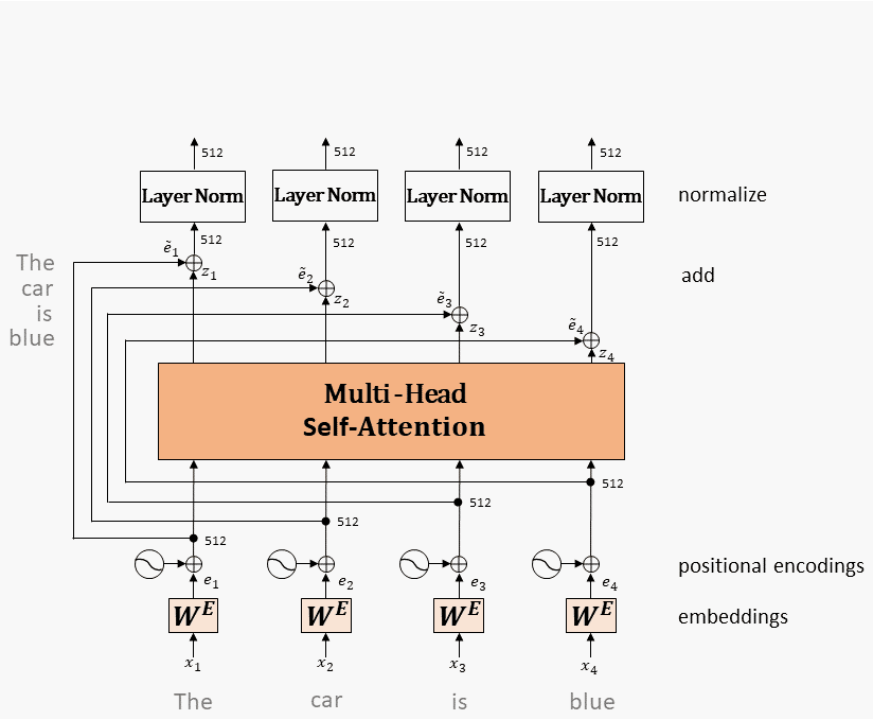
- The multi-head self-attention mechanism is the first sub-module of the encoder.
- It has a residual connection around it, and is followed by a **layer-normalization step**.
- Layer-normalization just **subtracts the mean of each vector and divides by its standard deviation**.



Drawing the Transformer Network from Scratch (Part 1): https://towardsdatascience.com/drawing-the-transformer-network-from-scratch-part-1-9269ed9a2c5e

Feed forward

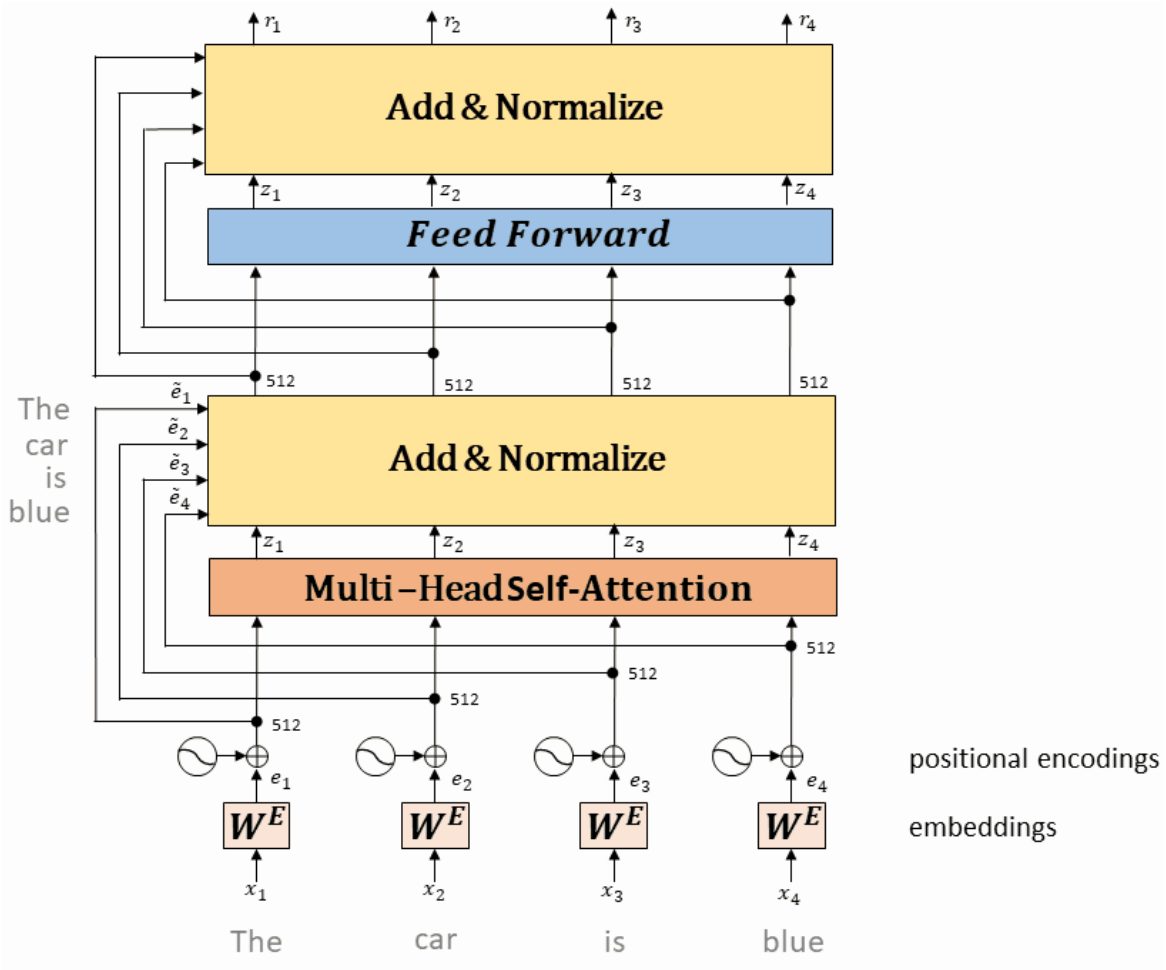
- The outputs of the self-attention layer are fed to a fully connected feed-forward network.
- This consists of two linear transformations with a ReLU activation in between.
  - The dimensionality of input and output is 512, and the inner-layer has dimensionality 2048.
  - The exact same feed-forward network is independently applied to each position, i.e. for each word in the input sequence.
- Next, we again employ a residual connection around the fully connected feed-forward layer, followed by layer normalization.



Drawing the Transformer Network from Scratch (Part 1): https://towardsdatascience.com/drawing-the-transformer-network-from-scratch-part-1-9269ed9a2c5e

Stack of Encoders

- The entire encoding component is a stack of six encoders. The encoders are all identical in structure, yet they do not share weights.



Drawing the Transformer Network from Scratch (Part 1): https://towardsdatascience.com/drawing-the-transformer-network-from-scratch-part-1-9269ed9a2c5e

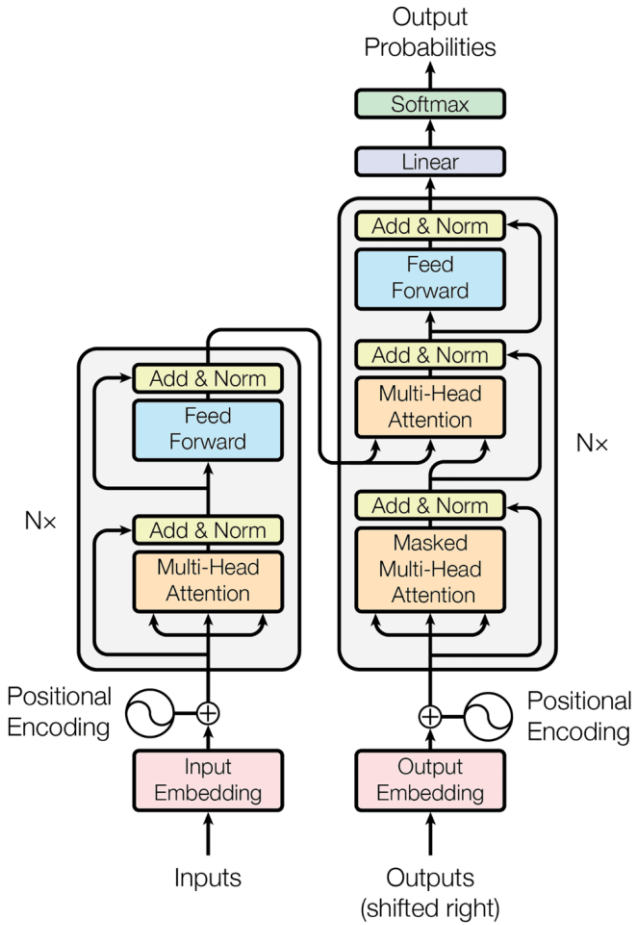
The whole process with Multiple Heads

Neural machine translation with a Transformer and Keras: https://www.tensorflow.org/text/tutorials/transformer

Transformer

Transformer

- Proposed in 2017 (*Attention Is All You Need*)
- A very resilient architecture
  - Minimal changes even 6 years later
  - Has been applied to many domains

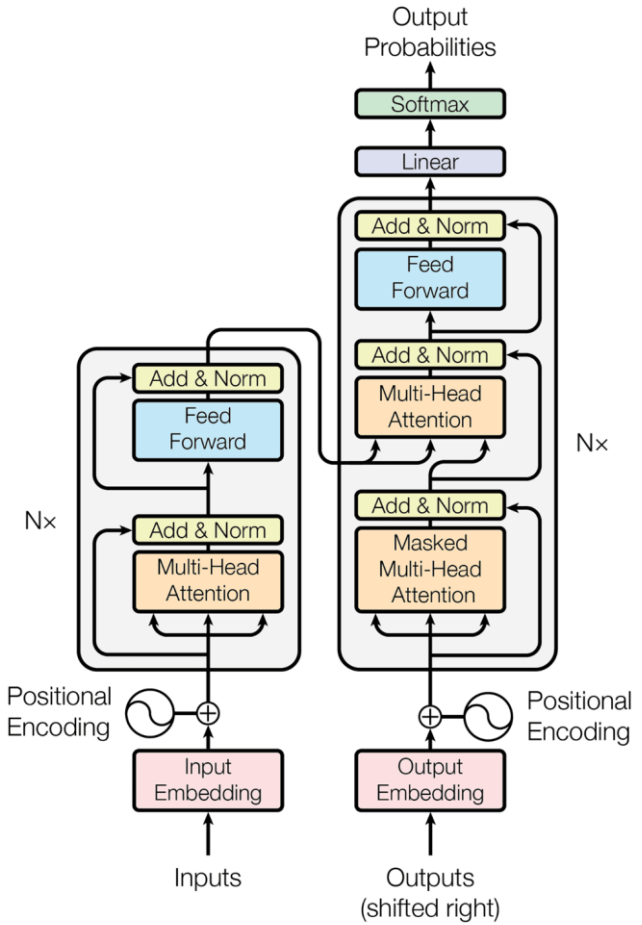


Source: Vaswani et al. *Attention Is All You Need*, https://arxiv.org/abs/1706.03762

Transformer

- Can be divided into an Encoder and a Decoder

- They both share the core pieces
  1. Self-Attention
  2. Multi-Headed Attention
  3. Residual Connections
  4. Layer Normalization
  5. Nonlinearities



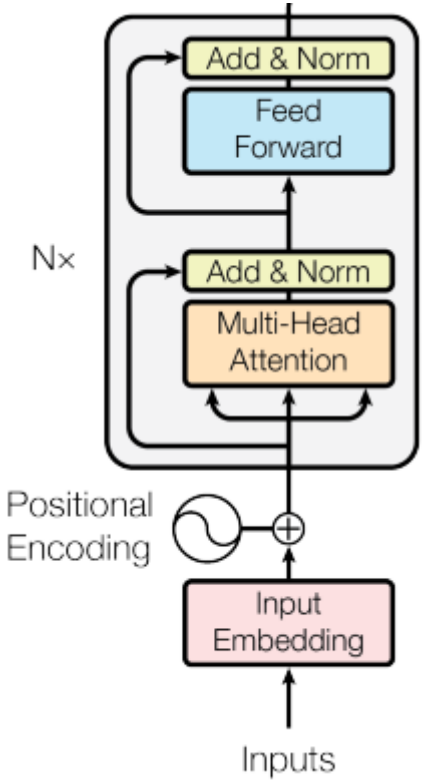
Source: Vaswani et al. *Attention Is All You Need*, https://arxiv.org/abs/1706.03762

Encoder

Encoder

- Components:
  1. “Multi-Headed Attention”
  2. “Add & Norm”
  3. Feed Forward Network

- Produces rich contextualized Embeddings
  - Can be used for tasks like Sentence Classification
  - or passed to the Decoder to generate more tokens



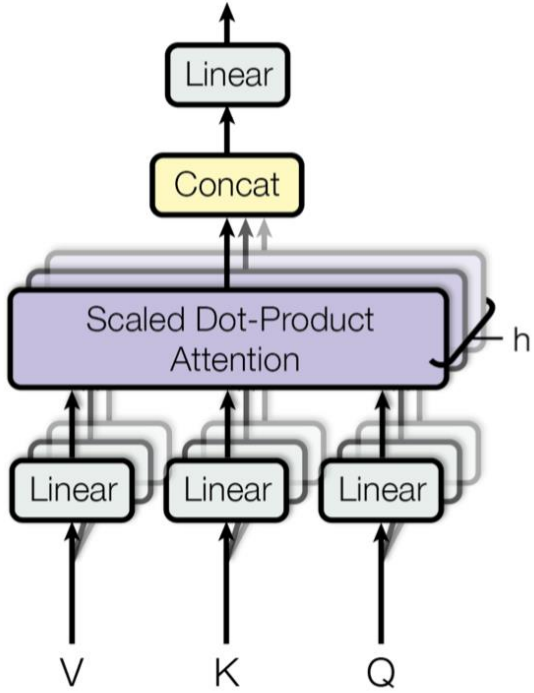
Source: Vaswani et al. *Attention Is All You Need*, https://arxiv.org/abs/1706.03762

Encoder: Multi-Headed Attention

Carry out the Attention operation many times in parallel

- Split input into smaller chunks
- Compute Attention over each subspace in parallel
- Independent Attention outputs are concatenated
- Outputs undergo projection into fixed dimensions

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O$$
$$\text{where head}_\text{i} = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$



Source: Vaswani et al. *Attention Is All You Need*, https://arxiv.org/abs/1706.03762

Encoder: Multi-Headed Attention

- Performing the same Attention procedure multiple times can lead to *much richer* representations of the same sentence being learned
  - If each head has a different set of Query vectors, having multiple heads can lead to asking multiple questions about the same token
  - If each head has a different set of Value vectors, having multiple heads can lead to tokens sharing different pieces of information with other tokens

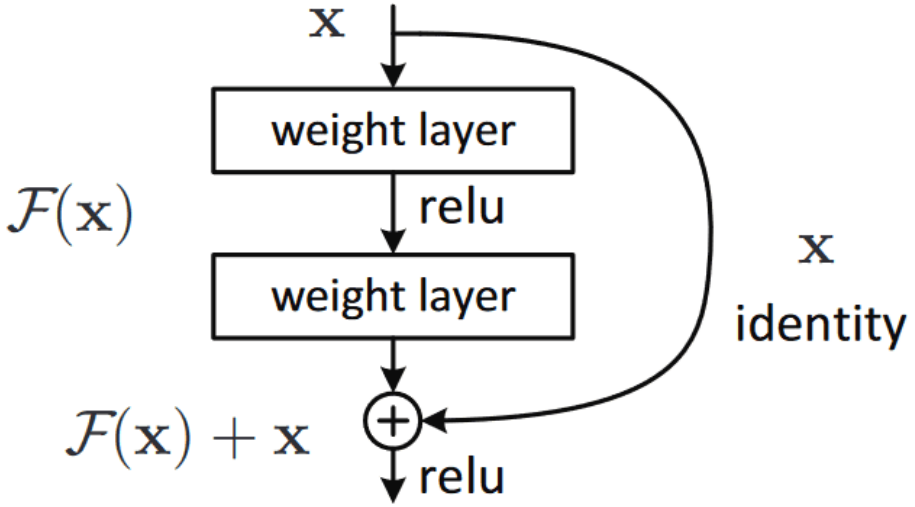
- When having multiple heads, we split the dimensionality of each of the vectors equally (e.g. dividing *d* by 8 if we have 8 heads)
  - This actually takes up roughly the same amount of time, since these heads are all computed upon in parallel

Source: Vaswani et al. *Attention Is All You Need*, https://arxiv.org/abs/1706.03762

Encoder: Skip Connections

- "Add" refers to Skip (Residual) Connections (a very common trick in Deep Learning!)
  - These provide an alternative path for gradients to flow
  - Mitigate Vanishing Gradients

- Proposed all the way back in 2015
  - Used for Image Classification
  - E.g. ResNet (Residual Network)



Source: He et al. *Deep Residual Learning for Image Recognition*, https://arxiv.org/abs/1512.03385

Encoder: Norm

- "Norm" refers to performing a *Layer Normalization*
  - In deep networks, there is often an issue of activations becoming too small or too large (which can lead to difficulties during training)
  - Activations should ideally follow a normal distribution to be well-behaved
  - Layer Normalization helps us normalize activations in the middle of a network

- The procedure is very similar to Standardization
  - Mean and SD tracked during training
  - The sum is carried out across units in a layer

$$\mu^l = \frac{1}{H}\sum_{i=1}^{H} a_i^l$$

$$\sigma^l = \sqrt{\frac{1}{H}\sum_{i=1}^{H}\left(a_i^l - \mu^l\right)^2}$$

Source: Layer Normalization - https://paperswithcode.com/method/layer-normalization

Encoder: Feed-Forward Network

- The “Feed-Forward Network” is really just two linear transformations/projections with ReLU activations in between
  - These intermediate layers allowed for intermediate processing of activations
  - Produce a convenient way of including non-linearities in between stacks
  - *“If Self-Attention and the multiple heads **aggregated** information from the tokens, then the Feed-forward layers enabled the tokens to **think** on the new information they got”*

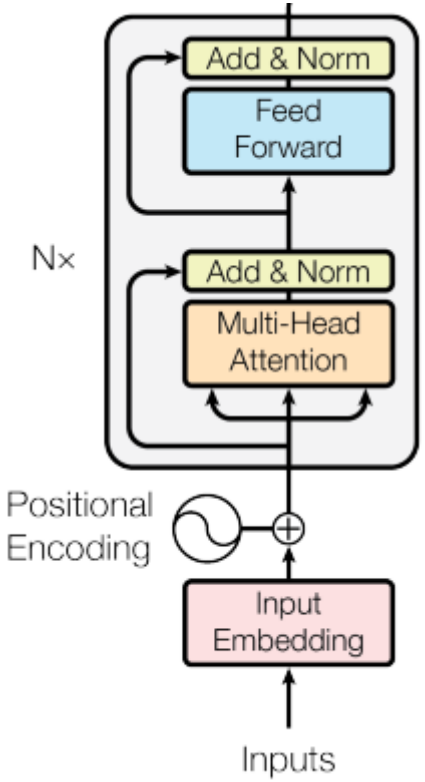
$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Source: Vaswani et al. *Attention Is All You Need*, https://arxiv.org/abs/1706.03762

Stacking the Encoder

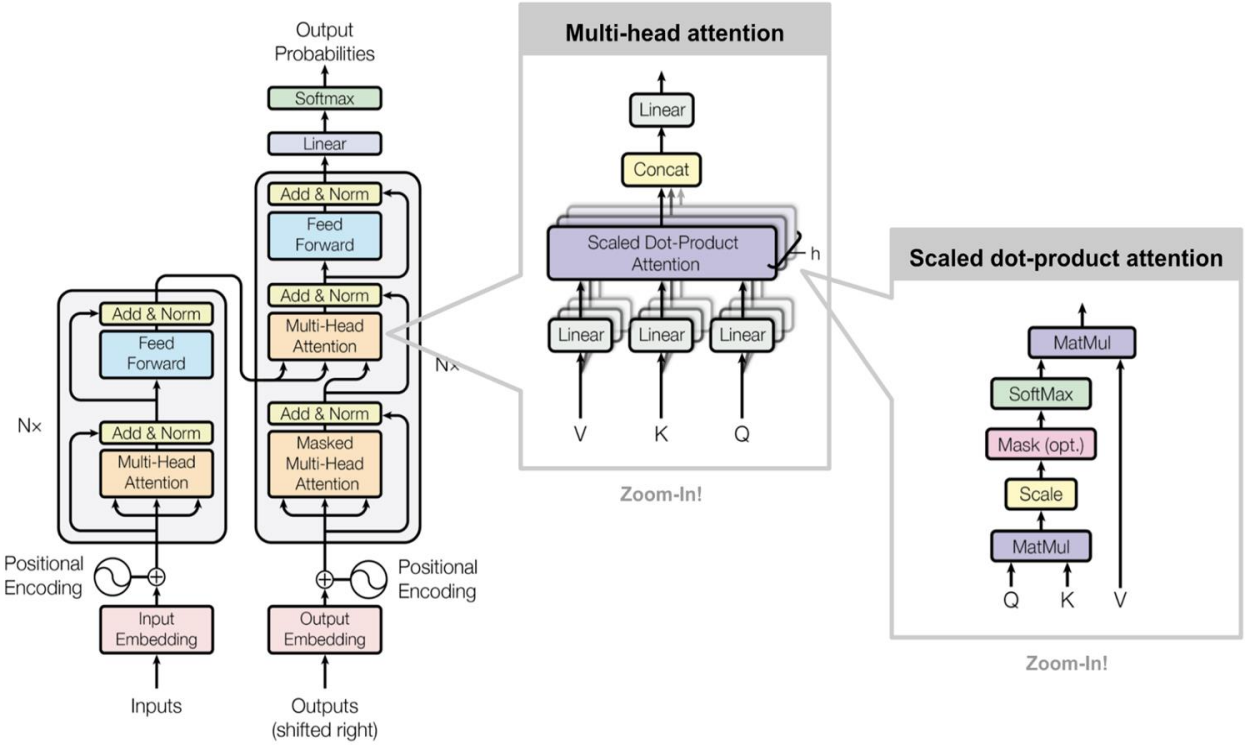
- This Encoder structure is stacked N times
  - The 2017 paper set N=6
  - Identical in structure but do not share weights

- The outputs of one stack is fed directly into the next one
  - Queries, Keys, and Values all over again
  - Refined representations with more layers
  - Can boost predictive power



Source: Vaswani et al. *Attention Is All You Need*, https://arxiv.org/abs/1706.03762

Zooming In

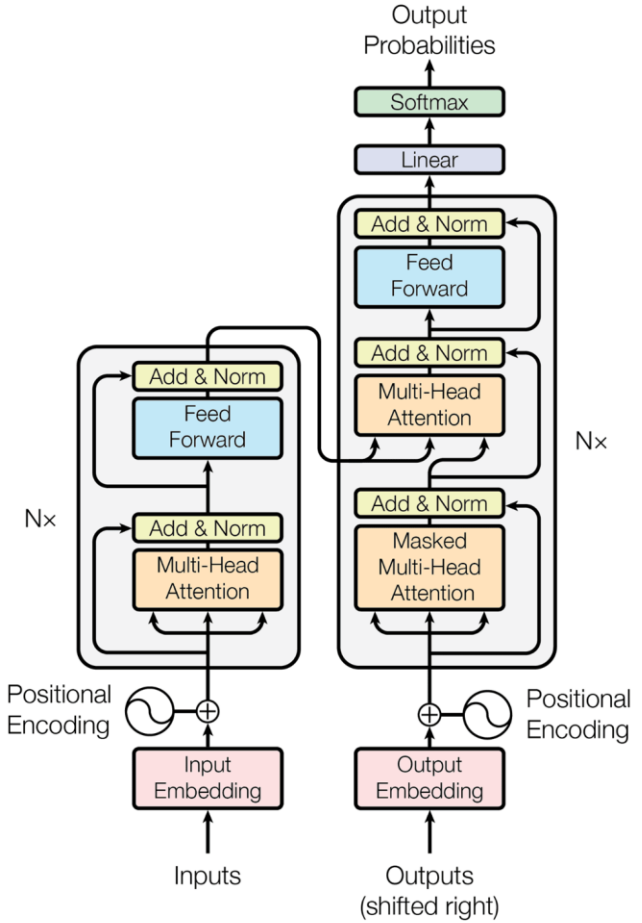


Source: Lilian Weng, The Transformer Family - https://lilianweng.github.io/posts/2023-01-27-the-transformer-family-v2/

Decoder

Decoder

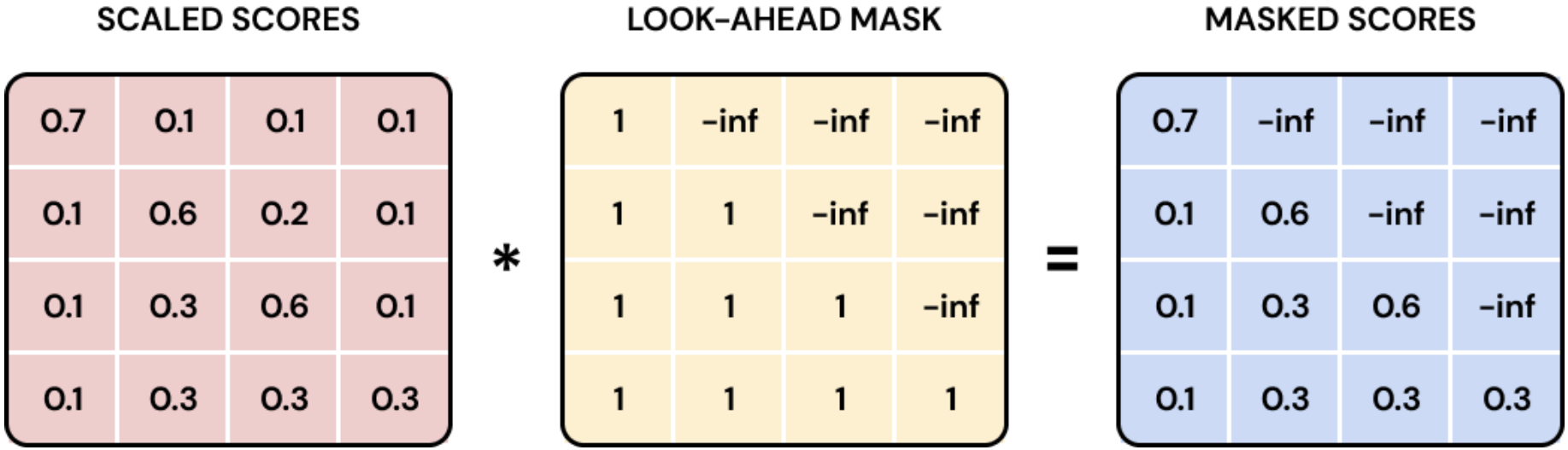
- The Encoder processes the input sequence
- The Keys and Values are drawn from the Encoder
- The Decoder has a masking element
  - Helps with the generation of tokens



Source: Vaswani et al. *Attention Is All You Need*, https://arxiv.org/abs/1706.03762

Decoder: First MHA

- Before the Softmax, we apply a Mask so that the module does not give any attention to *future tokens* (negative infinity corresponds to zero post-softmax)



Source: "Transformers: The Nuts and Bolts" - https://www.revistek.com/posts/transformer-architecture

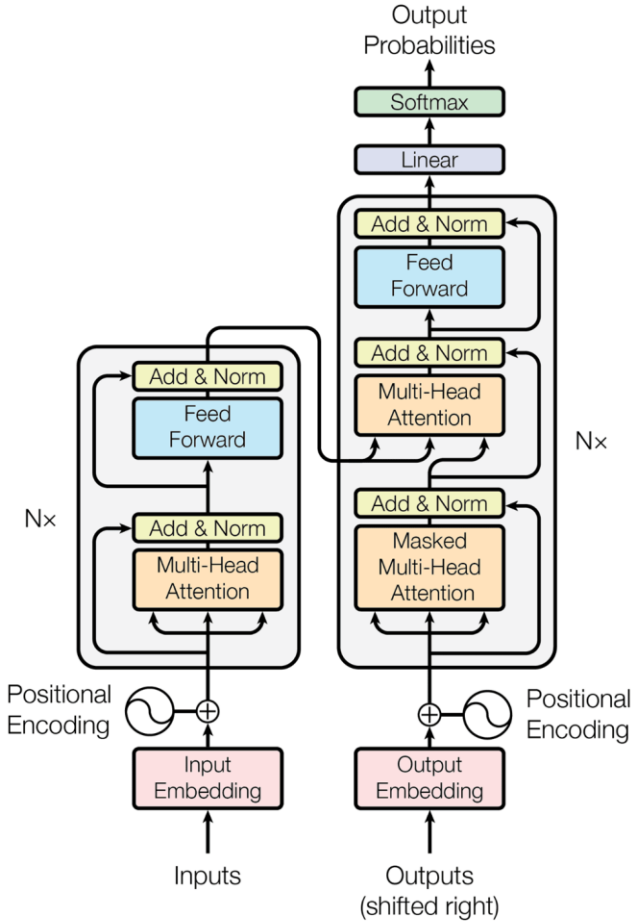
Decoder: Masking in Code

- A simple way to code this out would be to use a Lower Triangular Matrix
  - We could fill the zero-elements with negative infinities

```python
class Head(nn.Module):

    def __init__(self, head_size, emb_dim=32, block_size=8):
        super().__init__()

        self.key = nn.Linear(emb_dim, head_size, bias=False)
        self.query = nn.Linear(emb_dim, head_size, bias=False)
        self.value = nn.Linear(emb_dim, head_size, bias=False)

        self.register_buffer('tril', torch.tril(torch.ones(block_size, block_size)))

    def forward(self, x):

        B,T,C = x.shape # batch size, num tokens, embedding dim

        k = self.key(x)
        q = self.query(x)
        v = self.value(x)

        weights = q @ (k.transpose(-1, -2)) / (C**0.5)
        weights = weights.masked_fill(self.tril[:T, :T] == 0, float('-inf')) # decoder block
        weights = F.softmax(weights, dim=-1)

        out = weights @ v
        return out
```

Decoder: Second MHA

- There is no masking element for the second MHA
- Different sources of inputs
  - Key and Values specifically from Encoder
  - Queries generated from the first MHA

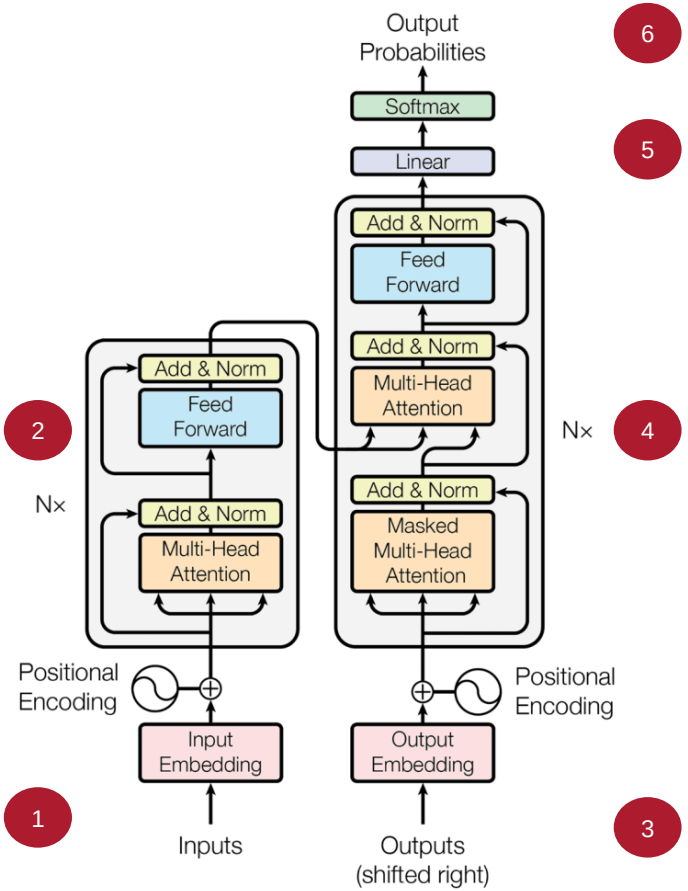


Source: Vaswani et al. *Attention Is All You Need*, https://arxiv.org/abs/1706.03762

Training Walkthrough

Training

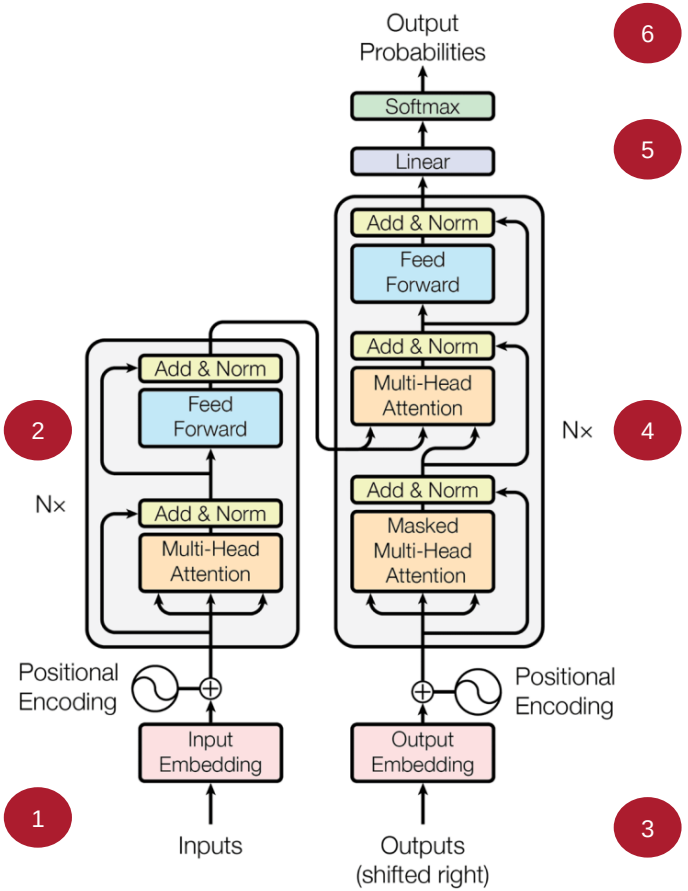
1. The inputs are converted to Embeddings, added to the Positional Encodings, and fed to the Encoder



Source: Vaswani et al. *Attention Is All You Need*, https://arxiv.org/abs/1706.03762

Training

2. The stack of Encoders processes the inputs and produces the encoded representations

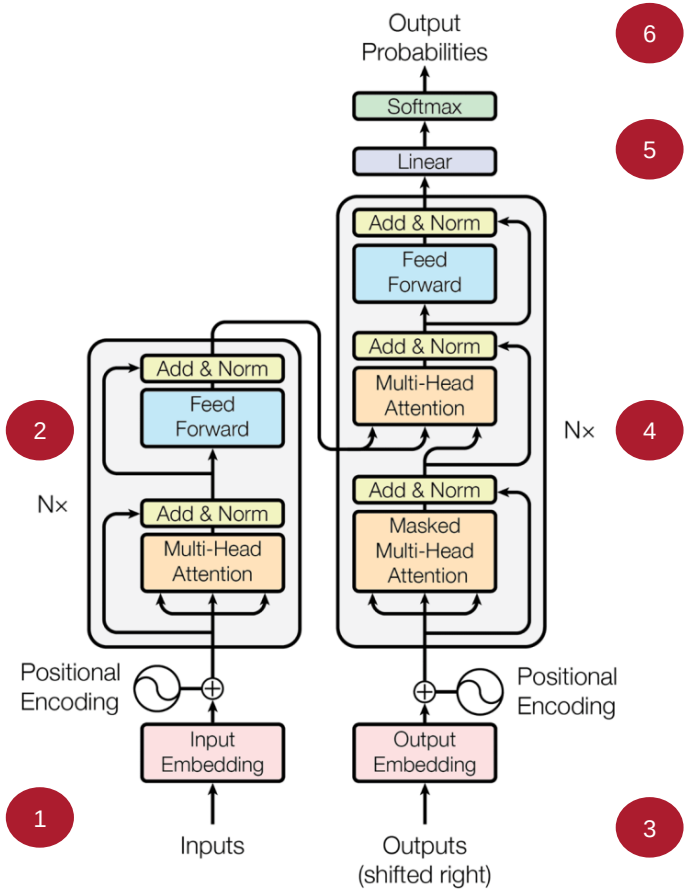


Source: Vaswani et al. *Attention Is All You Need*, https://arxiv.org/abs/1706.03762

Training

3. The **target** sequence is prepended with a start-of-sentence token, converted into Embeddings (with Position Encoding), and fed to the Decoder

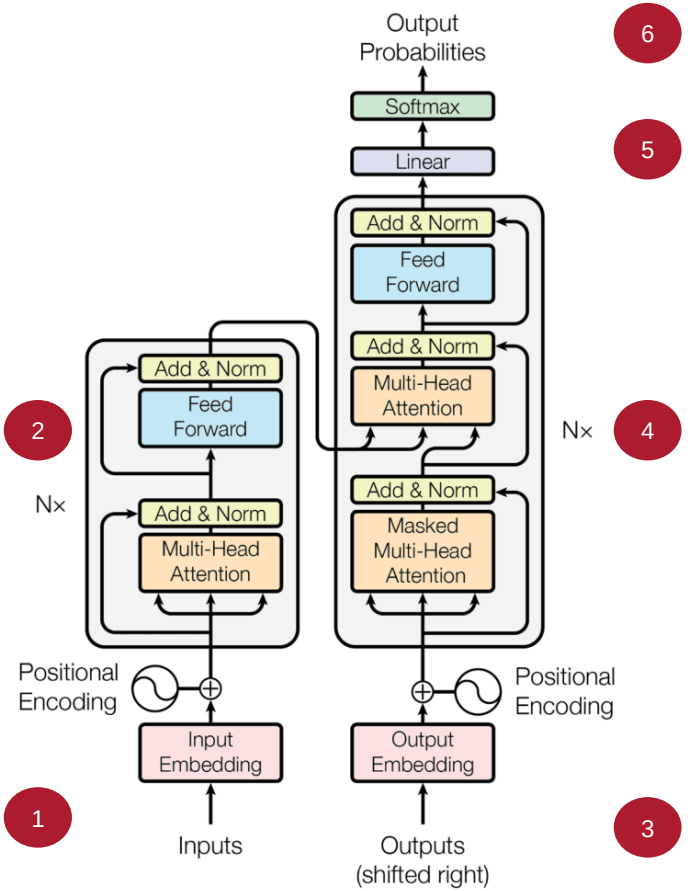
Note: During Training, we perform *Teacher Forcing* (feed in the ground truth tokens at each time step instead of what the model generates, to avoid accumulated errors)



Source: Vaswani et al. *Attention Is All You Need*, https://arxiv.org/abs/1706.03762

Training

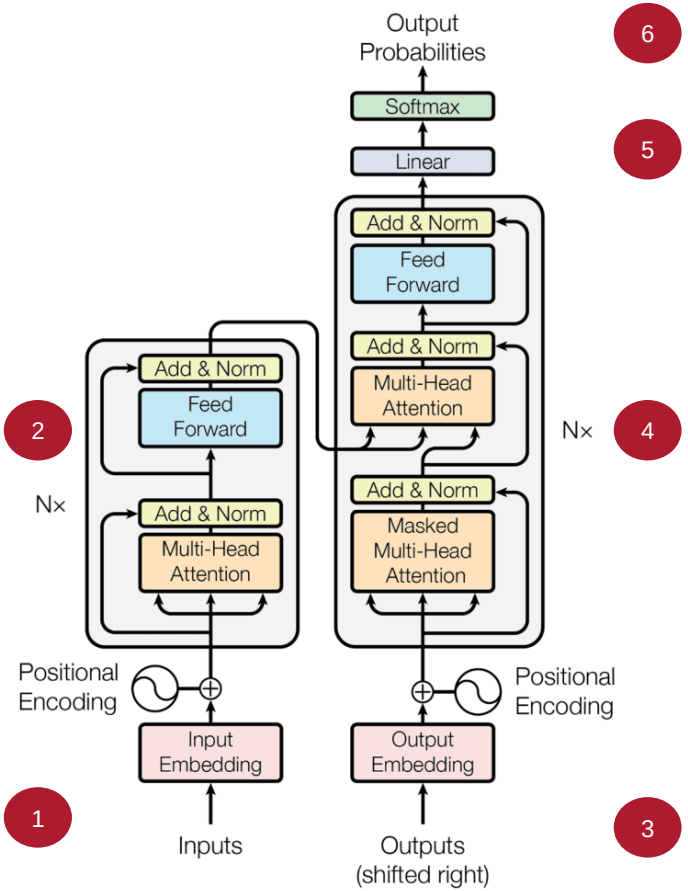
4. The stack of Decoders processes this, with the information coming from the Encoder, to produce an encoded representation of the target sequence



Source: Vaswani et al. *Attention Is All You Need*, https://arxiv.org/abs/1706.03762

Training

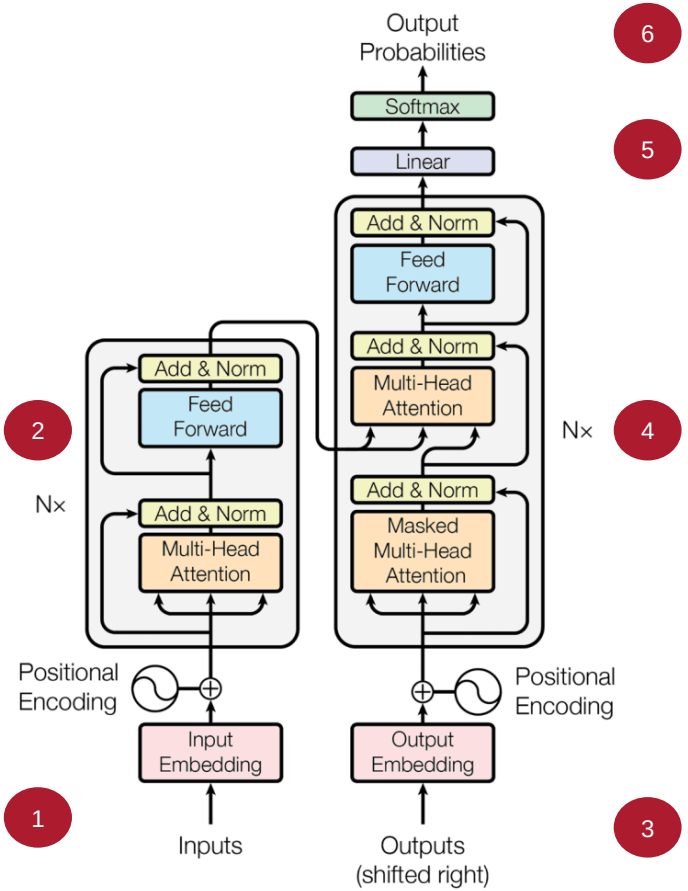
5. The final layer in the Decoder converts the outputs to word probabilities to allow for sampling the next token



Source: Vaswani et al. *Attention Is All You Need*, https://arxiv.org/abs/1706.03762

Training

6. The Loss Function (Cross Entropy) compares this output distribution with the target from the training data; the loss is used to generate gradients that train the entire Transformer

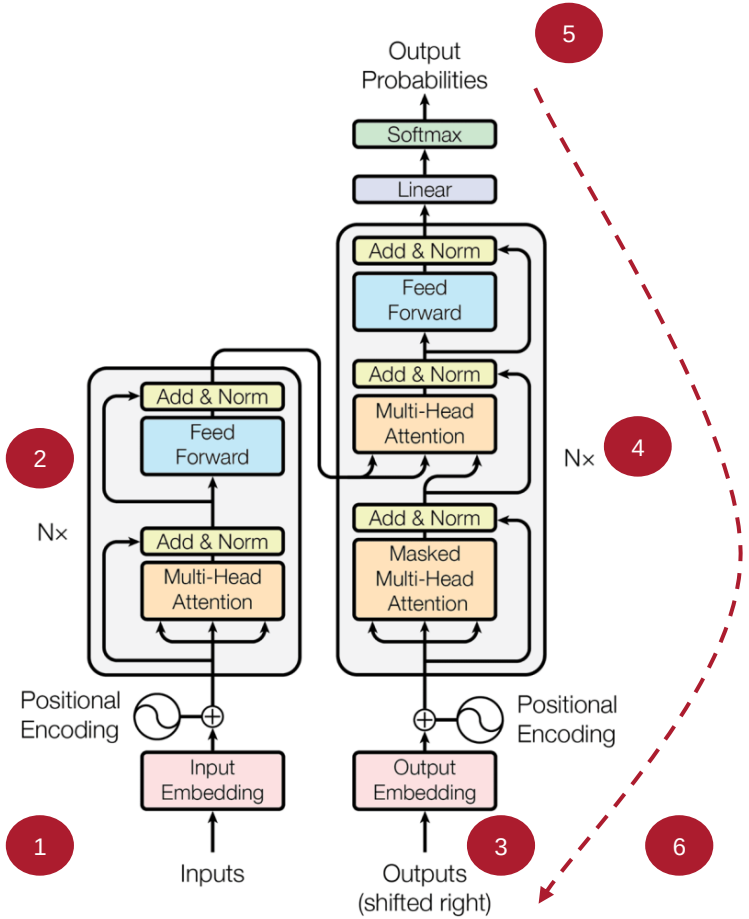


Source: Vaswani et al. *Attention Is All You Need*, https://arxiv.org/abs/1706.03762

Inference Walkthrough

Inference

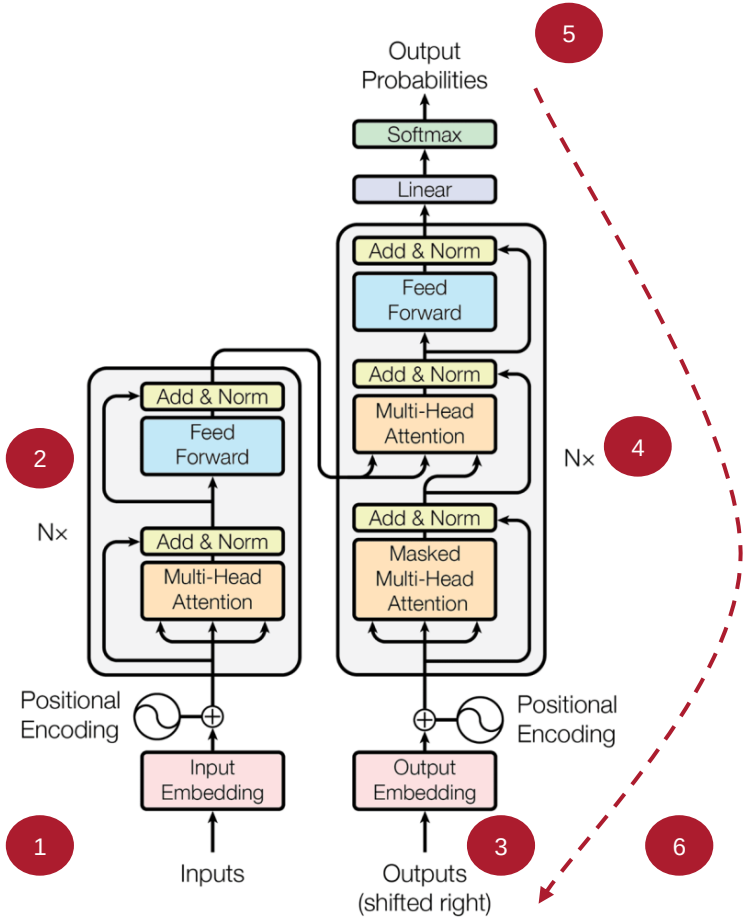
1. The inputs are converted to Embeddings, added to the Positional Encodings, and fed to the Encoder



Source: Vaswani et al. *Attention Is All You Need*, https://arxiv.org/abs/1706.03762

Inference

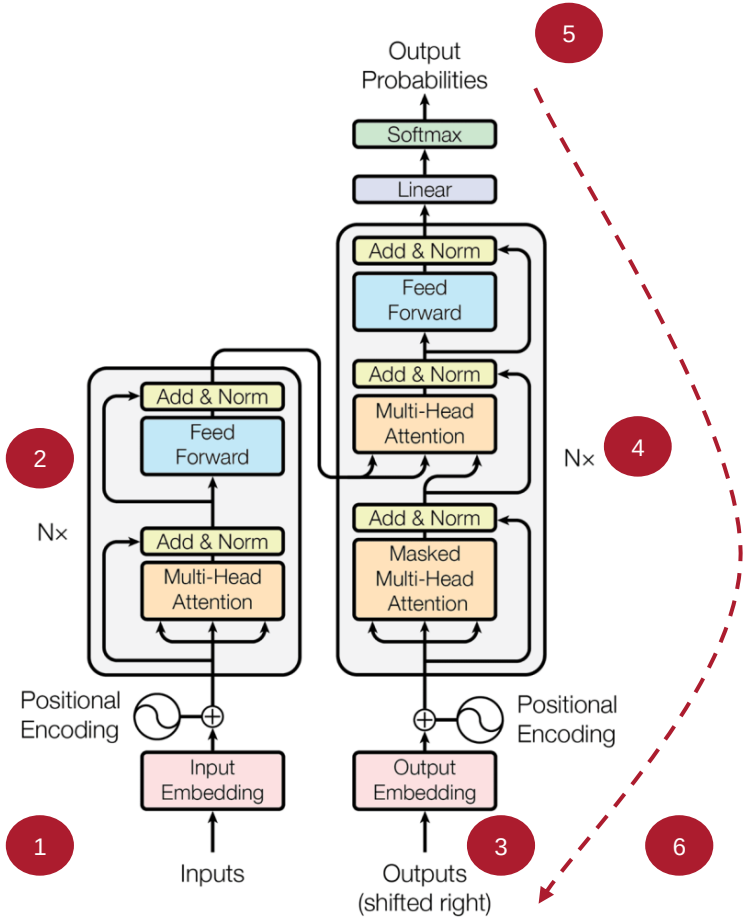
2. The stack of Encoders processes the inputs and produces the encoded representations



Source: Vaswani et al. *Attention Is All You Need*, https://arxiv.org/abs/1706.03762

Inference

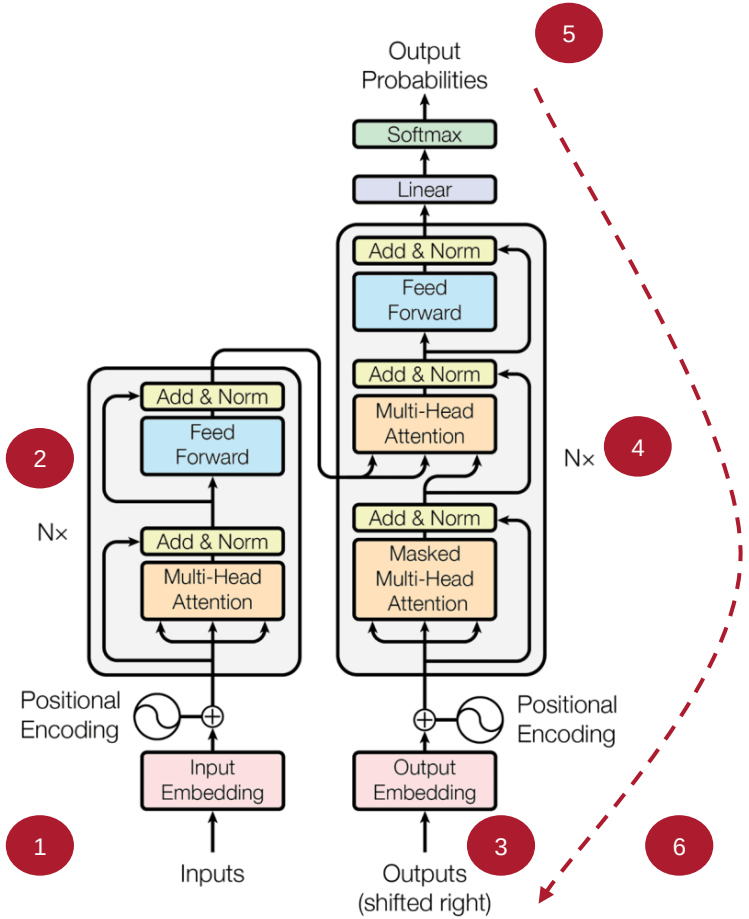
3. Since there are no targets now, we start with an empty sequence with only a start-of-sequence token, converted into Embeddings (added with Positional Encoding) and fed to the Decoder



Source: Vaswani et al. *Attention Is All You Need*, https://arxiv.org/abs/1706.03762

Inference

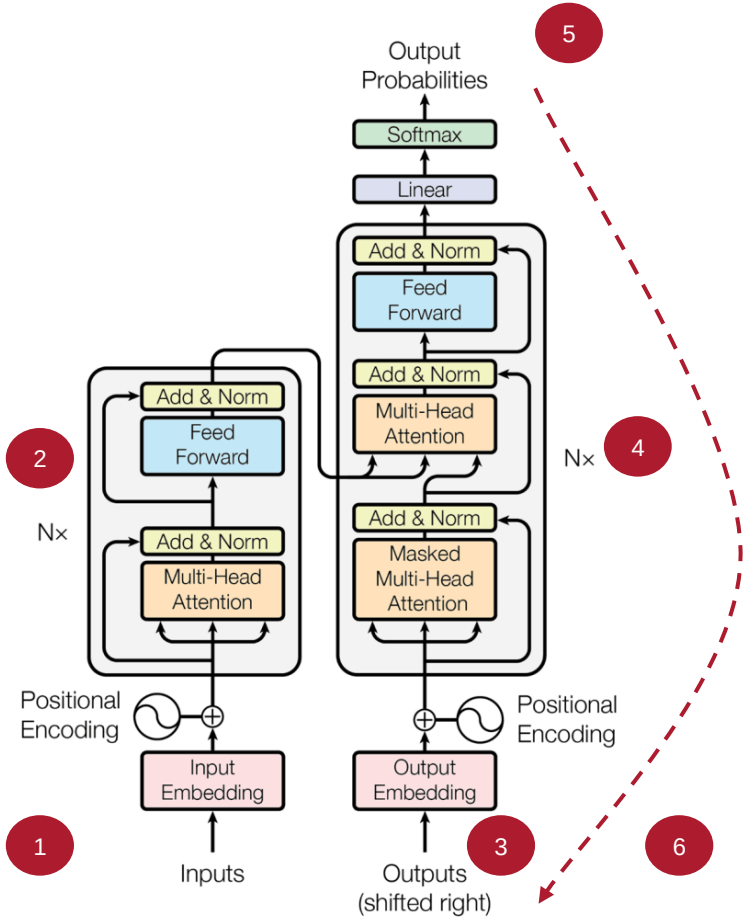
4. The stack of Decoders processes this, with the information coming from the Encoder, to produce an encoded representation of the target sequence



Source: Vaswani et al. *Attention Is All You Need*, https://arxiv.org/abs/1706.03762

Inference

5. The final layer in the Decoder converts the outputs to word probabilities to allow for sampling the next token

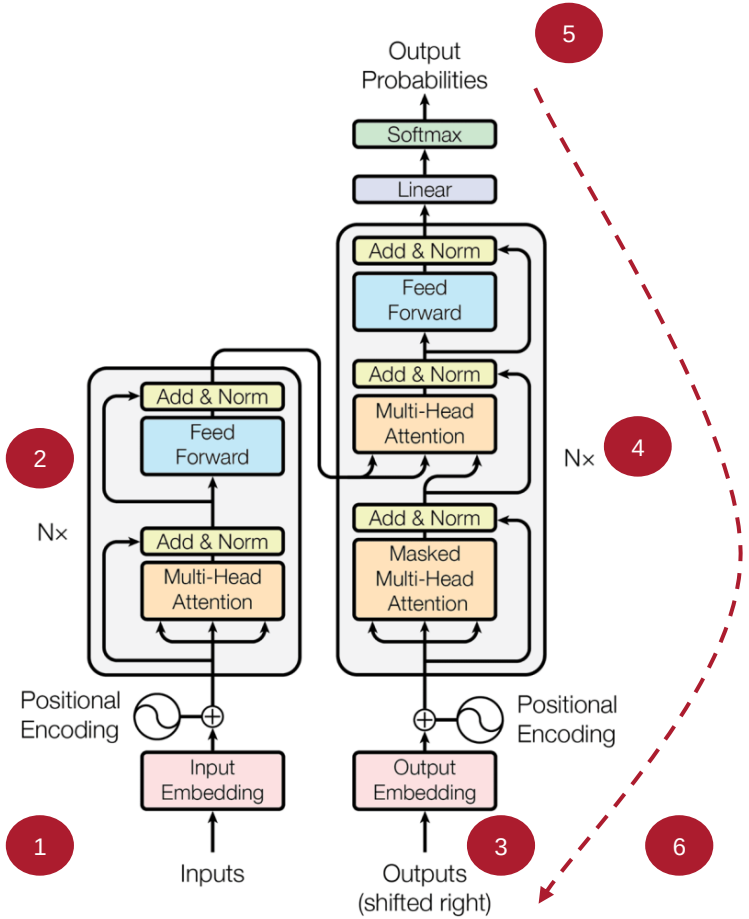


Source: Vaswani et al. *Attention Is All You Need*, https://arxiv.org/abs/1706.03762

Inference

6. The next token is sampled from this output, and appended to the Decoder's input sequence

Note: This process is repeated until some max threshold is reached on the number of tokens or an end-of-sequence token is sampled



Source: Vaswani et al. *Attention Is All You Need*, https://arxiv.org/abs/1706.03762

GPT (and friends)

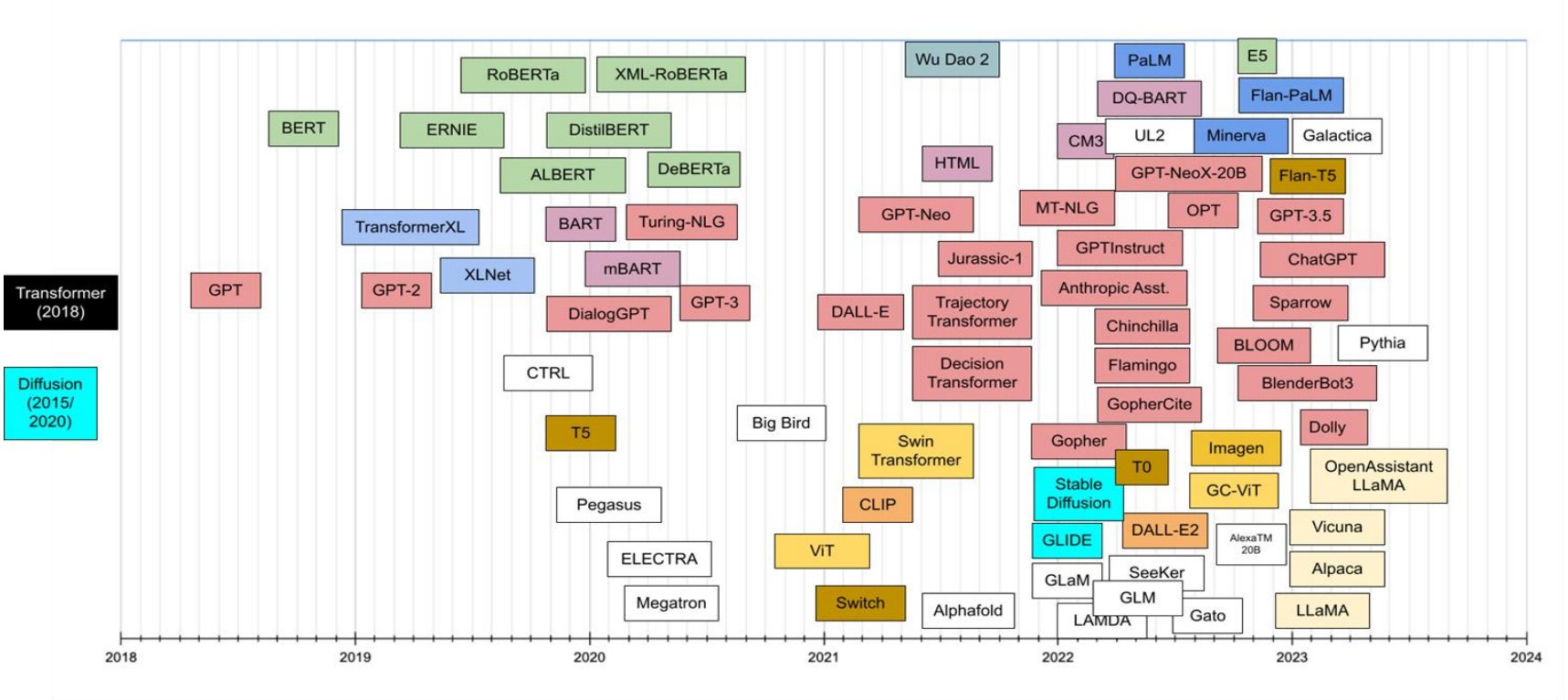
GPT-4

- GPT-4 is a member of the larger “GPT” family
  - “GPT”: “Generative Pretrained Transformer”
  - **Read**: “Generative Pretrained” (**not** “General Purpose”!)

- GPT-4 has a lot more bells and whistles as compared to a regular Transformer
  - *Multimodal in nature* (accepting text and image inputs)
  - *Reinforcement Learning with Human Feedback* (RLHF) during the training process for *alignment*
  - *Unknown* number of parameters

Source: OpenAI, GPT-4 - https://openai.com/research/gpt-4

State (as of January 2023)



Source: “Transformer Models: An Introduction and Catalog” - https://amatriain.net/blog/transformer-models-an-introduction-and-catalog-2d1e9039f376/

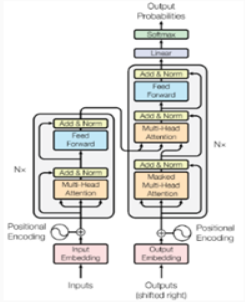
Resilience of the original Transformer

- Since 2017, the Transformer architecture has been applied to a plethora of tasks across a variety of domains with huge success
  - The core has remained majorly the same, with very few changes

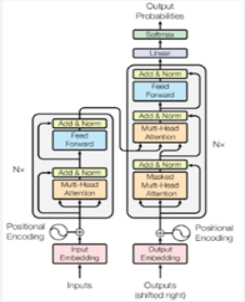
- Even the top players in the LLM space haven't deviated much from the original (like the GPT family, LLaMA etc.)

Source: Touvron et al. *LLaMA: Open and Efficient Foundation Language Models*, https://arxiv.org/abs/2302.13971

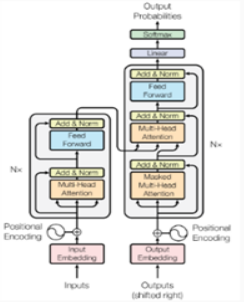
Computer Vision



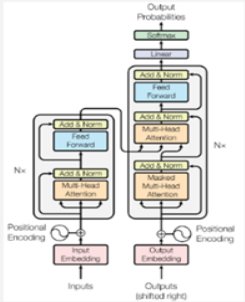
Natural Lang. Proc.



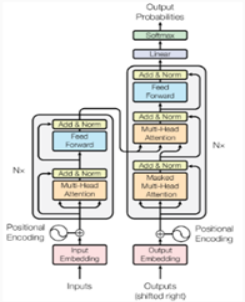
Reinf. Learning



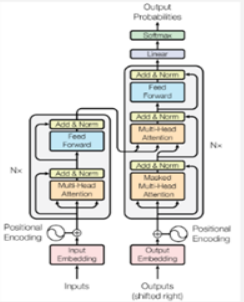
Speech



Translation



Graphs/Science



Source: Dr. Lucas Beyer - http://lucasb.eyer.be/transformer

Excerpt from the 2023 LLaMA paper

2.2 Architecture

Following recent work on large language models, our network is based on the transformer architecture (Vaswani et al., 2017). We leverage various improvements that were subsequently proposed, and used in different models such as PaLM. Here are the main difference with the original architecture, and where we were found the inspiration for this change (in bracket):

Pre-normalization [GPT3]. To improve the training stability, we normalize the input of each transformer sub-layer, instead of normalizing the output. We use the RMSNorm normalizing function, introduced by Zhang and Sennrich (2019).

SwiGLU activation function [PaLM]. We replace the ReLU non-linearity by the SwiGLU activation function, introduced by Shazeer (2020) to improve the performance. We use a dimension of $\frac{2}{3}4d$ instead of $4d$ as in PaLM.

Rotary Embeddings [GPTNeo]. We remove the absolute positional embeddings, and instead, add rotary positional embeddings (RoPE), introduced by Su et al. (2021), at each layer of the network.

The details of the hyper-parameters for our different models are given in Table 2.

Source: Touvron et al. *LLaMA: Open and Efficient Foundation Language Models*, https://arxiv.org/abs/2302.13971

Details of the GPT-3 variants

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or "GPT-3"	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

Source: Brown et al. *Language Models are Few-Shot Learners*, https://arxiv.org/abs/2005.14165

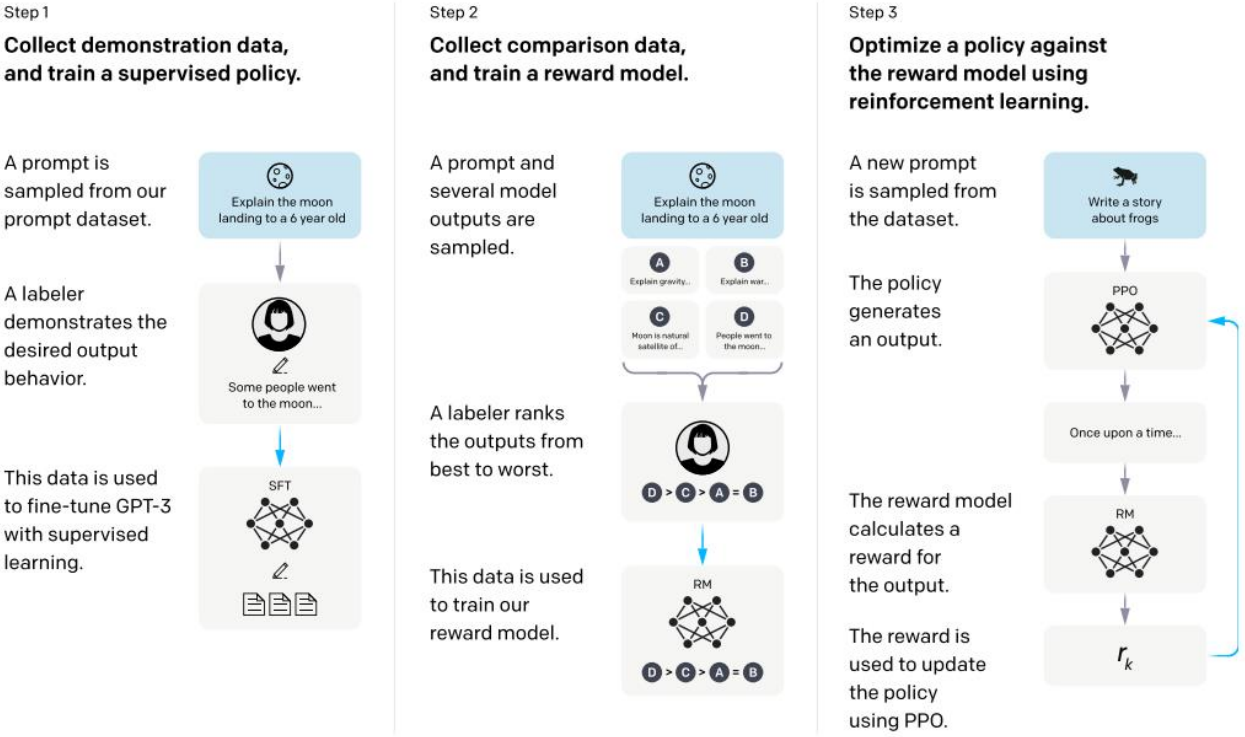
The Scale of these Models

Suppose you wanted to store *just* the weights of GPT-3 (not even considering doing a forward or backward pass, *just* to store the weights):

$$175\text{B} \cdot \frac{4\text{bytes}}{1024^2\text{bytes/GB}} > 660\text{GB}$$

Source: Brown et al. *Language Models are Few-Shot Learners*, https://arxiv.org/abs/2005.14165

Aligning Models (Supplementary)

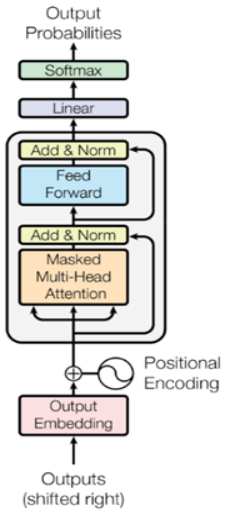


Source: Ouyang et al. *Training language models to follow instructions with human feedback*, https://arxiv.org/abs/2203.02155

Types of Transformer Models

Decoder-only GPT

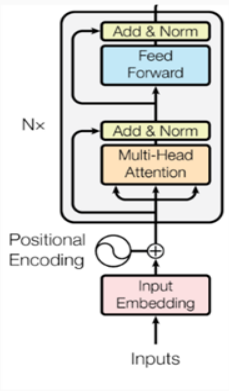
[sat_]



[START] [The_] [cat_]

Encoder-only BERT

[*] [*] [sat_] [*] [the_] [*]



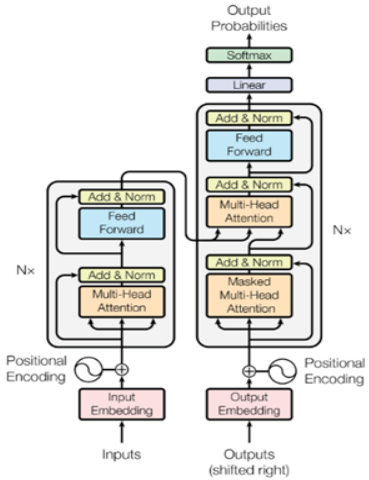
[The_] [cat_] [MASK] [on_] [MASK] [mat_]

Enc-Dec T5

Das ist gut.

A storm in Attala caused 6 victims.

This is not toxic.



Translate EN-DE: This is good.

Summarize: state authorities dispatched…

Is this toxic: You look beautiful today!

Source: Dr. Lucas Beyer - http://lucasb.eyer.be/transformer